Politécnico Castelo Branco
Escola Superior de Tecnologia

Automação e Inteligência em Redes de Comunicação de Dados: integração da metodologia Network DevOps (NetDevOps), numa arquitetura de redes híbrida

Projeto 2

Rafael Lourenço Cruz Nº 20230326

Rafael Ginga Victorino Nº 20231475

Orientador

Professor Doutor Vasco Nuno da Gama de Jesus Soares

Coorientador

Professor Doutor Alexandre José Pereira Duro da Fonte

Trabalho de Projeto apresentado à Escola Superior de Tecnologia do Instituto Politécnico de Castelo Branco para cumprimento dos requisitos necessários à obtenção do grau de Licenciado em Engenharia Informática, realizado sob a orientação científica do Professor Doutor Vasco Nuno da Gama de Jesus Soares e do Professor Doutor Alexandre José Pereira Duro da Fonte do Instituto Politécnico de Castelo Branco.

Junho de 2026

Composição do júri

Presidente do júri

Doutor, Osvaldo Arede Dos Santos

Professor Adjunto do Instituto Politécnico de Castelo Branco

oas@ipcb.pt

Arguente

Doutor, Carlos Manuel De Oliveira Alves

Professor Adjunto do Instituto Politécnico de Castelo Branco

cmoa@ipcb.pt

Orientador

Doutor, Vasco Nuno Da Gama De Jesus Soares

Professor Adjunto do Instituto Politécnico de Castelo Branco

vasco.g.soares@ipcb.pt

Agradecimentos

A concretização deste trabalho de projeto não teria sido possível sem o apoio, orientação e incentivo de um conjunto de pessoas e entidades, às quais expressamos a nossa mais profunda gratidão.

Em primeiro lugar, dirigimos um agradecimento muito especial aos nossos orientadores, Professor Doutor Vasco Nuno da Gama de Jesus Soares e Professor Doutor Alexandre José Pereira Duro da Fonte. Agradecemos não só a partilha de conhecimento, o rigor exigido e o tempo despendido, mas também a paciência e a orientação constante, que foram fundamentais para o nosso crescimento académico e profissional ao longo deste percurso.

De igual modo, expressamos o nosso sincero agradecimento à ARTE (Agência para a Reforma Tecnológica do Estado) e, de forma muito particular, ao Eng.º Nuno Batuca. Obrigado pela confiança depositada na nossa equipa, por toda a disponibilidade em partilhar a realidade operacional da infraestrutura e por nos terem proporcionado o desafio de desenvolver este projeto num contexto real e crítico. O acompanhamento próximo e as vossas indicações técnicas foram determinantes para o sucesso das soluções que implementámos.

Uma última palavra de agradecimento às nossas famílias e amigos, pelo apoio incondicional e pela compreensão durante as muitas horas dedicadas a este projeto.

A todos, o nosso muito obrigado.

Resumo

Este relatório descreve a conceção, a implementação e a validação de um protótipo de automação de rede em ciclo fechado para recuperação automática de falhas em infraestruturas híbridas. Desenvolvido em articulação com a ARTE — Agência para a Reforma Tecnológica do Estado, I.P. —, o trabalho estende as ferramentas de diagnóstico passivo construídas no Projeto I, passando da deteção da avaria para a sua correção autónoma. Adotando o paradigma **NetDevOps** e a abordagem de **Infraestrutura como Código (IaC)**, o sistema procura reforçar a resiliência dos canais de comunicação da ARTE, onde túneis VPN IPsec (*overlay*) operam sobre circuitos de transporte MPLS (*underlay*). A plataforma n8n assume o papel de motor de orquestração, articulando o PRTG, o NetBox — que serve de *Single Source of Truth* (SSoT) — e uma base de dados **MongoDB** destinada à auditoria e à conformidade.

O motor de execução foi desenvolvido em duas fases. Na primeira, estruturou-se um mecanismo de contingência (*SSH Fallback*) assente em *scripts* Python com a biblioteca Netmiko, para contornar as restrições de escrita impostas pela licença de avaliação na API do FortiOS. A atribuição de licenciamento comercial viabilizou, na segunda fase, a substituição deste mecanismo por chamadas à REST API sobre a base de configuração (CMDB) da firewall. Esta evolução eliminou a dependência de sessões interativas de linha de comandos, substituiu as credenciais em texto claro por autenticação com *token* restrito por *trusted hosts*, e alargou a auditoria às **regras de filtragem** — uma classe de avaria que o diagnóstico do Projeto I não classifica.

A arquitetura incorpora ainda uma sonda de conectividade por ICMP que suspende a remediação quando o equipamento remoto não responde, evitando tentativas de escrita inúteis e *timeouts* na rede de gestão. A componente analítica recorre ao **Metabase** para a leitura dos indicadores de resolução, sobre 24 incidentes registados e guardados no MongoDB, distribuídos por duas coleções correspondentes às duas fases. Os ensaios de Engenharia do Caos conduzidos em ambiente VMware Workstation Pro documentam ciclos completos de deteção, correção e convergência entre 4 e 7 segundos, face aos 15 a 30 minutos exigidos pela intervenção manual. Os resultados demonstram a viabilidade da automação em ciclo fechado para a gestão de infraestruturas críticas de comunicações na administração pública, ainda que a dimensão da amostra e as características do ambiente simulado limitem a extrapolação direta para produção.

.

Palavras-chave

NetDevOps, Automação em Ciclo Fechado, Autorremediação, VPN IPsec, Orquestração de Redes.

Abstract

This report describes the design, implementation and validation of a closed-loop network automation prototype for the automatic recovery of failures in hybrid infrastructures. Developed in collaboration with ARTE — Agência para a Reforma Tecnológica do Estado, I.P. — the work extends the passive diagnostic tooling built in Projeto I, moving from fault detection to autonomous correction. Drawing on the **NetDevOps** paradigm and the **Infrastructure as Code (IaC)** approach, the system aims to strengthen the resilience of ARTE's communication channels, where IPsec VPN tunnels (overlay) run over MPLS transport circuits (underlay). The n8n platform acts as the orchestration engine, coordinating PRTG, NetBox — serving as the Single Source of Truth (SSoT) — and a **MongoDB** database for auditing and compliance purposes.

The execution engine was developed in two phases. In the first, a fallback mechanism (SSH Fallback) was built on Python scripts using the Netmiko library, in order to work around the write restrictions imposed by the evaluation licence on the FortiOS API. The subsequent award of a commercial licence allowed this mechanism to be replaced by native REST API calls against the firewall's configuration database (CMDB). This shift removed the dependency on interactive command-line sessions, replaced plaintext credentials with token-based authentication restricted by trusted hosts, and extended auditing to **firewall filtering rules** — a class of fault that the Projeto I diagnostic engine does not classify.

The architecture further incorporates an ICMP connectivity probe that suspends remediation whenever the remote device fails to respond, thereby avoiding futile write attempts and timeouts on the management network. The analytical component relies on **Metabase** to present resolution indicators across 24 incidents recorded and stored in MongoDB, split between two collections corresponding to the two phases. Chaos Engineering trials conducted in a VMware Workstation Pro environment document complete detection, correction and convergence cycles of between 4 and 7 seconds, against the 15 to 30 minutes required by manual intervention. The results demonstrate the viability of closed-loop automation for managing critical communications infrastructure in public administration, although the sample size and the characteristics of the simulated environment limit direct extrapolation to production.

.

Keywords

NetDevOps, Closed-Loop Automation, Self-Healing Networks, IPsec VPN, Network Orchestration.

Índice geral

1. Introdução e estado da arte 1
   1.1 Enquadramento institucional: a missão da ARTE e da sua equipa de comunicações 3
   1.2 Descrição do problema 4
   1.3 Redes autonómicas, ciclo fechado e redes baseadas em intenção 5
   1.4 Objetivo 6
   1.5 Plano de trabalhos 7
   1.6 Organização do documento 9
2. Arquitetura da stack e engenharia de execução 11
   2.1 Enquadramento e modelação teórica do MTTR 11
   2.2 Identificação das técnicas e ferramentas específicas 12
   2.3 Procedimento de implementação 15
   2.4 Metodologia de injeção de falhas 47
   2.5 Síntese da implementação 48
3. Resultados 49
   3.1 Análise quantitativa e impacto no MTTR 49
   3.2 Validação dos cenários de Chaos Engineering e análise de logs 52
   3.3 Visibilidade analítica e indicadores BI (Metabase) 67
4. Conclusão 77
   4.1 Principais conclusões 77
   4.2 Análise crítica e limitações do ambiente simulado 78
   4.3 Trabalho futuro 80

Índice de figuras

Figura 1 - Aplicação da metodologia PDCA (Plan-Do-Check-Act) no desenvolvimento do projeto de automação (Fonte: Elaboração própria). 8

Figura 2 - Comandos de criação e ativação do ambiente virtual Python para a biblioteca Netmiko. 14

Figura 3 - Arquitetura tecnológica macro e fluxo sequencial de interação e controlo entre os componentes da arquitetura NetDevOps em ciclo fechado (Closed-Loop). 15

Figura 4 - Topologia lógica da infraestrutura híbrida da ARTE: mapeamento e abstração da camada de transporte físico (Underlay MPLS) e da camada de conectividade encriptada (Overlay IPsec). 16

Figura 5 - Fluxograma inicial da lógica reativa da arquitetura em ciclo fechado, baseada no reinício unificado de chaves criptográficas e desprovida de triagem física. 19

Figura 6 - Fluxograma finalizado da arquitetura autónoma com triagem preventiva de hardware (Pre-Check ICMP) e ramificações cirúrgicas de auto-remediação configuracional. 20

Figura 7 - Fluxograma da cascata de exclusão condicional para triagem profunda da VPN. 22

Figura 8 - Script Python (remediate_config.py) desenvolvido com a biblioteca Netmiko para a remediação configuracional ativa e retificação idempotente de desvios de gateway. 24

Figura 9 - Script Python (remediate_phase2.py) baseado na biblioteca Netmiko para a retificação ativa, tradução sintática de máscaras de rede e remediação configuracional de seletores de tráfego desconfigurados na Phase 2. 26

Figura 10 - Script Python (remediate_tunnel.py) baseado na biblioteca Netmiko, para a limpeza das associações de segurança (SAs) e renegociação do túnel IPsec via CLI. 27

Figura 11 - Canvas completo do fluxo de automação e orquestração em ciclo fechado (Closed-Loop) atualizado na plataforma n8n, evidenciando a árvore de triagem sequencial e o triplo ramo de atuação IaC (Restart, RGWY e Phase 2). 29

Figura 12 - Diagrama de blocos da macro-arquitetura e topologia da infraestrutura global licenciada (Fase 2). 30

Figura 13 - Pipeline de dados: mapeamento, transformação, achatamento (flattening) e visualização analítica (Fase 2). 30

Figura 15 - Painel de visualização das interfaces e sub-interfaces lógicas de túnel no FortiGate SPOKE (Fase 2). 31

Figura 14 - Painel de visualização das interfaces físicas e lógicas no FortiGate HUB sob licenciamento comercial (Fase 2). 31

Figura 16 - Configuração detalhada de endereçamento ponto-a-ponto Overlay na interface lógica de túnel HUB-TO-SPOKE (Fase 2). 32

Figura 17 - Micro-arquitetura lógica, árvore analítica e pipeline de decisão baseada em API nativa no n8n (Fase 2). 34

Figura 18 - Parâmetros lógicos de rede da Phase 1 do túnel HUB-TO-SPOKE apontando para o gateway físico do Spoke (Fase 2). 35

Figura 19 - Parâmetros lógicos de rede da Phase 1 do túnel SPOKE-TO-HUB apontando para o gateway físico do Hub (Fase 2). 35
Figura 20 - Parametrização de autenticação e propostas criptográficas da Phase 1 IKEv2 no FortiGate HUB..... 36
Figura 21 - Especificações de autenticação, chaves e propostas criptográficas espelhadas da Phase 1 IKEv2 no FortiGate SPOKE. 37
Figura 22 - Janela de criação e revisão dos seletores de tráfego (Proxy-IDs) locais e remotos da Phase 2 no nó HUB (Fase 2). 38
Figura 23 - Mapeamento detalhado dos blocos CIDR legítimos para a rede local e remota na Phase 2 selectors do FortiGate HUB (Fase 2). 38
Figura 24 - Configuração da rota estática no FortiGate HUB direcionando pacotes da sub-rede destino para a interface do túnel (Fase 2). 39
Figura 25 - Configuração da rota estática simétrica no FortiGate SPOKE direcionando pacotes para o gateway central da Sede (Fase 2)..... 40
Figura 26 - Parametrização da política de firewall avançada de aceitação de tráfego de saída LAN-to-SPOKE no FortiGate HUB (Fase 2). 41
Figura 27 - Parametrização da política de firewall de retorno/entrada SPOKE-to-LAN no FortiGate HUB (Fase 2). 41
Figura 28 - Parametrização da política de firewall avançada de aceitação de tráfego de saída LAN-to-HUB no FortiGate SPOKE (Fase 2). 42
Figura 29 - Parametrização da política de firewall avançada de aceitação de tráfego de saída LAN-to-HUB no FortiGate SPOKE (Fase 2). 42
Figura 30 - Detalhe das opções de Firewall/Network Options e Logging de tráfego na política 1 do FortiGate HUB (Fase 2). 43
Figura 31 - Detalhe das opções avançadas de inspeção Flow-based e Logging de sessões na política 2 do FortiGate HUB (Fase 2). 43
Figura 32 - Detalhe das opções de Firewall/Network Options e Logging de tráfego na política 1 do FortiGate SPOKE (Fase 2). 44
Figura 33 - Detalhe das opções avançadas de inspeção Flow-based e Logging de sessões na política de segurança 2 do FortiGate SPOKE (Fase 2). 44
Figura 34 - Notificação automática de incidente via SMTP enviada pelo orquestrador e interceptada em ambiente de sandbox (Mailtrap) após a simulação de falha física no circuito MPLS (Underlay). 54
Figura 35 - Fluxo do n8n (Fase 1): Execução ativa do ramo de automação para a causa phase2_mismatch. 55
Figura 36 - Syslogs do FortiGate HUB (Fase 1): Correção de sub-redes da Phase 2 via Netmiko e restabelecimento do túnel..... 56
Figura 37 - Fluxo do n8n (Fase 1): Execução ativa do ramo de automação para a causa rgwy_mismatch. 59
Figura 38 - Syslogs do FortiGate HUB (Fase 1): Transição automática de falha na Phase 1 (negotiate failure) para restabelecimento do túnel (tunnel-up). 60
Figura 39 - Execução do workflow n8n no desvio lógico para falha elétrica (Fase 1). .. 61
Figura 40 - Rasto de execução ativa no n8n (Fase 2) evidenciando o desvio imediato para o ramo de falha de transporte externo. 61

Figura 41 - Rasto de execução ativa no n8n (Fase 2) demonstrando a triagem preventiva de interrupção de energia local. 62
Figura 42 - Notificação crítica de inacessibilidade na filial recebida no grupo de Telegram (Fase 2). 62
Figura 43 - Rasto de execução ativa no n8n (Fase 2) demonstrando o isolamento e retificação automática de drift no Gateway Remoto. 63
Figura 44 - Rasto de execução ativa no n8n (Fase 2) evidenciando o percurso do ramo de correção e subida da Phase 2. 64
Figura 45 - Rasto de execução ativa no n8n (Fase 2) consolidando o "Combo Duplo" de reativação de políticas de firewall via API. 65
Figura 46 - Interface gráfica do MongoDB Compass no ambiente de gestão (Windows), exibindo as instâncias de incidentes estruturadas em documentos JSON e armazenadas na coleção incident_runs no IP de produção 10.0.0.60 para rastreabilidade operativa (Fase 1). 68
Figura 47 - Gráfico de barras verticais no Metabase para Análise de Causa-Raiz, contabilizando a frequência absoluta das tipologias de falha isoladas pelo orquestrador ao longo dos 13 incidentes da linha de base (Fase 1). 69
Figura 48 - Gráfico circular do tipo donut no Metabase, evidenciando a distribuição percentual proporcional das tipologias de falha isoladas ao longo dos 13 incidentes da linha de base (Fase 1). 70
Figura 49 - Gráfico circular do tipo donut no Metabase, evidenciando a distribuição percentual proporcional das tipologias de falha isoladas ao longo dos 13 incidentes da linha de base (Fase 1). 71
Figura 50 - Distribuição dos 13 incidentes da linha de base segundo a variável remediationAllowed, evidenciando a separação entre as ocorrências com remediação autorizada (true) e as bloqueadas por origem física ou externa (false) (Fase 1). 71
Figura 51 - Distribuição do parâmetro remediationResult nos 13 incidentes registados, com 53,8 % de remediações bem-sucedidas e 46,2 % não tentadas. 72
Figura 52 - Distribuição percentual dos estados finais (FinalState) dos 13 incidentes registados 73
Figura 53 - Contagem absoluta de incidentes por estado final (FinalState). 73
Figura 54 - Distribuição dos desfechos de remediação (remediationResult) ao longo dos 11 incidentes da coleção Enterprise, no Metabase (Fase 2). 74
Figura 55 - Distribuição percentual das causas-raiz isoladas ao longo dos 11 incidentes da coleção Enterprise, no Metabase (Fase 2). 75
Figura 56 - Tabela de auditoria no Metabase com o detalhe dos 11 incidentes da coleção Enterprise, evidenciando o cruzamento entre causa-raiz, desfecho da remediação e registos extraídos do FortiOS (Fase 2). 76
Figura 57 - Exemplo de configuração de um Automation Stitch na CLI do FortiOS para a deteção de eventos e envio nativo de webhooks para o orquestrador. 81

Índice de tabelas

Tabela 1 - Comparação técnica de protocolos e interfaces de gestão de infraestruturas de rede. 2
Tabela 2 - Comparação de capacidades lógicas e operacionais entre o Projeto I e o Projeto II. 4
Tabela 3 - Tabela de Encaminhamento Estático das Firewalls. 17
Tabela 4 - Matriz de Decisão Operacional do Orquestrador n8n. 28
Tabela 5 - Matriz Comparativa Tecnológica e Operacional (Fase 1 vs Fase 2) 45
Tabela 6 - Comparação de métricas temporais e operacionais entre o paradigma manual e o sistema autónomo. 52
Tabela 7 - Matriz de decisão, atuação e auditoria dos cenários de Chaos Engineering (Fases 1 e 2). 66
Tabela 8 - Comparação das características técnicas e operacionais entre o Projeto I (diagnóstico passivo) e o Projeto II (automação ativa em ciclo fechado). 78

Lista de abreviaturas, siglas e acrónimos

API	*Application Programming Interface*
ARP	*Address Resolution Protocol*
ARTE, I.P.	Agência para a Reforma Tecnológica do Estado, Instituto Público
ASN.1	*Abstract Syntax Notation One*
BI	*Business Intelligence*
CD	*Continuous Delivery*
CI	*Continuous Integration*
CLI	*Command Line Interface*
DHCP	*Dynamic Host Configuration Protocol*
DNS	*Domain Name System*
FTP	*File Transfer Protocol*
HTTP	*Hypertext Transfer Protocol*
HTTPS	*Hypertext Transfer Protocol Secure*
IaC	*Infrastructure as Code*
ICMP	*Internet Control Message Protocol*
IETF	*Internet Engineering Task Force*
IKE	*Internet Key Exchange*
IP	*Internet Protocol*
IPsec	*Internet Protocol Security*
ITSM	*Information Technology Service Management*
JSON	*JavaScript Object Notation*
KEX	Key Exchange
LAN	*Local Area Network*
MIB	*Management Information Base*
MPLS	*Multiprotocol Label Switching*
MTTD	*Mean Time to Detect*
MTTR	*Mean Time to Resolution* / *Mean Time to Repair*
NETCONF	*Network Configuration Protocol*
NetDevOps	*Network Development and Operations*
NetOps	*Network Operations*

NGFW	*Next-Generation Firewall*
PDCA	*Plan, Do, Check, Act* (Planear, Executar, Verificar, Agir)
PRTG	*Paessler Router Traffic Grapher*
RAM	*Random Access Memory*
RBAC	*Role-Based Access Control*
REST	*Representational State Transfer*
RESTCONF	*REST-like Network Configuration Protocol*
RFC	*Request for Comments*
RGWY	*Remote Gateway*
SA	*Security Association*
SDN	*Software-Defined Networking*
SLA	*Service Level Agreement*
SMIv2	*Structure of Management Information Version 2*
SMTP	*Simple Mail Transfer Protocol*
SNMP	*Simple Network Management Protocol*
SO	Sistema Operativo
SER	*Site Reliability Engineering*
SSH	*Secure Shell*
SSoT	*Single Source of Truth* (Fonte Única da Verdade)
TCP	*Transmission Control Protocol*
TI	Tecnologias de Informação
UDP	*User Datagram Protocol*
URI	*Uniform Resource Identifier*
URL	*Uniform Resource Locator*
VM	*Virtual Machine* (Máquina Virtual)
VPN	*Virtual Private Network* (Rede Privada Virtual)
WAN	*Wide Area Network* (Rede de Área Alargada)
XML	*Extensible Markup Language*
YAML	*YAML Ain't Markup Language*
YANG	*Yet Another Next Generation*

1. Introdução e estado da arte

O crescimento dos serviços digitais e a dispersão geográfica das delegações da Administração Pública aumentaram a complexidade das infraestruturas de telecomunicações. Durante décadas, a gestão de redes assentou no modelo tradicional, designado *Legacy NetOps*: intervenções manuais, equipamento a equipamento, feitas de forma reativa na linha de comandos (CLI) [1]. Este modelo não acompanha o ritmo exigido pelas organizações atuais nem garante os níveis de resiliência pretendidos, o que levou o setor a adotar o modelo NetDevOps [2]. Esta filosofia adapta as melhores práticas do desenvolvimento de software — nomeadamente a integração contínua (CI) e a entrega contínua (CD) — ao universo físico das telecomunicações, tratando a rede como código dinâmico (*Network as Code* ou *Infrastructure as Code*) [2].

Historicamente, a monitorização e a gestão destas infraestruturas baseavam-se no protocolo *Simple Network Management Protocol* (SNMP), concebido para a recolha passiva de métricas por rotinas de *polling* estruturado. No entanto, o SNMP apresenta limitações estruturais: a ausência total de suporte para transações atómicas e complexas, a dificuldade de leitura de estados lógicos em arquiteturas híbridas e as vulnerabilidades de segurança inerentes às operações de escrita (*SET*) aceleraram a transição rumo às Redes Definidas por Software (SDN) [2]. Ao separar o plano de controlo do plano de dados, a arquitetura SDN centraliza a lógica de controlo e atua como **o cérebro** de tratamento de eventos sobre uma perspetiva logicamente centralizada [2].

Nesta linha evolutiva, o *Internet Engineering Task Force* (IETF) consolidou a modelação de dados ao introduzir a linguagem YANG (RFC 7950) [3], padronizando as configurações e os estados operacionais do hardware em árvores estruturadas e previsíveis (RFC 8340) [4]. Para manipular estas árvores de dados em tempo real, a engenharia de redes recorre hoje a protocolos de nova geração. O NETCONF (RFC 6241) [5] viabiliza o processamento de *payloads* XML cifrados mediante sessões seguras SSH, enquanto o RESTCONF (RFC 8040) [6] expõe interfaces Web RESTful perfeitamente compatíveis com estruturas de dados em formato JSON [7]. A **Tabela 1** sistematiza e contrasta as características técnicas da camada de transporte, os esquemas de modelação de dados e as garantias transacionais oferecidas por cada um destes mecanismos de gestão.

Tabela 1 - Comparação técnica de protocolos e interfaces de gestão de infraestruturas de rede.

Característica Técnica	Protocolo SNMP	Protocolo NETCONF (RFC 6241)	Protocolo RESTCONF (RFC 8040)	Programação CLI via Netmiko
Camada de Transporte	UDP (Portas 161/162)	SSH (Porta TCP 830)	HTTP/HTTPS [7]	SSH (Porta TCP 22)
Codificação de Dados	ASN.1 (Binary)	XML	JSON/XML [7]	Texto Limpo / CLI
Modelo de Dados	SMIv2 (MIBs)	YANG (RFC 7950)	YANG (RFC 7950) [7]	Proprietário do Fabricante
Suporte de Transações	Não Suporta	Sim (Commit/Rollback)	Não (Operações Atómicos)	Programático (Lógica de Script)
Limitações Experimentais	Inadequado para Escrita	Requer Ativação Manual	Bloqueado em Licenças Trial	Sem Restrições de Escrita

Mesmo perante o avanço técnico proporcionado pelo NETCONF e pelo RESTCONF, as restrições e os custos associados ao licenciamento proprietário continuam a introduzir atrito severo em ambientes laboratoriais e de simulação. As firewalls de nova geração (*Next-Generation Firewalls* ou NGFW) — tais como as instâncias virtuais FortiGate executadas sob licenças de avaliação (*Trial*) — bloqueiam nativamente qualquer operação de escrita na sua API REST. Desenhar e testar sistemas de autorremediação ativa (*Self-Healing Networks*) nestas condições requer, inevitavelmente, uma estratégia de orquestração híbrida e adaptável [2],[41].

Por conseguinte, de modo a superar esta barreira operacional no âmbito do **Projeto II**, a arquitetura do motor de automação foi dividida em duas fases evolutivas de desenvolvimento. A primeira, designada **Fase 1 (Mecanismo de Contingência — *SSH Fallback*)**, desenha um cenário onde o orquestrador n8n atua como o motor de tratamento de eventos, instanciando de forma assíncrona *scripts* Python isolados. Estes utilizam a biblioteca Netmiko para emular interações administrativas diretamente na CLI via SSH, contornando com sucesso as restrições da licença *Trial*. Já a **Fase 2 (Integração de APIs Nativas — *Enterprise*)** concretiza a transição para um ambiente com licenciamento comercial (Enterprise), libertando a utilização de chamadas síncronas HTTP nativas (métodos PUT e POST) contra a base de dados de configuração (*Configuration Management Database* ou CMDB) do FortiOS. Esta evolução tecnológica substitui a emulação de terminal por um modelo de orquestração ágil e atómico, suportado por nós HTTP Request nativos na plataforma n8n. A transição definitiva do diagnóstico passivo e da sinalização convencional para sistemas governados por automação em ciclo

fechado (*Closed-Loop Automation*) constitui o objeto central deste trabalho, alinhando-se com os vetores emergentes de redes baseadas em intenção (*Intent-Based Networking* ou IBN).

1.1 Enquadramento institucional: a missão da ARTE e da sua equipa de comunicações

A Agência para a Reforma Tecnológica do Estado, I.P. (ARTE), instituída ao abrigo do Decreto-Lei n.º 96/2025, assume a tutela estratégica da modernização administrativa e da cibersegurança do setor público em Portugal. O organismo é responsável por salvaguardar a operação contínua de plataformas críticas da Administração Pública, incluindo o portal gov.pt, gerindo a conectividade de centenas de balcões de atendimento ao cidadão — como as Lojas do Cidadão e os Espaços do Cidadão. Do ponto de vista técnico, a Equipa de Comunicações da ARTE funciona como um fornecedor de serviços de internet (*Internet Service Provider* ou ISP) interno para a administração pública portuguesa. A topologia física adota uma lógica híbrida estruturada em modelo *Hub-and-Spoke*: a infraestrutura de transporte (*underlay*) recorre a circuitos dedicados MPLS para suportar a malha lógica de túneis VPN IPsec (*overlay*), que interligam os balcões remotos (*Spokes*) ao centro de processamento de dados central (*Hub*).

A etapa preliminar desta investigação aplicada no **Projeto I**, concentrou-se na redução significativa do Tempo Médio de Diagnóstico (*Mean Time to Diagnose* ou MTTD). O isolamento automatizado de falhas lógicas e físicas combinou a capacidade do orquestrador n8n com a telemetria do PRTG, reduzindo os processos de triagem manual de vários minutos para poucos segundos. Ainda assim, a monitorização passiva e a emissão reativa de alertas revelaram-se insuficientes face aos exigentes Acordos de Nível de Serviço (*Service Level Agreements* ou SLAs) impostos ao atendimento público. Uma simples quebra de conectividade numa Loja do Cidadão acarreta a interrupção imediata da prestação de serviços essenciais, tais como a emissão de cartões de cidadão, passaportes ou certidões, acumulando prejuízos operacionais e danos reputacionais para a Administração Pública.

Com o início do **Projeto II**, as prioridades transitaram da mera observabilidade passiva do MTTD para a redução do Tempo Médio de Resolução (*Mean Time to Repair* ou MTTR). Desde logo, verificou-se que mesmo a rápida deteção da queda de um túnel VPN IPsec devido ao desalinhamento de parâmetros na **Fase 2** é insuficiente para satisfazer os requisitos operacionais da agência. O orquestrador deve triar a falha assim que esta é detetada e decidir entre duas vias: repor a conformidade da configuração face à Fonte Única de Verdade — regras de filtragem, seletores de tráfego (*Proxy-IDs*) e *gateway* remoto — ou forçar a renegociação das associações de segurança (*Security Associations* ou SAs); e, quando a causa-raiz for externa, como uma quebra do circuito MPLS do operador

ou um corte de energia na delegação, abortar a remediação e escalar o incidente, sem intervenção humana em nenhum dos casos

A redução do MTTR com recurso a ciclos autocorretivos fechados é o elemento essencial para garantir a estabilidade da rede na ARTE nesta nova fase.

A **Tabela 2** clarifica os avanços operativos e conceptuais face ao modelo original de diagnóstico passivo desenvolvido no **Projeto I**.

Tabela 2 - Comparação de capacidades lógicas e operacionais entre o Projeto I e o Projeto II.

Dimensão Operacional	Projeto I (Diagnóstico Passivo)	Projeto II (Auto-Remediação Ativa)
Métrica Principal de Sucesso	Minimização do MTDD (de minutos para segundos)	Controlo do MTRR (remediação autónoma sub-segundo)
Tipo de Comunicação com a API	Consultas de leitura (GET) em chamadas HTTP	Chamadas PUT/POST CMDB com contingência SSH Fallback
Acionador e Fluxo de Automação	Notificação assíncrona unidirecional para o Telegram	Ciclo fechado (*Closed-Loop*): Diagnóstico, Atuação e Validação
Tratamento de Falhas Ocultas	Ausência de validação para cortes de energia locais	Sondagem preventiva com Pre-Check ICMP no n8n
Armazenamento e Histórico	Inexistente (ausência de registos persistentes)	Base de Dados MongoDB (coleções *incident_runs* e *enterprise*) [8]
Painel de Visualização	Mensagens informativas em texto limpo no Telegram	Dashboards dinâmicos de *Business Intelligence* no Metabase [8]

1.2 Descrição do problema

A operação e a monitorização de redes híbridas baseadas na sobreposição lógica — nomeadamente topologias com *Overlay* VPN IPsec montado sobre um *Underlay* físico MPLS — introduzem uma opacidade operacional severa na equipa de engenharia. Os sistemas de monitorização convencionais assentes no SNMP fracassam de forma sistemática quando são chamados a correlacionar a quebra de um túnel lógico com a degradação ou queda do circuito de telecomunicações físico subjacente. Se um operador de telecomunicações externo sofrer uma rotura de fibra no circuito MPLS de um *Spoke*, as firewalls perdem de imediato a conectividade VPN IPsec. Contudo, os alarmes de rede gerados limitam-se a reportar a inoperabilidade do túnel, sem identificar a causa-raiz física que despoletou a avaria.

A transição experimental do **Projeto I** para o ambiente operacional e de simulação da ARTE revelou outros pontos cegos cruciais. O primeiro prende-se

com a ausência de um mecanismo de triagem para situações de inacessibilidade física do equipamento remoto. Caso ocorra um corte de eletricidade local na região ou o quadro elétrico de uma Loja do Cidadão seja desligado para ações de manutenção, a firewall remota desliga-se de imediato. Perante a inoperabilidade física do equipamento, o orquestrador n8n detetará de pronto a quebra da sessão IPsec. Sem uma lógica de verificação prévia, o motor dispararia de forma cega rotinas autónomas de alteração de *software* ou forçaria o reinício das chaves criptográficas. Trata-se de uma intervenção inútil, dada a total ausência de alimentação elétrica no hardware remoto. Este comportamento não só polui a rede de gestão com fluxos de sinalização desnecessários, como também induz os administradores em erro ao produzir diagnósticos incorretos.

Outra lacuna manifesta-se na ausência de um repositório centralizado de auditoria capaz de identificar desvios de configuração (*Configuration Drifts*) [9]. No quotidiano operacional, é comum que os engenheiros apliquem alterações manuais e pontuais diretamente na CLI dos equipamentos para resolver incidentes urgentes [9] [1]. Modificam-se rotas estáticas, ajustam-se *Proxy-IDs* ou suspendem-se temporariamente regras de filtragem à margem dos fluxos formais de aprovação. Se estas intervenções *ad-hoc* não forem devidamente reconciliadas com a Fonte Única de Verdade (SSoT), centralizada no NetBox [10], gera-se um desvio silencioso (*silent drift*) entre o estado ideal planeado e a configuração real em produção [9] [11]. Sem um mecanismo capaz de persistir estes dados em tempo real e de os expor num painel analítico central, a equipa perde a capacidade de rastrear a degradação e a conformidade da infraestrutura nacional.

Para efeitos de controlo e conformidade, a severidade matemática do desvio de configuração (Severity) de uma configuração ativa C face ao conjunto de regras lógicas documentadas R pode ser modelada analiticamente pela **Equação 1.1 — Cálculo da Severidade do Desvio de Configuração**.

$$\text{Severity}\,(C, R) = \sum_{i=1}^{|R|} w_i \cdot 1\,(r_i \text{ violada por } C) \qquad (1.1)$$

onde $\mathbf{1}$ representa a função indicadora que assume o valor unitário caso a política específica $r_i \in R$ seja violada por C, devolvendo zero no caso oposto. O parâmetro w_i define o peso ou o nível de criticidade atribuído a essa regra específica de conformidade [9].

1.3 Redes autonómicas, ciclo fechado e redes baseadas em intenção

A engenharia de redes autorregenerativas e a automação de infraestruturas de missão crítica têm registado uma evolução teórica e normativa relevante. O paradigma clássico de supervisão centrado na mera recolha passiva de métricas via SNMP foi progressivamente preterido em benefício de modelos de telemetria

baseados em APIs programáticas e na modelação estrita de dados. É esta transição estrutural que define o núcleo operativo do NetDevOps [2], [11]. Sob esta abordagem, as tarefas manuais executadas pontualmente na CLI dão lugar a fluxos automatizados e integrados que tratam a rede sob os princípios de *Infrastructure as Code* (IaC) e *Network as Code* (NaC), permitindo gerir e versionar a intenção do administrador com o mesmo rigor metodológico que rege o ciclo de vida do desenvolvimento de software moderno.

A conceção de ecossistemas em ciclo fechado (*closed-loop*) assenta diretamente da teoria dos sistemas autonómicos e do clássico ciclo de controlo MAPE-K (ou na sua variação MADA — *Monitor-Analyze-Decide-Act*) [12],[42]. No espectro normativo, o grupo de trabalho ANIMA (*Autonomic Networking Integrated Model and Approach*) do IETF tem liderado a normalização desta arquitetura. As orientações consolidadas nas diretivas RFC 7575 [13], RFC 8993 e RFC 9222 propõem uma mudança de modelo, preconizando que os ativos devem ser dispor de capacidades intrínsecas de autogestão, autoconfiguração, autoproteção e autocura (*self-healing*), diminuindo o nível de supervisão humana constante [14], [15]. Na vertente prática, a lógica em ciclo fechado opera por meio de um processo contínuo de conciliação: confronta-se sistematicamente o estado real reportado pelos dados de telemetria operacional com o estado desejado mapeado no repositório de intenção.

Neste contexto, a RFC 9315 consagra a Fonte Única de Verdade (SSoT) como o pilar estrutural das redes orientadas por intenção (*Intent-Based Networking* ou IBN) [10]. Sem a precisão deste ponto de ancoragem de dados, qualquer motor de orquestração carece da visibilidade necessária para calcular desvios lógicos (*configuration drift*) ou disparar de forma precisa as ações corretivas destinadas a restabelecer o equilíbrio do sistema. Paralelamente, as técnicas de verificação formal e as simulações suportadas por Gémeos Digitais de Rede (*Network Digital Twins* ou NDT) permitem validar alterações antes de qualquer intervenção física ou lógica [9]. Modelos topológicos normalizados pela RFC 8345 permitem antever o impacto de uma reconfiguração na infraestrutura ativa, mitigando o risco de colapso de tabelas de encaminhamento e mantendo as rotinas de autocura a limites de tolerância perfeitamente seguros. O presente **Projeto II** constitui uma materialização prática deste paradigma, elegendo o NetBox como *SSoT* operacional [10] e confiando ao orquestrador n8n a governabilidade executiva do ciclo de autocura.

1.4 Objetivo

O propósito central deste **Projeto II** reside na conceção, implementação e validação experimental de uma arquitetura de automação em ciclo fechado (*Closed-Loop Automation*) desenhada à medida para a infraestrutura de redes híbridas da ARTE. O sistema proposto visa diagnosticar incidentes de forma totalmente autónoma, executar ações de remediação em tempo real, validar a

conectividade pós-intervenção e auditar com rigor todas as modificações registadas na infraestrutura.

Para materializar esta visão, o plano de engenharia estruturou-se em torno de quatro metas fundamentais, desenhadas como objetivos integrados. A **primeira meta** foca-se no desenvolvimento de um fluxo de pré-validação preventiva (*Pre-Check*) no n8n, encarregue de realizar testes de conectividade (*pings*) contínuos ao endereço IP público do *Spoke* remoto. O intuito é distinguir falhas lógicas, sobre as quais é possível atuar, de situações em que o equipamento está inacessível no local.

A **segunda meta** assenta no desenvolvimento de um motor de execução híbrido que coloca em perspetiva e compara as abordagens de atuação da **Fase 1** e da **Fase 2**. Este motor de execução recorre à emulação de CLI por SSH via Netmiko como mecanismo de contingência e à injeção síncrona, recorrendo a chamadas PUT nativas, diretamente na CMDB da firewall FortiGate como solução definitiva.

A **terceira meta** prevê a integração de uma base de dados MongoDB em ambiente de contentores Docker [8]. Foram configuradas coleções dedicadas — incident_runs para a **Fase 1** e incident_runs_enterprise para a **Fase 2** — destinadas a armazenar o histórico forense completo de cada execução em formato JSON, salvaguardando os requisitos de auditoria e conformidade de segurança.

Por fim, a **quarta meta** consubstancia-se na implementação de uma plataforma Metabase para *Business Intelligence* (BI). Esta ferramenta visa a exposição visual e intuitiva de métricas de gestão e governação de TI, sustentada por uma amostragem real de **24** incidentes acumulados. Desta forma, torna-se possível avaliar continuamente o Tempo Médio de Resolução (MTTR), as taxas de sucesso e os desvios de configuração identificados na rede da ARTE.

O cálculo do $MTTR$ para um conjunto de incidentes N formalizado através da **Equação 1.2 - Cálculo do MTTR Autónomo.**

$$MTTR = \frac{\sum_{i=1}^{N}(T_{resolution,i} - T_{detection,i})}{N} \qquad (1.2)$$

onde $T_{detection,i}$ corresponde ao instante em que o sistema de monitorização deteta a falha lógica, enquanto $T_{resolution,i}$ representa o momento exato em que o orquestrador valida o restabelecimento do túnel IPsec, confirmando a sua conformidade com a Fonte de Verdade [1] [12].

1.5 Plano de trabalhos

A estruturação metodológica do **Projeto II** foi pautada pelo ciclo iterativo de melhoria contínua *Plan-Do-Check-Act* (PDCA) [16]. Esta abordagem assegurou que cada etapa do desenvolvimento estivesse estritamente alinhada com a transição entre a monitorização puramente passiva (herdada do **Projeto I**) e a autorremediação ativa em ciclo fechado. A **Figura 1** ilustra a dinâmica e a

aplicabilidade do modelo PDCA ao longo do ciclo de vida deste projeto, destacando a orientação para a automação proativa da infraestrutura.

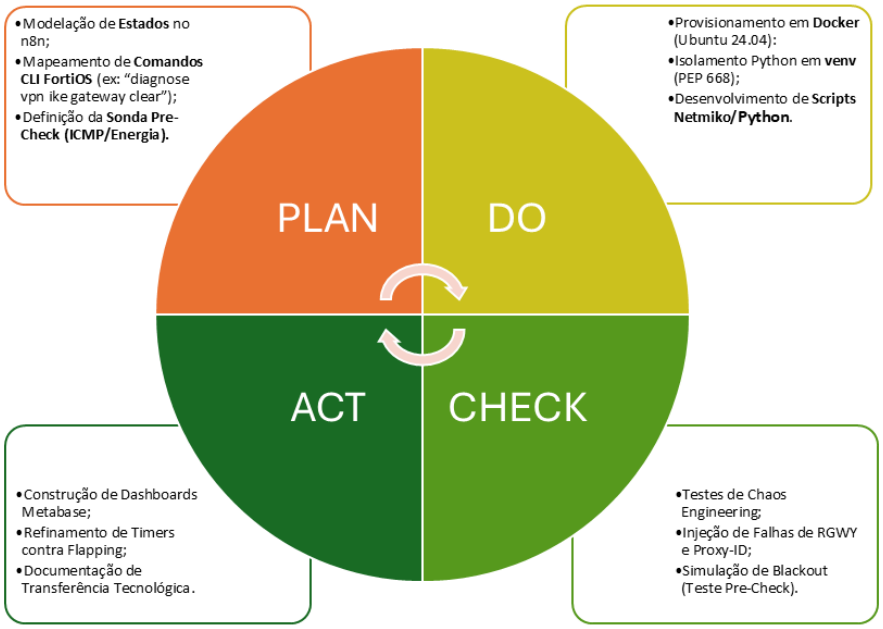


Figura 1 - Aplicação da metodologia PDCA (Plan-Do-Check-Act) no desenvolvimento do projeto de automação (Fonte: Elaboração própria).

A fase de **Planeamento (*Plan*)** centrou-se na modelação detalhada e na definição dos diagramas de estado da máquina de autorremediação a executar no n8n. Nesta etapa, a equipa delineou a especificação técnica da sonda de *Pre-Check* assente no ICMP para detetar a inacessibilidade do equipamento remoto, ao mesmo tempo que projetou a arquitetura de dados necessária para suportar tanto o mecanismo das versões Trail (Fase 1 via SSH) quanto a transição para a API comercial (Fase 2 via REST).

A etapa de **Execução (*Do*)** materializou a implementação prática e a migração de toda a *stack* de serviços para uma arquitetura de contentores Docker em ambiente Ubuntu Server. Sob as diretrizes do PEP 668, as dependências de software em Python foram rigorosamente isoladas em ambientes virtuais. Desenvolveram-se os *scripts* em Python assentes na biblioteca Netmiko para assegurar o *Fallback* por SSH e estruturaram-se os novos nós de chamadas síncronas para comunicação nativa com a REST API.

A fase de **Verificação (*Check*)** submeteu o sistema a ensaios rigorosos baseados nos princípios de Engenharia do Caos (*Chaos Engineering*). O laboratório experimental foi exposto à injeção controlada de falhas físicas — tais como a simulação de quebras de circuitos MPLS e cortes de energia — e falhas lógicas, incluindo desvios em *gateways*, alterações em sub-redes e modificações

arbitrárias em regras de filtragem das firewalls. A precisão e a fiabilidade do diagnóstico automatizado foram validadas através da análise minuciosa dos percursos de execução assinalados a verde (*execution workflows*) no n8n.

A etapa de **Ação e Padronização (*Act*)** encerrou o ciclo PDCA ao consolidar o repositório histórico no MongoDB e ao estruturar as consultas de agregação (*queries*) no Metabase. Esta integração culminou na criação de painéis analíticos dinâmicos voltados para o controlo do MTTR e para a auditoria forense dos 24 incidentes reais integrados no estudo. O encerramento desta fase culminou na redação e sistematização de procedimentos técnicos, garantindo uma transferência de conhecimento fluida e robusta para a Equipa de Comunicações da ARTE, I.P.

1.6 Organização do documento

Este documento organiza-se em quatro capítulos principais, estruturados de forma a acompanhar o progresso do projeto, desde o enquadramento teórico inicial até à análise crítica dos resultados operacionais.

O **Capítulo 1 (Introdução e estado da arte)** contextualiza a atividade institucional da ARTE, diagnostica os problemas de opacidade operacional na infraestrutura híbrida e define os objetivos do projeto, bem como o seu plano de trabalhos assente no ciclo PDCA. Adicionalmente, estabelece a fundamentação teórica, abordando os avanços recentes em redes autónomas baseadas em intenção e contrastando as limitações do protocolo SNMP com a flexibilidade das interfaces programáticas modernas como o NETCONF e o RESTCONF.

Seguidamente, o **Capítulo 2 (Arquitetura da stack e engenharia de execução)** pormenoriza a infraestrutura de serviços virtualizados em contentores Docker (MongoDB e Metabase) e as integrações de API coordenadas pelo n8n em articulação com o NetBox (SSoT). Detalha igualmente o isolamento de dependências Python em conformidade com o PEP 668, e debruça-se sobre o desenvolvimento do motor híbrido de execução, comparando o mecanismo de contingência por CLI SSH (Fase 1) com o modelo de injeção direta via chamadas REST API à CMDB das firewalls (Fase 2). Descreve ainda a metodologia de injeção controlada de falhas que sustenta os ensaios apresentados no capítulo seguinte.

O **Capítulo 3 (Resultados)** quantifica o impacto da automação no MTTR em cada uma das fases e apresenta a validação dos dez cenários de Engenharia do Caos, analisando os *syslogs* do FortiOS que documentam o restabelecimento automático do túnel VPN IPsec. Expõe, por fim, os painéis analíticos desenvolvidos no Metabase a partir dos incidentes persistidos no MongoDB.

Por último, o **Capítulo 4 (Conclusões)** sintetiza os contributos técnicos proporcionados pela introdução da automação em ciclo fechado na administração pública. Este capítulo confronta os resultados do **Projeto I** (diagnóstico passivo) com os do **Projeto II** (remediação ativa), discute a relação entre o MTTR e a disponibilidade global da infraestrutura híbrida, identifica as limitações do ambiente

simulado e traça o roteiro evolutivo para a ARTE, I.P., apontando caminhos de desenvolvimento no âmbito do novo paradigma NetOps 2.0 [17], assentes em AIOps, GitOps, integração com sistemas de ITSM e ChatOps sobre o Microsoft Teams.

2. Arquitetura da stack e engenharia de execução

Este capítulo descreve a materialização e a validação da arquitetura de automação em ciclo fechado (*closed-loop automation*) e autorregeneração ativa (*active self-healing*). Concebido no seio da Escola Superior de Tecnologia do IPCB, em estreita cooperação com a ARTE, I.P., a solução corporiza uma mudança metodológica. Ao transitar de esquemas tradicionais de diagnóstico passivo para um sistema autonómico bidirecional, a arquitetura responde à necessidade de reduzir o Tempo Médio de Resolução (MTTR) perante incidentes de conectividade [18] [19].

2.1 Enquadramento e modelação teórica do MTTR

Nas infraestruturas híbridas contemporâneas, onde as redes lógicas de sobreposição (*overlay* VPN IPsec) dependem da estabilidade dos circuitos de transporte físico subjacentes (*underlay* MPLS), qualquer anomalia tende a desencadear alarmes em cascata. O diagnóstico manual subsequente revela-se tipicamente demorado e impõe certa carga cognitiva aos administradores de sistemas. Esta abordagem clássica, dominada por intervenções pontuais e varreduras manuais via CLI (*Command Line Interface*), fragmenta a atenção dos engenheiros, gerando constantes quebras de foco operacionais (*context switching*) que comprometem a produtividade global da equipa.

Traduzir esta ineficiência sob uma perspetiva analítica ajuda a evidenciar o impacto imediato da automação ativa. O ciclo geral de mitigação de incidentes traduz-se no tempo de inatividade total (T_{total}), formalizado através da **Equação 2.1 - Tempo de Inoperabilidade Operacional Total**:

$$T_{total} = T_{deteção} + T_{triagem} + T_{remediação} \quad (2.1)$$

No sistema preliminar do **Projeto I**, cujo âmbito se limitava ao diagnóstico passivo, os esforços de otimização concentravam-se exclusivamente no parâmetro $T_{deteção}$ por meio de monitorização e alertas estáticos. Por conseguinte, as parcelas temporais dedicadas à triagem ($T_{triagem}$)e à remediação ($T_{remediação}$) dependiam da rapidez, discernimento e disponibilidade do profissional de serviço.

A introdução de uma lógica bidirecional em ciclo fechado altera radicalmente esta dinâmica. O tempo de triagem deixa de ser subjetivo e passa a obedecer a rotinas algorítmicas determinísticas que atuam na ordem dos milissegundos, permitindo assumir na prática que ($T_{triagem} \rightarrow 0$). Ao mesmo tempo, as ações de remediação automatizadas comprimem a variável $T_{remediação}$ ao limite físico imposto pelo tempo de processamento e aplicação de comandos na CLI do ativo de rede [20]. Desta forma, o teto de eficiência teórica do ciclo de autocura passa a ser delimitado matematicamente pela **Equação 2.2 - Limite de Eficiência Algorítmica (*Closed-Loop*)**:

$$\lim_{T_{triagem}, T_{remediação} \to 0} MTTR \approx T_{deteção} + T_{remediação} \quad (2.2)$$

A salvaguarda da estabilidade e da cibersegurança da infraestrutura assentou numa estratégia de implementação incremental. Para o efeito, definiu-se um processo evolutivo organizado em quatro etapas encadeadas: a modelação inicial das redes virtuais, o isolamento entre os planos de controlo e de dados, a normalização de dados na Fonte Única de Verdade (SSoT) e, por fim, a consolidação das rotinas de remediação ativa. Esta abordagem faseada mitiga o risco de surgimento de ciclos de retroalimentação positiva (*feedback loops*), garantindo a estabilidade da topologia híbrida e prevenindo reconfigurações que possam desestabilizar os ativos de rede durante períodos críticos [21].

2.2 Identificação das técnicas e ferramentas específicas

A composição da *stack* tecnológica resultou de uma avaliação criteriosa das necessidades de negócio e de engenharia da ARTE, I.P., validada ao longo de sucessivas sessões de acompanhamento técnico. As decisões refletem o estado da arte em NetDevOps e asseguram que o núcleo funcional de automação assente sobre uma infraestrutura de suporte resiliente.

2.2.1 Componentes principais da solução

A seleção dos componentes de rede focou-se na reprodução fiel das condições operacionais reais observadas na ARTE, I.P. No que toca à segurança perimetral e terminação das ligações VPN, a arquitetura mobiliza firewalls FortiGate NGFW de sistema operativo FortiOS. Estes equipamentos disponibilizam um plano de encaminhamento robusto e expõem a REST API do fabricante, a qual o orquestrador interroga ciclicamente para obter leituras estruturadas sobre o estado operacional do hardware. Esta telemetria de precisão abrange desde métricas de tráfego tradicionais com o volume de tráfego de entrada e saída medido em *bytes* (MB, GB ou TB), até variáveis de controlo mais complexas, tais como os seletores de tráfego dinâmicos (*Proxy-IDs*).

Durante a fase laboratorial, deparámo-nos com constrangimentos práticos incontornáveis: as instâncias de virtualização do FortiGate operam sob uma licença *Trial* que inibe operações de escrita via REST API no FortiOS. O fabricante restringe nativamente a execução de verbos HTTP de modificação (nomeadamente POST, PUT e PATCH), limitando o acesso a consultas informativas do tipo GET e inviabilizando alterações diretas na CMDB do equipamento [15]. Para contornar esta restrição técnica sem comprometer a viabilidade do protótipo, desenhou-se um canal de controlo alternativo *out-of-band* baseado em SSH. Este mecanismo invoca *scripts* autónomos em Python para forçar as ações de autocura diretamente na CLI, uma estratégia cujos fundamentos são detalhados nas secções seguintes [22].

Assegurar a coerência topológica e erradicar falhas humanas decorrentes de documentação dispersa ou desatualizada exigiram a centralização dos dados lógicos. Sob este prisma, o protótipo adotou o NetBox como a Fonte Única de Verdade (SSoT). A plataforma atua como o repositório fundamental de gestão de endereços IP (IPAM) e de modelação lógica dos circuitos. A integração de uma *SSoT* é o pilar que impede a deriva silenciosa de configurações (*configuration drift*) — um fenómeno insidioso que ocorre quando alterações locais e manuais nas firewalls desviam a infraestrutura ativa do modelo homologado, degradando as políticas de segurança e comprometendo o tráfego IPsec [20].

No centro operacional da arquitetura situa-se a plataforma de orquestração n8n. Operando como o motor de decisão, o n8n articula-se dinamicamente com as diversas APIs e efetua o tratamento nativo de *payloads* em formato JSON. O processamento ágil de objetos estruturados facilita as avaliações condicionais sobre as respostas emitidas pelas firewalls e pelo NetBox, assegurando que as rotinas corretivas são executadas com base em árvores lógicas determinísticas e perfeitamente controladas [23].

A recolha contínua de telemetria e a sinalização inicial de anomalias estão a cargo do Paessler PRTG Network Monitor. A ferramenta funciona como o acionador primário (*trigger*) do sistema, avaliando o estado operacional e o desempenho dos túneis por meio de consultas cíclicas (*polling*). Assim que as métricas recolhidas excedem os limiares de tolerância estipulados pela equipa de comunicações, o PRTG despacha de imediato notificações estruturadas via *webhooks* HTTP (em formato JSON) para um *endpoint* escutado no n8n [24].

2.2.2 Infraestrutura de suporte e ferramentas auxiliares

O suporte de base à arquitetura laboratorial assentou sobre o sistema operativo Ubuntu Server 24.04 LTS, escolhido pela sua elevada estabilidade, suporte continuado e ampla compatibilidade com ambientes de virtualização por contentores. Embora as fases iniciais do projeto tenham recorrido ao Oracle VirtualBox para a simulação de recursos, a emulação das *appliances* virtuais FortiGate (FGT-VM64) deparou-se com limitações severas no que concerne à estabilidade das interfaces de rede e à estruturação de barramentos de dados complexos [25]. Os nós virtualizados FortiGate exigem suporte robusto a instruções de virtualização avançada de CPU e comportam-se de forma sensível na emulação de múltiplas interfaces em modo promíscuo — requisitos que o VirtualBox não conseguiu colmatar de forma consistente sob carga de tráfego contínua. Perante esta limitação técnica, a equipa migrou de forma estratégica o ambiente de emulação para o VMware Workstation Pro, uma transição extinguiu as perdas de tráfego nos comutadores virtuais (*vSwitches*) e garantiu o funcionamento ininterrupto do FortiOS.

O isolamento e a execução dos microsserviços de automação foram viabilizados por intermédio de contentores Docker. Recorrendo a ficheiros de configuração YAML, a orquestração do sistema de dados — composto pelo n8n, a base de dados

documental MongoDB e a plataforma analítica Metabase — é completamente declarativa, em concordância com os princípios de *Infrastructure as Code* (IaC) [20]. Esta arquitetura confere portabilidade ao sistema, permitindo que toda a *stack* seja reconstruída e executada em qualquer servidor anfitrião da ARTE, I.P. mediante uma instrução simplificada e única. O modelo mitiga discrepâncias entre versões de bibliotecas e evita incompatibilidades com as dependências do sistema operativo hospedeiro [26].

No ambiente do Ubuntu Server 24.04, o desenvolvimento de rotinas em Python destinadas à triagem e remediação colidiu com as novas diretivas de segurança estabelecidas pelo PEP 668 [27], visto que esta especificação impede a instalação arbitrária de pacotes externos via utilitário *pip* diretamente no interpretador Python global do sistema operativo, gerando uma mensagem de erro indicativa de um ambiente gerido externamente (externally-managed-environment) [28]. imposição do PEP 668 visa proteger a integridade dos serviços internos e ferramentas de sistema que dependem de versões específicas de bibliotecas Python, impedindo desequilíbrios na estabilidade do sistema decorrentes de atualizações ou sobreposições indesejadas de pacotes [29].

Para contornar esta restrição de segurança, no projeto foi implementado o isolamento estrito das execuções Python em ambientes virtuais (venv). A configuração envolveu a criação e a estruturação de um diretório dedicado em */opt/venv_netmiko*, processado através dos comandos ilustrados na **Figura 2** [30]:

```
sudo apt-get update && sudo apt-get install -y python3-pip python3-venv
python3 -m venv /opt/venv_netmiko
source /opt/venv_netmiko/bin/activate
pip install --upgrade pip
pip install netmiko
```

Figura 2 - Comandos de criação e ativação do ambiente virtual Python para a biblioteca Netmiko.

A criação deste ambiente virtual isolado, livre de interferências do sistema operativo, permitiu alojar de forma segura a biblioteca Netmiko [22]. Esta biblioteca destaca-se pela sua robustez na orquestração de sessões SSH programáticas em dispositivos de redes híbridas [31]. Ao integrar a biblioteca Netmiko, o orquestrador n8n é capaz de instanciar autonomamante os *scripts* de contingência através da *shell* local do servidor. Esta solução ágil contorna o bloqueio de escrita observado na REST API do FortiGate sob licença de demonstração (*Trial*), permitindo que o sistema execute comandos corretivos diretamente na CLI do hardware por um canal totalmente seguro e passível de auditoria contínua [25].

2.3 Procedimento de implementação

Concluída a caracterização técnica da *stack*, importa agora detalhar a parametrização prática da arquitetura NetDevOps. Esta opção traduz os fundamentos teóricos da computação autónoma e da Infraestrutura como Código (IaC) num ambiente de testes de alta fidelidade, capaz de espelhar as condições encontradas na ARTE, I.P. [19]

Para estruturar as interações entre os diversos sistemas e desenhar o fluxo de controlo, delineou-se o modelo conceptual apresentado na **Figura 3**. Este diagrama estabelece a lógica sequencial do ciclo fechado (*closed-loop*), estendendo-se desde a telemetria inicial até à persistência analítica e ao escalonamento formal de incidentes. A execução deste fluxo obedece a uma ordem estritamente sistemática, garantindo que as plataformas cooperem de forma previsível, segura e totalmente auditável [32].

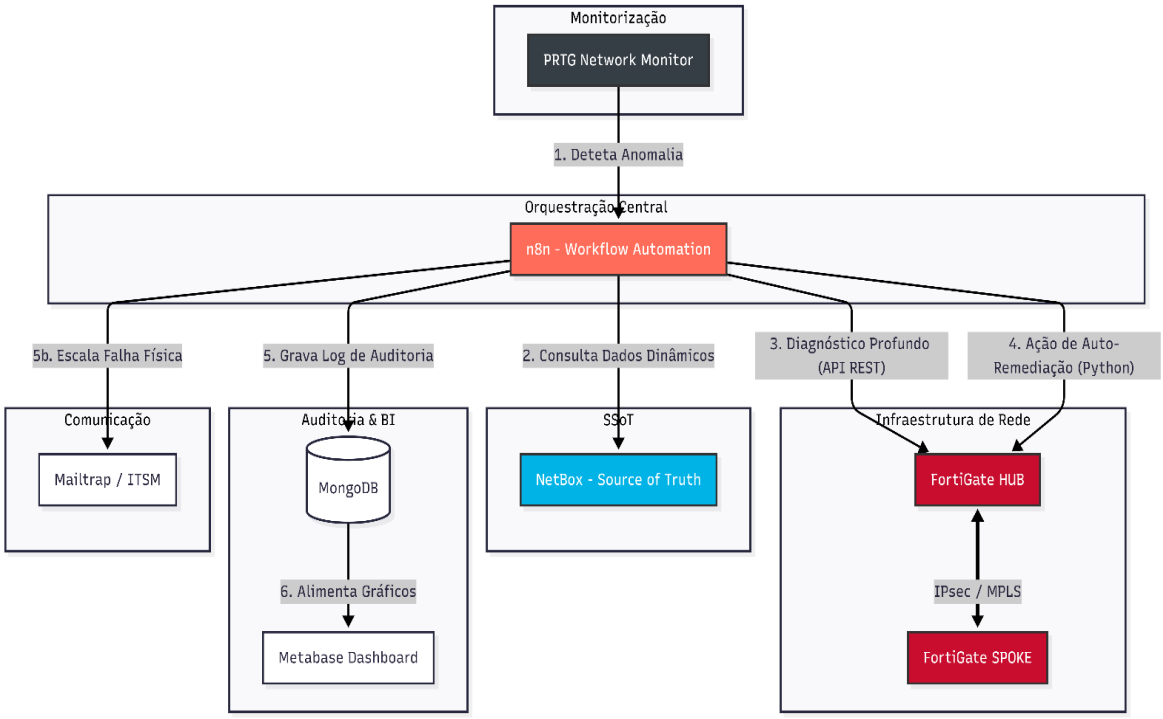


Figura 3 - Arquitetura tecnológica macro e fluxo sequencial de interação e controlo entre os componentes da arquitetura NetDevOps em ciclo fechado (Closed-Loop).

Do ponto de vista prático, a engenharia da solução divide-se em dois eixos interdependentes. O primeiro compreende a segmentação física e lógica da infraestrutura virtualizada, salvaguardando o isolamento do tráfego de *underlay* e *overlay*. O segundo debruça-se sobre a lógica algorítmica de orquestração. Conforme planeado, o desenvolvimento estruturou-se em duas fases distintas: a **Fase 1**, condicionada pelas restrições de escrita das licenças *Trial*, e a **Fase 2**, impulsionada pelo beneficio trazido das licenças *Enterprise* cedidas para o efeito.

2.3.1 Configuração da infraestrutura de rede

A montagem prática do laboratório iniciou-se com a definição e o isolamento das fronteiras lógicas e físicas da topologia. A arquitetura tira partido do hipervisor VMware Workstation Pro para segmentar as redes virtuais privadas (VMnets), simulando a dispersão geográfica de um cenário WAN. Este isolamento estrito impede que tráfego residual se propague entre os domínios de gestão, o plano de transporte MPLS e os *endpoints* criptográficos.

A **Figura 4** ilustra a sobreposição estrutural e o esquema de endereçamento adotado para a comunicação entre a Sede (*Hub*) e a filial remota (*Spoke*), evidenciando a demarcação estrita entre o plano de transporte e o plano de dados seguro. Esta segmentação lógica foi organizada em quatro áreas funcionais e operacionais distintas; conforme se detalha de seguida.

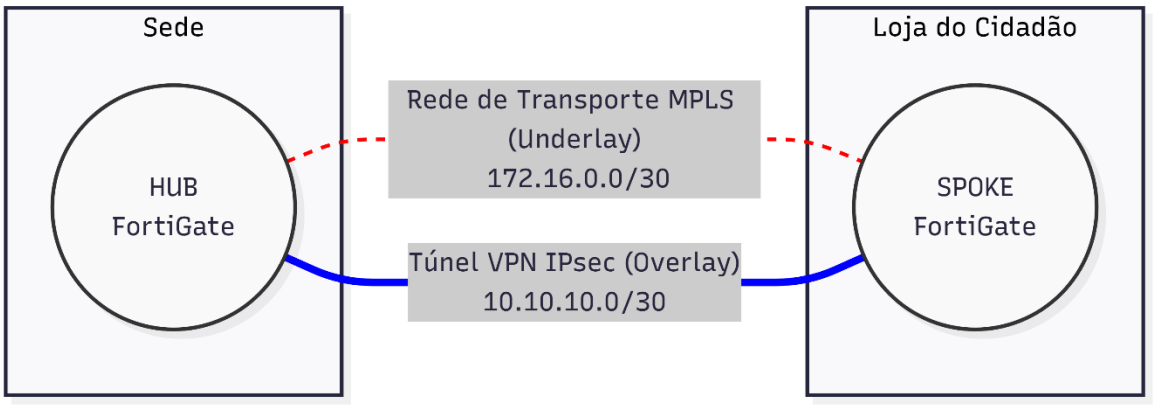


Figura 4 - Topologia lógica da infraestrutura híbrida da ARTE: mapeamento e abstração da camada de transporte físico (Underlay MPLS) e da camada de conectividade encriptada (Overlay IPsec).

Primeiro, a **VMnet2 (LAN Hub / Gestão Out-of-Band - 10.0.0.0/24)** define o domínio isolado dedicado em exclusivo ao tráfego de controlo e administração da infraestrutura. Nele residem a interface física port2 do FortiGate HUB (10.0.0.1), o servidor de monitorização PRTG e a máquina Ubuntu Server (10.0.0.60) que aloja a *stack* Docker com o n8n, mongoDB e mais tarde Metabase. Esta segregação assegura que os canais de gestão se mantenham operacionais mesmo perante falhas catastróficas nos circuitos de transporte.

Segundo, a **VMnet5 (MPLS Underlay - 172.16.0.0/30)** funciona como uma rede ponto a ponto destinada a simular a infraestrutura de transporte físico fornecida por um operador público de telecomunicações. Interliga diretamente a interface port3 (WAN) do FortiGate HUB (172.16.0.1) à interface port2 (WAN) do FortiGate SPOKE remoto (172.16.0.2).

Noutro vetor, a **VMnet4 (Endpoints IPsec Overlay - 10.10.10.0/30)** agrega as interfaces de terminação lógica de túnel necessárias para consolidar a associação de segurança criptográfica entre ambos os dispositivos. O *gateway* central do lado do HUB assume o endereço IP (10.10.10.1) na interface lógica port1 (túnel),

enquanto o SPOKE utiliza o endereço IP (10.10.10.2) na sua interface lógica port3 (túnel).

A fechar este alinhamento, a **VMnet3 (LAN Spoke remota - 192.168.10.0/24)** define a sub-rede local privada associada à filial remota (por exemplo, uma Loja do Cidadão), conectada fisicamente à interface port1 do FortiGate SPOKE (192.168.10.1), que atua como o seu *default gateway*.

Com o intuito de viabilizar o encaminhamento de dados bidirecional entre os dois extremos através do túnel cifrado, procedeu-se à configuração de rotas estáticas nas tabelas de encaminhamento de ambos os dispositivos, conforme se detalha na **Tabela 3**.

Tabela 3 - Tabela de Encaminhamento Estático das Firewalls.

Equipamento de Rede	Rede de Destino	Máscara de Rede	Gateway Próximo Salto	Interface de Saída	Distância Administrativa
FortiGate Hub	192.168.10.0	255.255.255.0	10.10.10.2	HUB-TO-SPOKE	10
FortiGate Hub	172.16.0.0	255.255.255.252	Diretamente ligada	port3 (WAN)	0
FortiGate SPOKE	10.0.0.0	255.255.255.0	10.10.10.1	SPOKE-TO-HUB	10
FortiGate SPOKE	172.16.0.0	255.255.255.252	Diretamente ligada	port2 (WAN)	0

Ao nível das políticas de segurança de firewall de ambos os ativos, definiram-se regras com ação de aceitação explícita (accept) as<sociadas ao serviço ALL. As diretivas de segurança LAN-to-SPOKE no HUB e LAN-to-HUB no SPOKE viabilizam o fluxo de tráfego interno em direção ao túnel, ao passo que as regras simétricas de retorno (SPOKE-to-LAN e HUB-to-LAN) garantem a entrega segura do tráfego proveniente do *overlay* às sub-redes locais correspondentes.

2.3.2 Fase 1: orquestração e diagnóstico em ambiente trial (mecanismo SSH Fallback)

A lógica de *closed-loop* implementada na plataforma n8n funciona como um sistema de suporte à decisão operacional da rede. O motor executa sequencialmente uma cadeia de fases analíticas determinísticas para identificar e remediar incidentes [31]. Ao longo do desenvolvimento do projeto, o desenho conceptual deste fluxo sofreu uma evolução arquitetural iterativa, imposta pela necessidade de mitigar pontos identificados em cenários realistas da ARTE, I.P.

Inicialmente, a abordagem focava-se num modelo puramente reativo de reposição de estados, conforme esquematizado na **Figura 5**. Perante qualquer falha lógica nos seletores de tráfego (*Proxy-ID*), gateways remotos mal configurados ou interrupções de tráfego silenciosas, o fluxo encaminhava a execução diretamente para duas ações: o reinício global do túnel IPsec ou trocar o IP do gateway remoto pelo IP real de acordo com a SSoT. Contudo, esta arquitetura inicial revelava-se fraca perante anomalias na infraestrutura local das filiais, por não dispor de qualquer mecanismo para despistar falhas de natureza física.

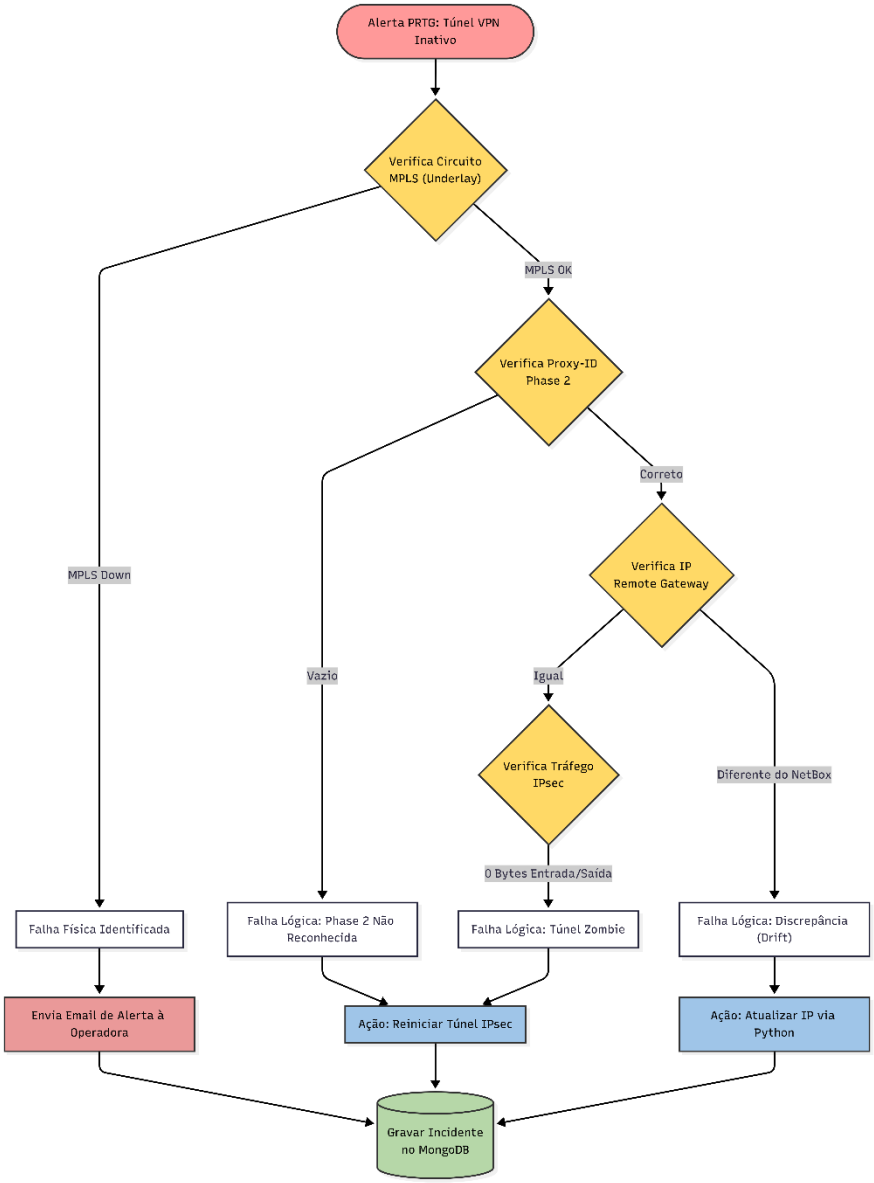


Figura 5 - Fluxograma inicial da lógica reativa da arquitetura em ciclo fechado, baseada no reinício unificado de chaves criptográficas e desprovida de triagem física.

De modo a responder às exigências operacionais e isolar situações de inacessibilidade física do equipamento remoto o modelo foi profundamente reestruturado. A arquitetura evoluiu para o paradigma de redes baseadas em intenção (*Intent-Based Networking*), mapeado no fluxo sequencial da **Figura 6**.

A versão consolidada introduz uma sonda preventiva de conectividade física (*Pre-Check* via ICMP) antes de qualquer interação com a API REST ou a CLI dos equipamentos. Com este refinamento do bloco de decisão, o sistema passou a segregar as anomalias de forma granular, mapeando-as para *playbooks* específicos de IaC.

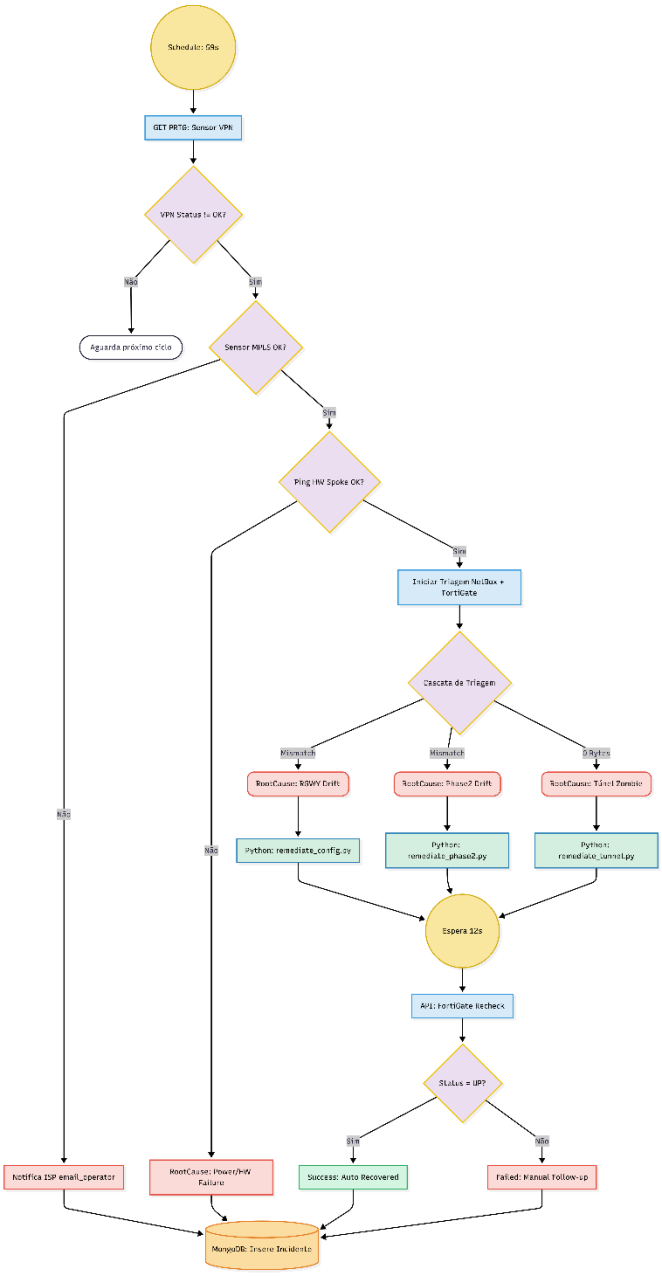


Figura 6 - Fluxograma finalizado da arquitetura autónoma com triagem preventiva de hardware (Pre-Check ICMP) e ramificações cirúrgicas de auto-remediação configuracional.

Etapa 1: Trigger e Diagnóstico de Underlay

O acionador do fluxo foi configurado para executar em intervalos de 10 segundos, interrogando ciclicamente a API do PRTG Network Monitor para obter o estado do sensor associado ao túnel IPsec [33]. Quando o sensor deteta o estado diferente de OK, o fluxo prossegue para o diagnóstico; caso contrário, termina sem consumir recursos adicionais.

Detetada a queda, o n8n interroga a API REST do NetBox (SSoT) para obter o endereço IP do circuito MPLS associado ao dispositivo afetado. Com esse endereço, o fluxo consulta novamente a API do PRTG, desta vez para o sensor da interface de transporte. Caso o sensor se demonstrar igualmente diferente do estado OK, conclui-se que a indisponibilidade tem origem na camada física e o motor suspende o diagnóstico sobre a VPN. O fluxo envia então um alerta via SMTP para o operador de telecomunicações externo e persiste o incidente no MongoDB sob o estado *escalated_to_provider.*

Etapa 2: Mecanismo de Pré-Verificação Física (Pre-Check)

Confirmado que o sensor do circuito MPLS figura no estado OK, o n8n prossegue para a validação física. Este fluxo executa um teste ICMP a partir da *shell* do Ubuntu Server, tendo como alvo o endereço IP WAN do FortiGate SPOKE remoto. Para assegurar o determinismo da prova sem depender do comportamento por omissão do sistema operativo, fixa-se o tempo limite de resposta (*Round-Trip Timeout* ou RTO) em 1 segundo por eco, sendo que a perda dos três pacotes sequenciais força a devolução de um código de saída de erro pelo *kernel*.

Importa sublinhar que este teste é executado após a exclusão da camada de transporte: quando o fluxo alcança esta etapa, a operacionalidade do circuito já foi verificada. O destino do teste é o endereço WAN do SPOKE, alcançável independentemente do estado do túnel IPsec, pelo que uma falha de negociação criptográfica não afeta o resultado.

O workflow avalia então o código de saída através da variável exitCode. Se o retorno for diferente de zero (exitCode≠0), conclui-se que o equipamento remoto está inacessível apesar de o transporte se encontrar operacional. Esta condição é compatível com uma falha de alimentação na delegação, uma avaria de hardware, ou uma rotura no circuito ainda não refletida no sensor de monitorização. O orquestrador não discrimina entre estas hipóteses — em qualquer delas, a atuação remota é inviável. O fluxo suspende as tentativas de ligação subsequentes, prevenindo o bloqueio por *timeouts*, e persiste a ocorrência no MongoDB com o rótulo *power_or_hardware_failure*.

Etapa 3: Triagem Avançada da VPN e Atuação IaC (SSH Fallback)

Caso o *pre-check* físico termine com sucesso, registando *exitCode==0* e atestando que o equipamento remoto se encontra ativo na rede WAN, o n8n realiza uma requisição HTTP GET à REST API do FortiGate HUB [34].

O processamento do *payload* JSON recebido é executado de forma linear. Esta abordagem elimina a concorrência na árvore de decisão e impede que subestados paralelos despoletem ações de remediação concorrentes na firewall. Este mecanismo de triagem sequencial profunda encontra-se modelado analiticamente na cascata de exclusão da **Figura 7**, que ilustra a priorização dos testes condicionais sob três níveis distintos de verificação.

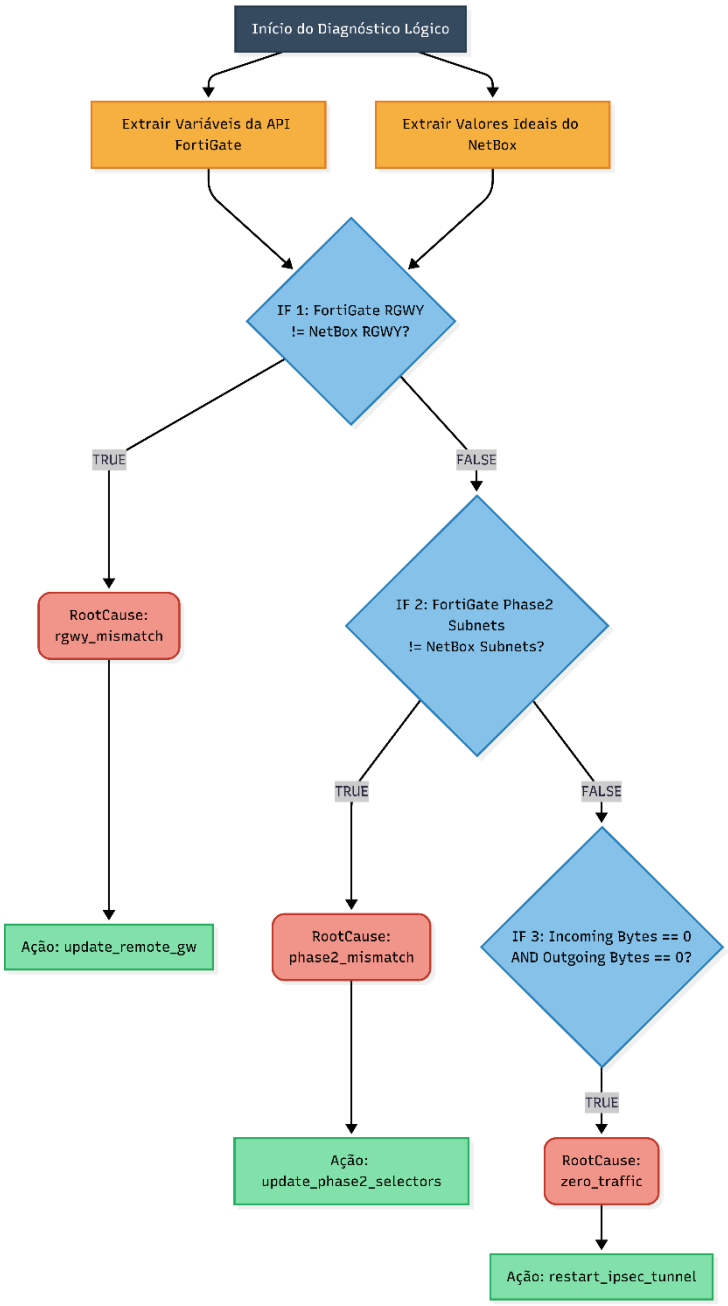


Figura 7 - Fluxograma da cascata de exclusão condicional para triagem profunda da VPN.

O algoritmo opera por exclusão de partes numa sequência bem definida. Inicia com a verificação do IP de *Gateway* no primeiro bloco condicional (IF 1), validando se o parâmetro remote-gw ativo na firewall diverge da informação documentada no NetBox. Caso se detete uma discrepância, o sistema isola a causa-raiz sob a etiqueta *rgwy_mismatch* e agenda a ação de retificação *update_remote_gw*.

Se o gateway estiver em conformidade, a execução avança para o segundo nível de triagem (IF 2), focado na validação dos seletores de *Phase 2*. Nesta etapa, os *Proxy-IDs* ativos são comparados com as sub-redes legítimas na SSoT. Qualquer anomalia detetada assume o estado *phase2_mismatch*, ativando o procedimento *update_phase2_selectors*.

Quando a integridade das configurações é validada em ambos os patamares, o fluxo prossegue para a análise volumétrica do tráfego (IF 3) para aferir a passagem de pacotes. Se os contadores de bytes de entrada e saída registarem em simultâneo um valor nulo, o orquestrador diagnostica a ocorrência de um túnel órfão (zero_traffic), direcionando autonomamente o workflow para a ação de mitigação *restart_ipsec_tunnel*.

A **primeira linha de remediação** trata diretamente o desvio de gateway remoto (rgwy_mismatch). Se o endereço IP configurado no Remote Gateway (remote-gw) do túnel no FortiGate HUB divergir do endereço de rede correto documentado no NetBox devido a desvios configuracionais manuais indesejados [20], o n8n invoca o *script* de remediação *remediate_config.py*. Este *script*, executado de forma isolada no ambiente virtual Python (venv) [36] conforme ilustrado na **Figura 8**, conecta-se com segurança à firewall via Netmiko/SSH [22] para injetar uma matriz de blocos de comando diretamente no ramo hierárquico da interface IPsec, restabelecendo a conformidade operativa do plano de controlo sem causar indisponibilidade.

```
 GNU nano 7.2                                                remediate_config.py *
import sys
import time
from netmiko import ConnectHandler
def update_remote_gateway(tunnel_name, expected_ip):
    fortigate = {
        'device_type': 'fortinet',
        'host': '10.0.0.1',
        'username': 'admin',
        'password': 'admin',
        'port': 22,
    }
    print(f"[*] A iniciar remediacao do Gateway Remoto para o tunel: {tunnel_name}")
    try:
        net_connect = ConnectHandler(**fortigate)
        print("[+] Ligacao SSH estabelecida com sucesso")
        config_commands = [
            'config vpn ipsec phase1-interface',
            f'edit "{tunnel_name}"',
            f'set remote-gw {expected_ip}',
            'next',
            'end'
        ]
        print(f"[*] A aplicar a config do novo RGWY: {expected_ip}...")
        output = net_connect.send_config_set(config_commands)
        net_connect.send_command(f'diagnose vpn tunnel clear {tunnel_name}')
        print("[+] Config aplicada com sucesso via CLI")
        print(output)
        net_connect.disconnect()
        return True
    except Exception as e:
        print(f"[-] Erro durante a remediacao SSH: {e}")
        return False
if __name__ == "__main__":
    if len(sys.argv) !=3:
        print("Uso : python3 remediate_config.py <Nome_do_Tunel> <Novo_IP_Gateway>")
        sys.exit(1)
    tunnel_arg = sys.argv[1]
    ip_arg = sys.argv[2]
    success = update_remote_gateway(tunnel_arg, ip_arg)
    if success:
        sys.exit(0)
    else:
        sys.exit(1)
```

Figura 8 - Script Python (remediate_config.py) desenvolvido com a biblioteca Netmiko para a remediação configuracional ativa e retificação idempotente de desvios de gateway.

A **segunda linha de remediação**, resolve-se o desalinhamento criptográfico na *Phase 2* (phase2_mismatch). A integração deste subestado justifica-se pela necessidade de ensinar a arquitetura autónoma da capacidade para distinguir falhas operativas transitórias de desvios de conformidade estruturais (*Configuration Drifts*). No modelo de triagem original, qualquer inconsistência nos seletores de tráfego resultava num problema pendente de resolução de um profissional. Contudo, demonstrou-se empiricamente que, se um erro humano introduzir uma alteração manual incorreta nas sub-redes locais ou remotas nas firewalls, reiniciar as chaves criptográficas é incapaz de restaurar o serviço, perpetuando falhas de negociação.

Para resolver este ponto, a lógica do orquestrador n8n evoluiu para o paradigma de redes baseadas em intenção, isolando o desvio como phase2_mismatch ao cruzar os *Proxy-IDs* ativos com o inventário dinâmico do NetBox. Em vez de aplicar uma ação de força bruta, o fluxo em ciclo fechado extrai os blocos CIDR (*Classless Inter-Domain Routing*) legítimos da SSoT e executa o *script* remediate_phase2.py

no ambiente virtual. Este componente IaC, ilustrado na **Figura 9**, utiliza a biblioteca *ipaddress* para converter dinamicamente as sub-redes para o formato sintático exigido pela CLI do FortiOS e injeta as novas diretivas na firewall via SSH/Netmiko, eliminando a configuração antiga e forçando a subida imediata do tráfego.

```
import sys
import time
import ipaddress
from netmiko import ConnectHandler

def convert_to_netmask(cidr_str):

    if "/" in cidr_str:
        ip, mask = cidr_str.split("/")
        if "." in mask:
            return f"{ip} {mask}"
        else:
            net = ipaddress.IPv4Network(cidr_str, strict=False)
            return f"{net.network_address} {net.netmask}"
    return cidr_str

def update_phase2(tunnel_name, p2_name, expected_src, expected_dst):

    fortigate = {
        'device_type': 'fortinet',
        'host': '10.0.0.1',
        'username':'admin',
        'password': 'admin',
        'port': 22,
    }

    src_forti_format = convert_to_netmask(expected_src)
    dst_forti_format = convert_to_netmask(expected_dst)

    print(f"[*] A inicar remediacao de Phase2 para o tunel: {tunnel_name} (P2: {p2_name})")

    try:

        net_connect = ConnectHandler(**fortigate)
        print("[+] Ligacao SSH estabelecida com sucesso")

        config_commands = [
            'config vpn ipsec phase2-interface',
            f'edit "{p2_name}"',
            f'set src-subnet {src_forti_format}',
            f'set dst-subnet {dst_forti_format}',
            'next',
            'end'
        ]

        print(f"[+] A aplicar a config de sub-redes: SRC={src_forti_format} | DST={dst_forti_format}")
```

```
        print(f"[+] A aplicar a config de sub-redes: SRC={src_forti_format} | DST={dst_forti_format}")
        output = net_connect.send_config_set(config_commands)
        print("[+] Config aplicada com sucesso via CLI")
        print(output)


        print("[*] A enviar comando para limpar a Phase 1")
        output_clear = net_connect.send_command_timing(f'diagnose vpn ike gateway clear name {tunnel_name}')
        print(output_clear)

        print("[+] Phase 1 Limpa. A aguardar 5s")
        time.sleep(5)

        print("[*] A forcar a subida do tunel (Phase2)")
        output_up = net_connect.send_command_timing(f'diagnose vpn tunnel up {p2_name}')
        print(output_up)

        print("[*] A dar tempo a FG para negociar o tunel antes de fechar o SSH")
        time.sleep(4)

        net_connect.disconnect()
        return True

    except Exception as e:
        print(f"[-] Erro durante a remediacao SSH: {e}")
        return False

if __name__ == "__main__":
    if len(sys.argv) != 5:
        print("Uso: python3 remediate_phase2.py <Tunnel_Phase1> <Nome_Phase2> <Src_Subnet> <Dst_Subnet>")
        sys.exit(1)

    tunnel_arg = sys.argv[1]
    p2_arg = sys.argv[2]
    src_arg = sys.argv[3]
    dst_arg = sys.argv[4]

    success = update_phase2(tunnel_arg, p2_arg, src_arg, dst_arg)

    if success:
        sys.exit(0)
    else:
        sys.exit(1)
```

Figura 9 - Script Python (remediate_phase2.py) baseado na biblioteca Netmiko para a retificação ativa, tradução sintática de máscaras de rede e remediação configuracional de seletores de tráfego desconfigurados na Phase 2.

A **terceira linha de remediação** corresponde ao estado *zero_traffic*, o ramo terminal da árvore: uma vez excluídos o desvio de gateway e o desalinhamento de seletores, a persistência de contadores a zero com o túnel a reportar administrativamente o estado DOWN indica que a falha pode residir no estado criptográfico da sessão — condição habitualmente designada por túnel *zombie*. Perante este cenário, o n8n executa o *script remediate_tunnel.py*, apresentado na **Figura 10**, que recorre à biblioteca Netmiko para estabelecer uma sessão SSH com a CLI do FortiGate HUB e executar *diagnose vpn ike gateway clear HUB-TO-SPOKE*, limpando as associações de segurança para forçar nova negociação [35],[40].

As credenciais visíveis na figura correspondem ao ambiente laboratorial. A **Fase 2** substitui este mecanismo por autenticação com *token* de API e restrição de *trusted hosts*.

.

```
  GNU nano 7.2                                                remediate_tunnel.py *
import sys
from netmiko import ConnectHandler
def restart_ipsec_tunnel(tunnel_name):
    fortigate = {
        'device_type': 'fortinet',
        'host': '10.0.0.1',
        'username': 'admin',
        'password': 'admin',
        'port': 22,
        'global_delay_factor': 2,
        'disabled_algorithms': {'kex':['diffie-hellman-group14-sha1', 'diffie-hellman-group1-sha1']},
    }
    print(f"[*] A iniciar remediacao (Restart) do tunel IPsec: {tunnel_name}")
    try:
        net_connect = ConnectHandler(**fortigate)
        print("[+] Ligacao SSH estabelecida com sucesso.")
        command = f"diagnose vpn ike gateway clear name {tunnel_name}"
        print(f"[*] A executar comando {command}.")
        output = net_connect.send_command(command)
        print(f"[+] Tunel reiniciado com sucesso via CLI.")
        print(output)
        net_connect.disconnect()
        return True
    except Exception as e:
        print(f"[-] Erro durante o restart via SSH: {e}")
        return False
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Uso: python3 remediate_tunnel.py <Nome_do_Tunel>")
        sys.exit(1)
    tunnel_arg = sys.argv[1]
    success = restart_ipsec_tunnel(tunnel_arg)
    if success:
        sys.exit(0)
    else:
        sys.exit(1)
```

Figura 10 - Script Python (remediate_tunnel.py) baseado na biblioteca Netmiko, para a limpeza das associações de segurança (SAs) e renegociação do túnel IPsec via CLI.

Etapa 4: Closed-Loop Revalidation e Persistência

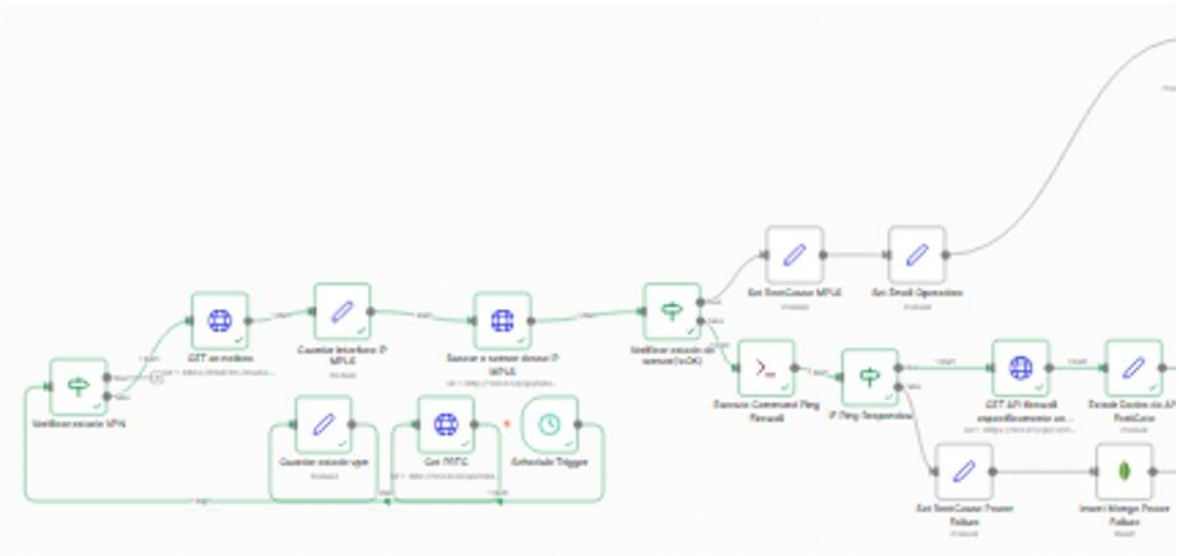
Aplicada a ação corretiva, o orquestrador introduz uma pausa de 12 segundos (nó Wait), destinada a permitir a conclusão da negociação IKE e a instalação das novas associações de segurança [37]. Decorrido este intervalo, o n8n reinterroga o estado do túnel através de nova chamada à REST API da firewall. Se a resposta confirmar o restabelecimento do serviço, o incidente é classificado sob o estado *auto_recovered*; caso a anomalia persista, o fluxo assinala a remediação como falhada, mantendo o incidente sinalizado para análise da equipa. Independentemente do desfecho, os metadados de telemetria e os tempos de execução são consolidados e persistidos na coleção *incident_runs* do MongoDB [38].

De forma a consolidar o mapeamento lógico de todas as variáveis analíticas, condições de contorno da infraestrutura e *playbooks* de remediação descritos, estruturou-se a Matriz de Decisão Operacional apresentada na **Tabela 4**. Este mapa lógico funciona como uma referência determinística para a tomada de decisão do orquestrador, correlacionando os alarmes recebidos com a exata tipologia de falha isolada e a correspondente ação de autocura aplicada.

Tabela 4 - Matriz de Decisão Operacional do Orquestrador n8n.

Estado PRTG	Estado MPLS (PRTG)	Ping Spoke (exitCode)	API REST FGT: Indicadores	Ação Corretiva Executada	Estado Final MongoDB
DOWN	Offline	(Não executado)	(Abortado)	Enviar Alerta SMTP ao Operador	escalated_to_provider
DOWN	Online	exitCode≠0	(Abortado)	Registar equipamento inacessível	power_or_hardware_failure
DOWN	Online	exitCode==0	zero_traffic	SSH: Limpeza de SAs via Netmiko (remediate_tunnel.py)	auto_recovered
DOWN	Online	exitCode==0	phase2_mismatch	SSH: Retificação ativa de sub-redes (remediate_phase2.py)	auto_recovered
DOWN	Online	exitCode==0	rgwy_mismatch	SSH: Correção de Gateway (remediate_config.py)	auto_recovered

A leitura analítica da **Tabela 4** demonstra que o orquestrador apenas avança para as rotinas críticas de *SSH Fallback* após superar com êxito as validações cumulativas das etapas anteriores. Este controlo estrito neutraliza o risco de injeção de instruções IaC em equipamentos inoperantes ou isolados da rede WAN. A consolidação prática de todas as regras e subestados lógicos culmina com o desenho do fluxo de automação dentro do ambiente de desenvolvimento. A **Figura 11** exibe o *canvas* detalhado do *workflow* construído na plataforma n8n, revelando a modularidade e o encadeamento sequencial dos nós de integração, triagem preventiva e autorremediação ativa que governam a arquitetura.



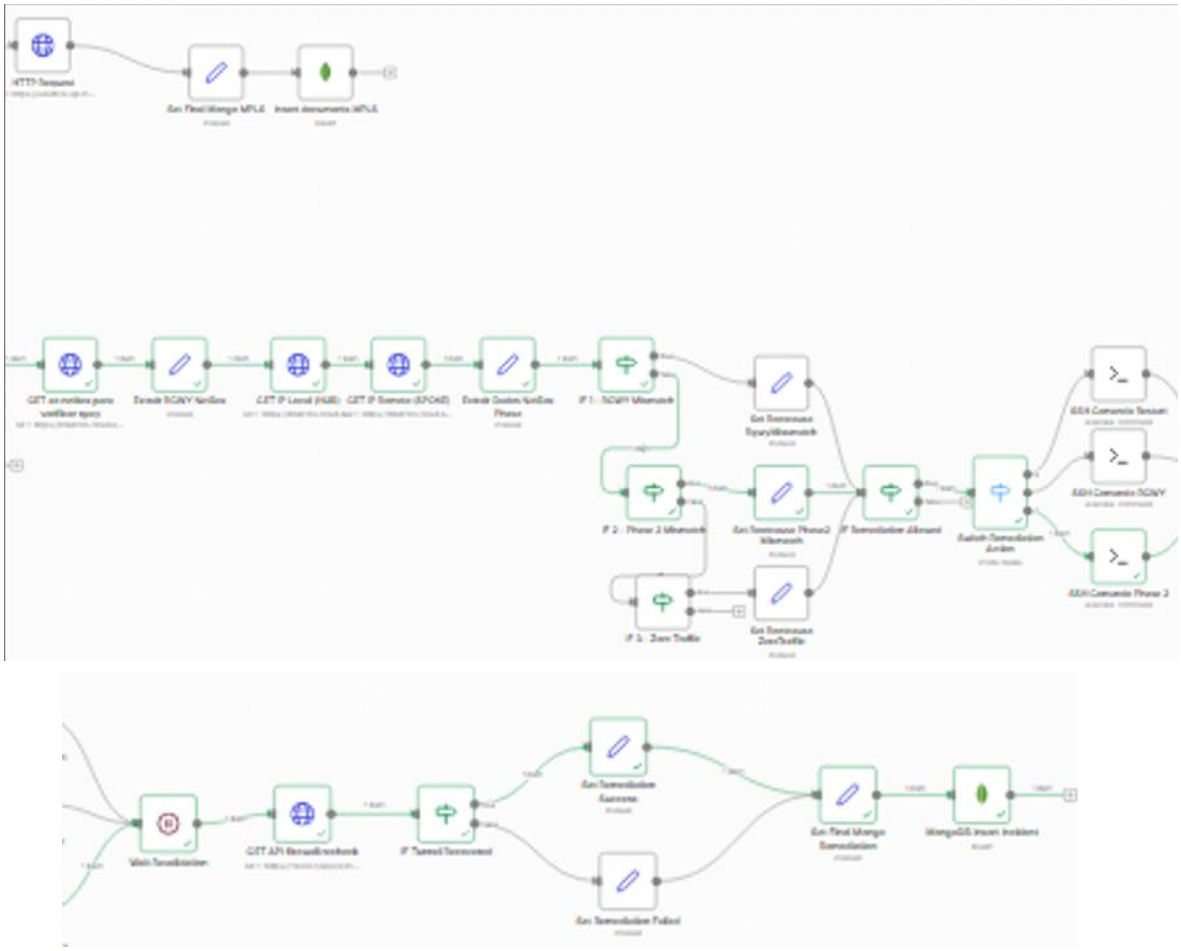


Figura 11 - Canvas completo do fluxo de automação e orquestração em ciclo fechado (Closed-Loop) atualizado na plataforma n8n, evidenciando a árvore de triagem sequencial e o triplo ramo de atuação IaC (Restart, RGWY e Phase 2).

2.3.3 Fase 2: orquestração enterprise em ciclo fechado (REST API Nativa e Event-Driven)

A transição para a **Fase 2** foi impulsionada pela introdução das licenças comerciais fornecidas pela ARTE, I.P., ultrapassando definitivamente os bloqueios de escrita na base de dados de configuração (CMDB) do FortiOS. Ao viabilizar o uso irrestrito dos métodos HTTP de alteração de estado (como POST e PUT), a arquitetura prescindiu da necessidade de aplicar comandos de terminal via SSH (Fase 1), migrando para uma integração nativa e síncrona assente na REST API do fabricante.

Sobre esta nova abordagem, o fluxo de automação do n8n comunica diretamente com os *endpoints* formais do FortiOS, envolvendo a total eliminação de *scripts* Python locais no servidor e contornando os constrangimentos do isolamento de bibliotecas impostos pelo PEP 668. De modo a acomodar esta expansão de capacidades e delimitar as fronteiras de controlo síncrono sobre HTTPS, a topologia evoluiu para a arquitetura apresentada na **Figura 12**.

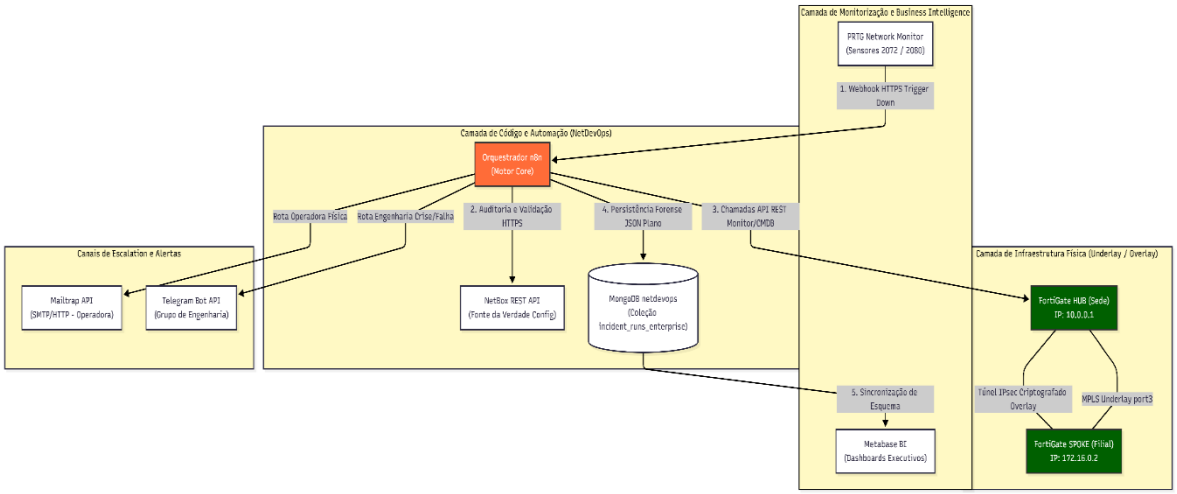


Figura 12 - Diagrama de blocos da macro-arquitetura e topologia da infraestrutura global licenciada (Fase 2).

A comunicação e a recolha de telemetria operam agora com recurso a requisições síncronas HTTPS sobre a porta padrão 443, consumindo dados estruturados em formato JSON. O processamento subsequente, a simplificação das estruturas NoSQL complexas e a persistência dos dados no MongoDB para posterior consumo e análise visual no Metabase seguem o fluxo de transformação ilustrado na **Figura 13**, salvaguardando a integridade das informações sem introduzir latência ou impactar o desempenho dos equipamentos.

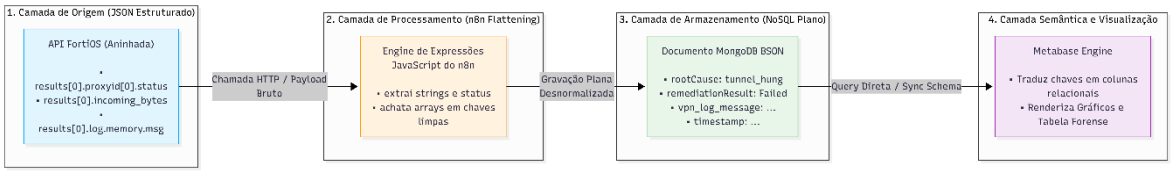


Figura 13 - Pipeline de dados: mapeamento, transformação, achatamento (flattening) e visualização analítica (Fase 2).

O mapeamento detalhado e o estado de prontidão operacional das interfaces físicas e lógicas licenciadas no ambiente laboratorial encontram-se documentados na **Figura 14** e na **Figura 15**.

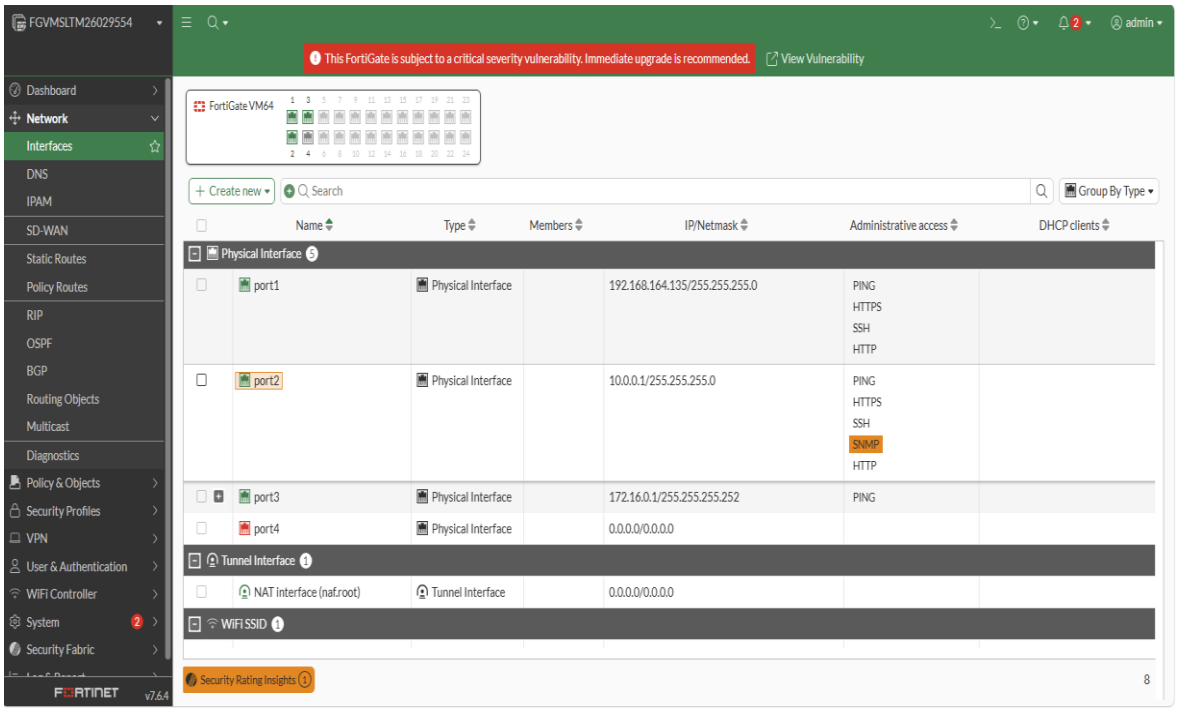


Figura 15 - Painel de visualização das interfaces físicas e lógicas no FortiGate HUB sob licenciamento comercial (Fase 2).

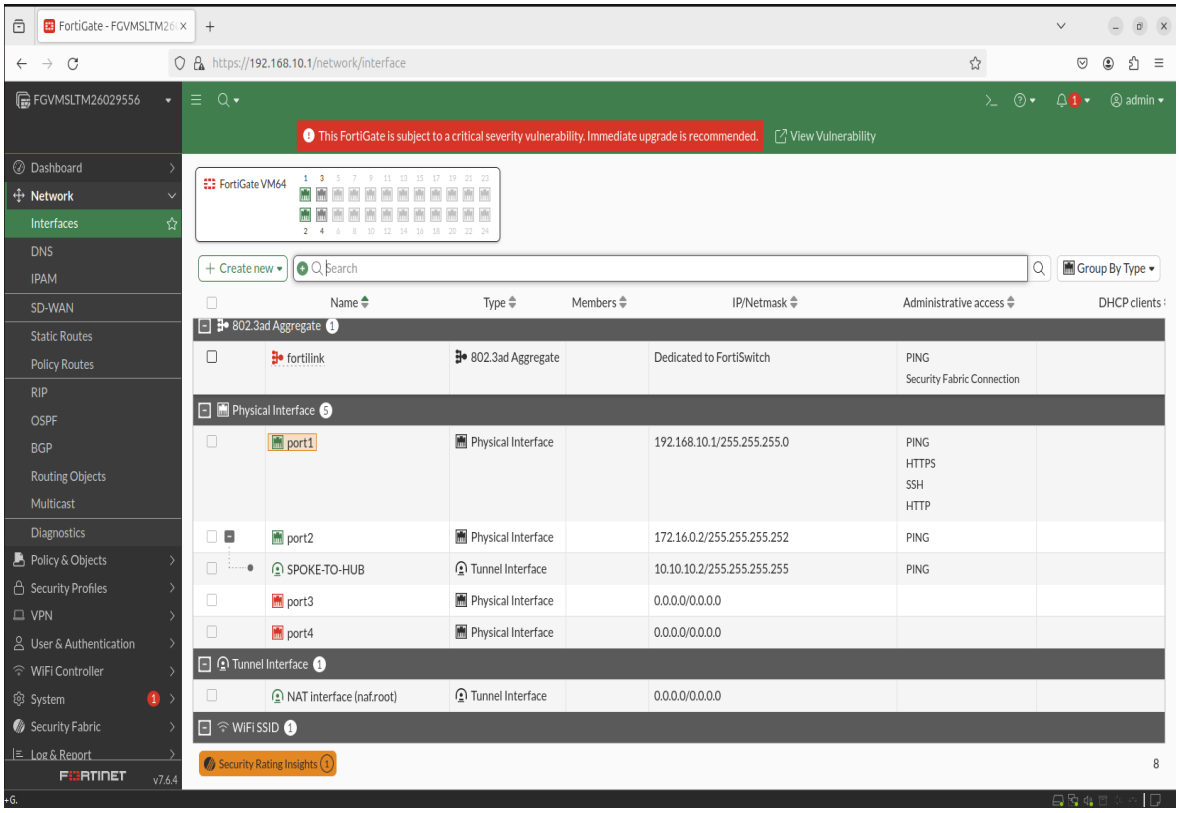


Figura 14 - Painel de visualização das interfaces e sub-interfaces lógicas de túnel no FortiGate SPOKE (Fase 2).

O novo *pipeline* do orquestrador foi reestruturado sob uma árvore de decisão que prioriza a validação de conformidade com o NetBox antes de instanciar qualquer procedimento de reinicialização. O sistema valida em sequência o gateway remoto, os seletores de *Phase 2*, políticas certas e avança para a auditoria e atribuição do endereçamento ponto a ponto virtual no *overlay* (gama $10.10.10.0/30$), cuja parametrização explícita na interface lógica de túnel é ilustrada na **Figura 16**. Este encadeamento lógico estruturado e o fluxo de decisão baseado em API encontram-se detalhadamente descritos no diagrama de micro-arquitetura da **Figura 17.**

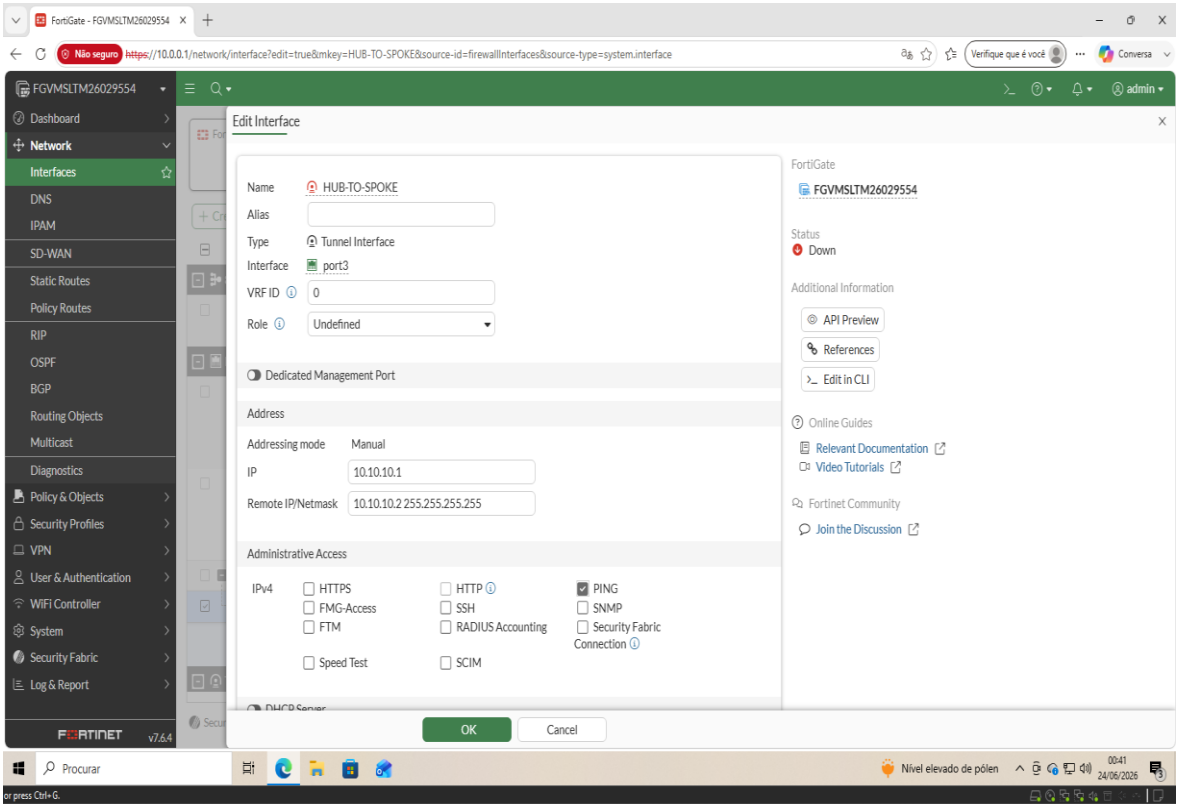
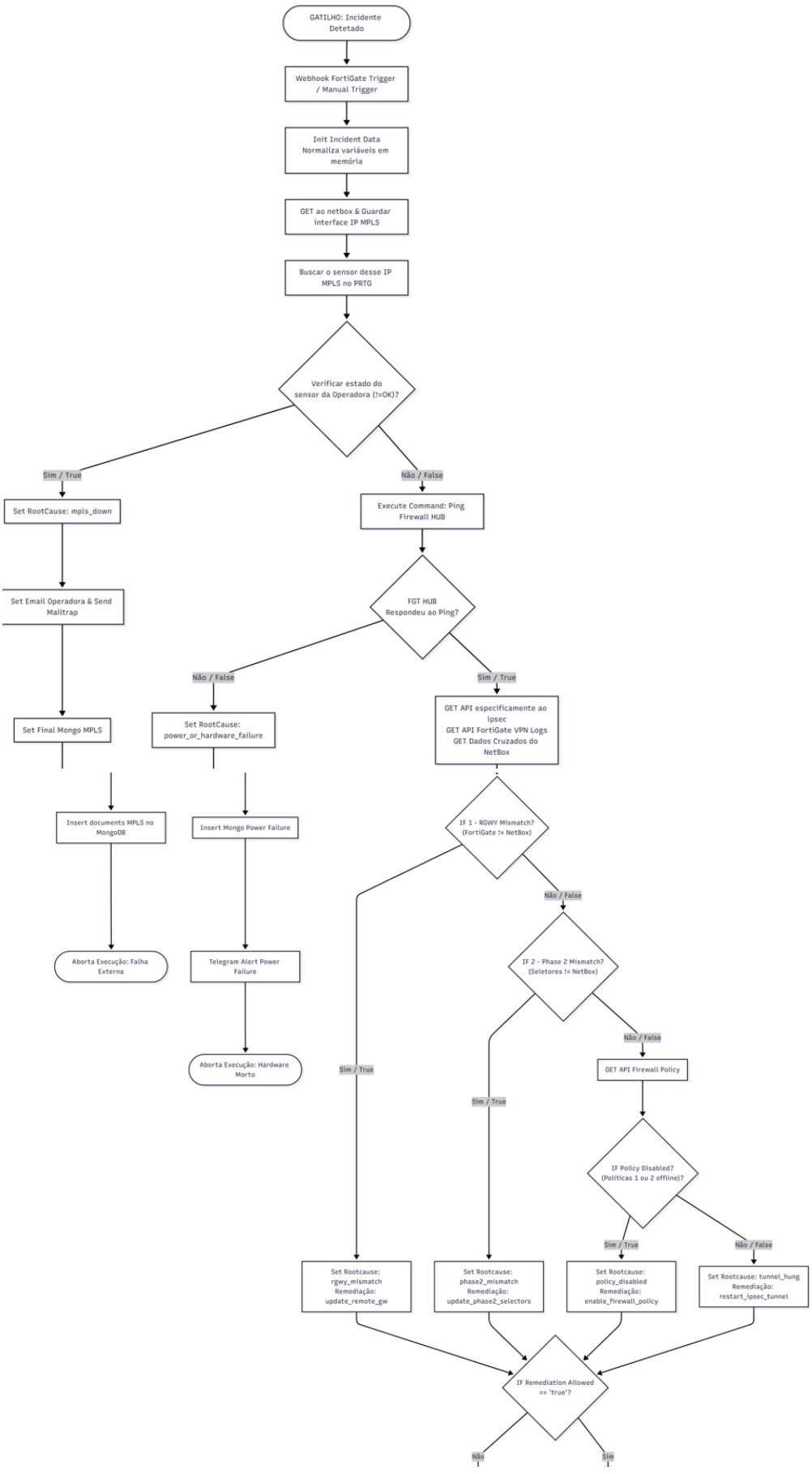


Figura 16 - Configuração detalhada de endereçamento ponto-a-ponto Overlay na interface lógica de túnel HUB-TO-SPOKE (Fase 2).


GATILHO: Incidente Detetado
Webhook FortiGate Trigger / Manual Trigger
Init Incident Data Normaliza variáveis em memória
GET ao netbox & Guardar interface IP MPLS
Buscar o sensor desse IP MPLS no PRTG
Verificar estado do sensor da Operadora (!=OK)?
Sim / True
Não / False
Set RootCause: mpls_down
Set Email Operadora & Send Mailtrap
Set Final Mongo MPLS
Insert documents MPLS no MongoDB
Aborta Execução: Falha Externa
Execute Command: Ping Firewall HUB
FGT HUB Respondeu ao Ping?
Não / False
Sim / True
Set RootCause: power_or_hardware_failure
Insert Mongo Power Failure
Telegram Alert Power Failure
Aborta Execução: Hardware Morto
GET API especificamente ao ipsec
GET API FortiGate VPN Logs
GET Dados Cruzados do NetBox
IF 1 - RGWY Mismatch? (FortiGate != NetBox)
Sim / True
Não / False
IF 2 - Phase 2 Mismatch? (Seletores != NetBox)
Sim / True
Não / False
GET API Firewall Policy
IF Policy Disabled? (Políticas 1 ou 2 offline)?
Sim / True
Não / False
Set Rootcause: rgwy_mismatch Remediação: update_remote_gw
Set Rootcause: phase2_mismatch Remediação: update_phase2_selectors
Set Rootcause: policy_disabled Remediação: enable_firewall_policy
Set Rootcause: tunnel_hung Remediação: restart_ipsec_tunnel
IF Remediation Allowed == 'true'?
Não
Sim

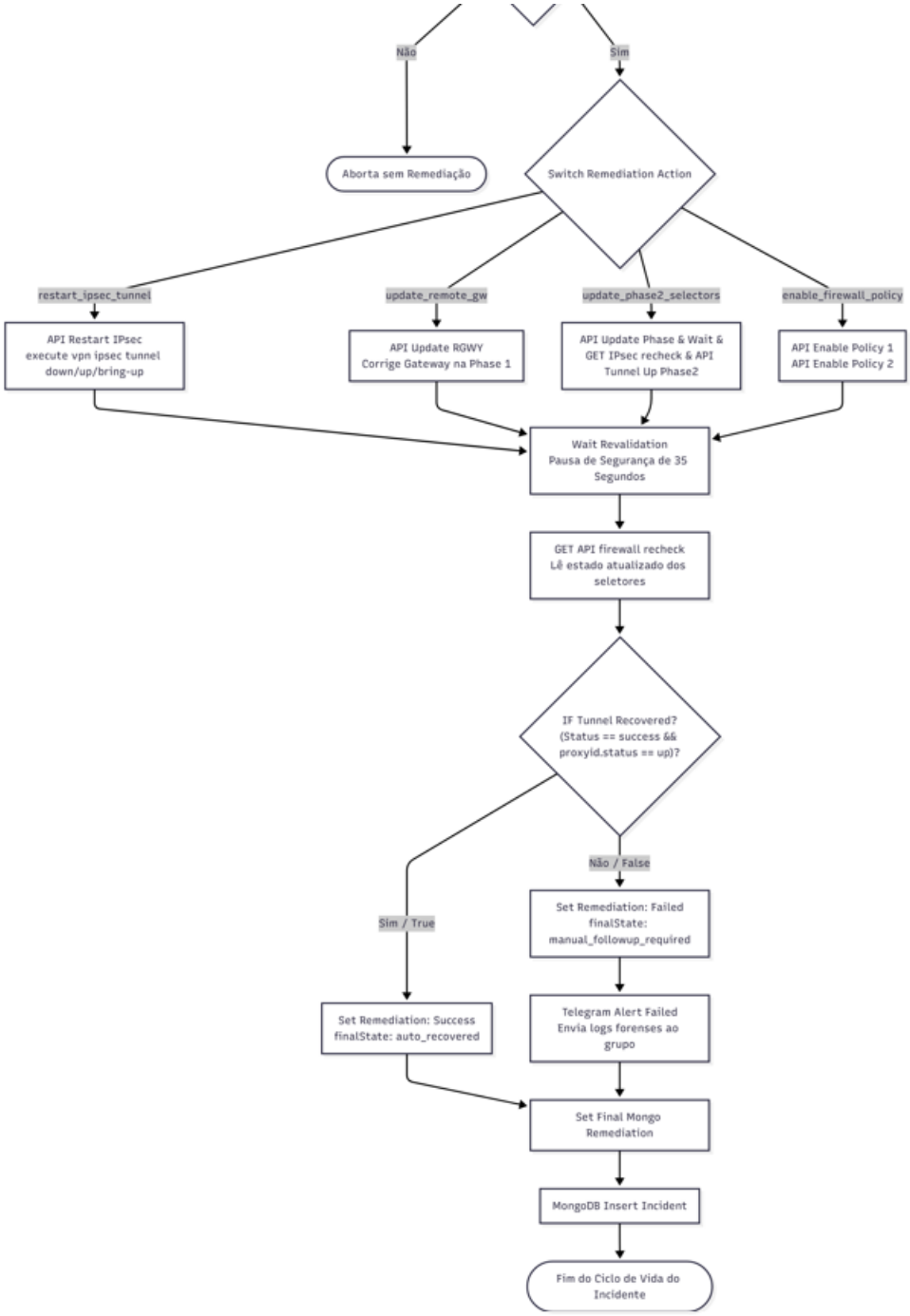


Figura 17 - Micro-arquitetura lógica, árvore analítica e pipeline de decisão baseada em API nativa no n8n (Fase 2).

Adicionalmente, a API REST monitoriza os parâmetros de emparelhamento do *underlay* e o comportamento do protocolo *Dead Peer Detection* (DPD) configurado em modo *On Demand*. Este controlo garante que as tabelas de *Phase 1* no HUB e no SPOKE apontam com rigor para as interfaces físicas corretas da rede de transporte MPLS (VMnet5), conforme detalhado na **Figura 18** e na **Figura 19**.

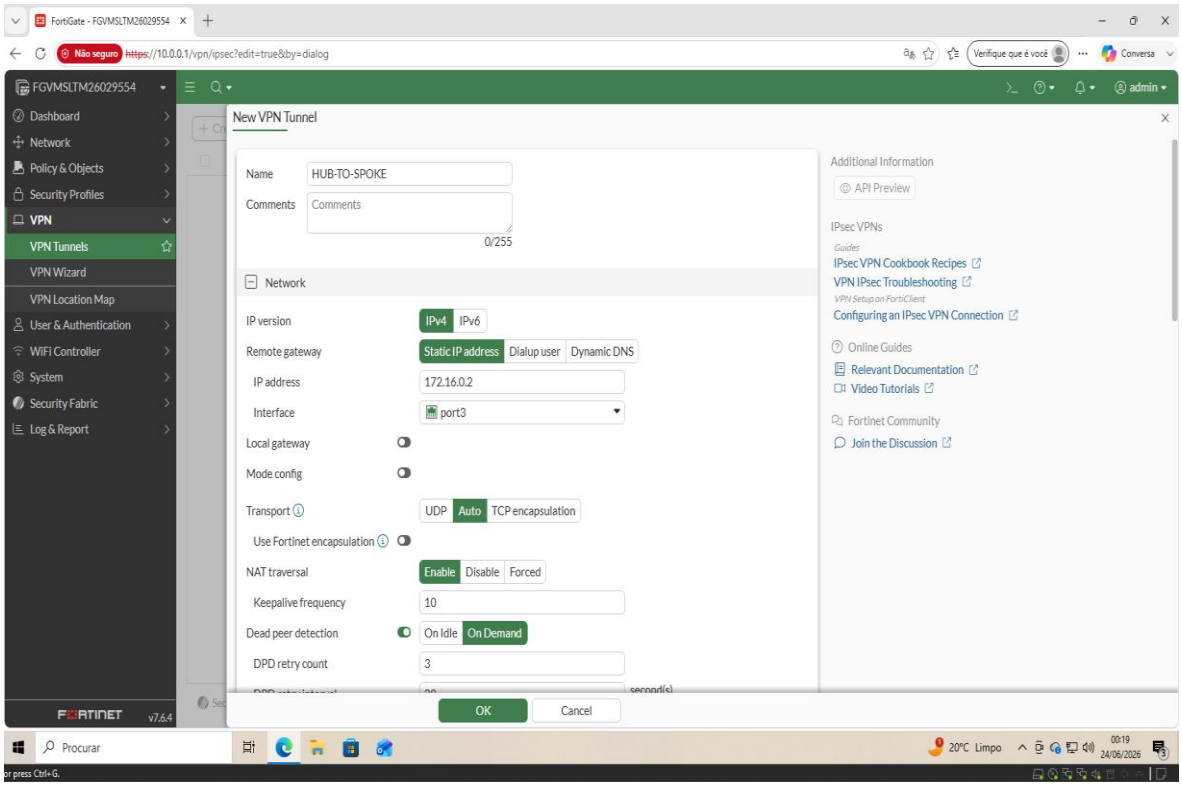


Figura 18 - Parâmetros lógicos de rede da Phase 1 do túnel HUB-TO-SPOKE apontando para o gateway físico do Spoke (Fase 2).

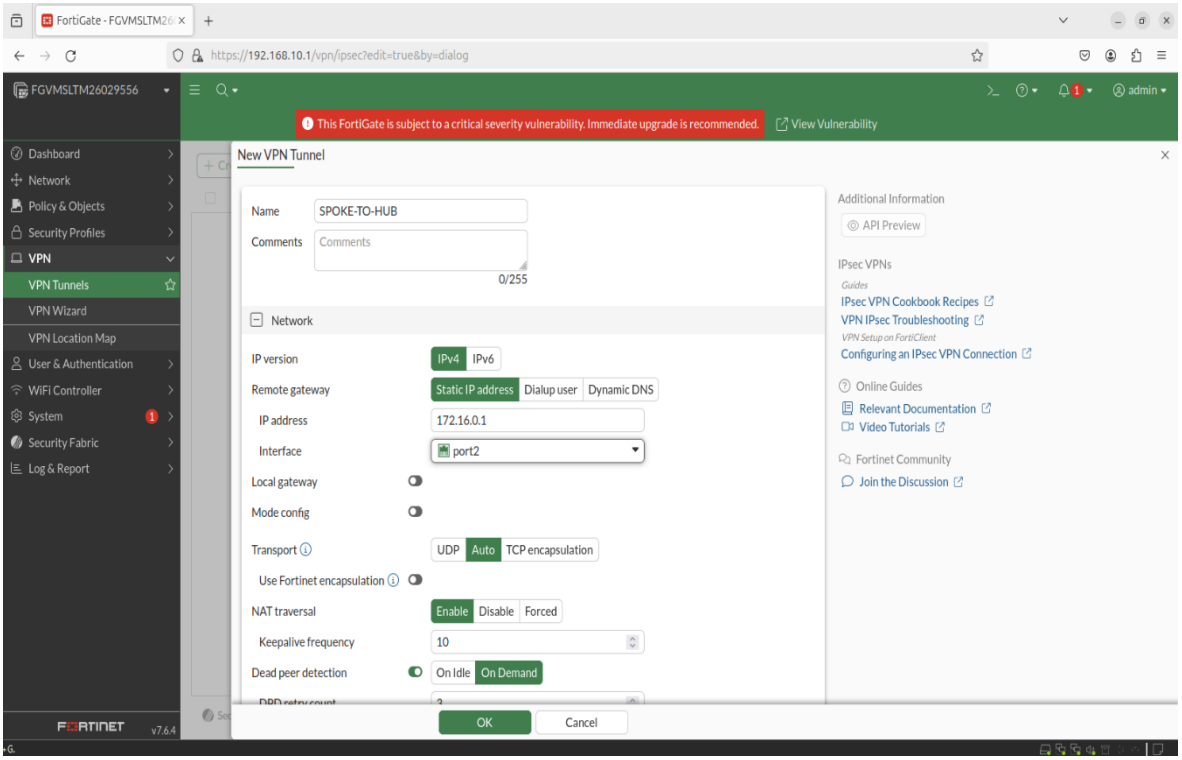


Figura 19 - Parâmetros lógicos de rede da Phase 1 do túnel SPOKE-TO-HUB apontando para o gateway físico do Hub (Fase 2).

A simetria dos parâmetros criptográficos entre os dois extremos foi estabelecida e verificada manualmente na fase de montagem do laboratório, constituindo a linha de base sobre a qual o orquestrador opera. Ambas as extremidades foram parametrizadas com autenticação por chave pré-partilhada, IKE versão 2 e propostas de *Phase 1* idênticas (AES128/AES256 com SHA256 e as variantes GCM com PRFSHA256/384), com o grupo Diffie-Hellman 20 selecionado em ambos os lados. As **Figuras 20 e 21** documentam esta parametrização no HUB e no SPOKE, respetivamente. A validação automática destes parâmetros pela REST API não foi implementada no âmbito deste projeto, ficando identificada como evolução futura no Eixo 1.

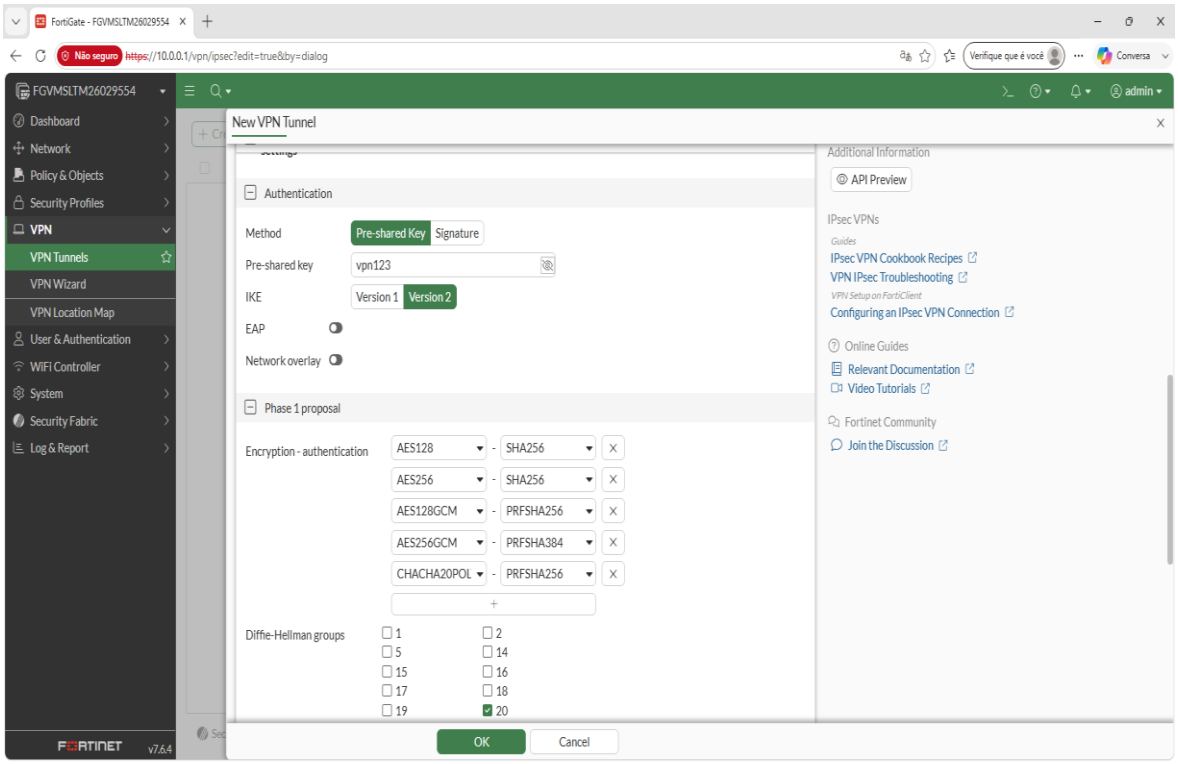


Figura 20 - Parametrização de autenticação e propostas criptográficas da Phase 1 IKEv2 no FortiGate HUB.

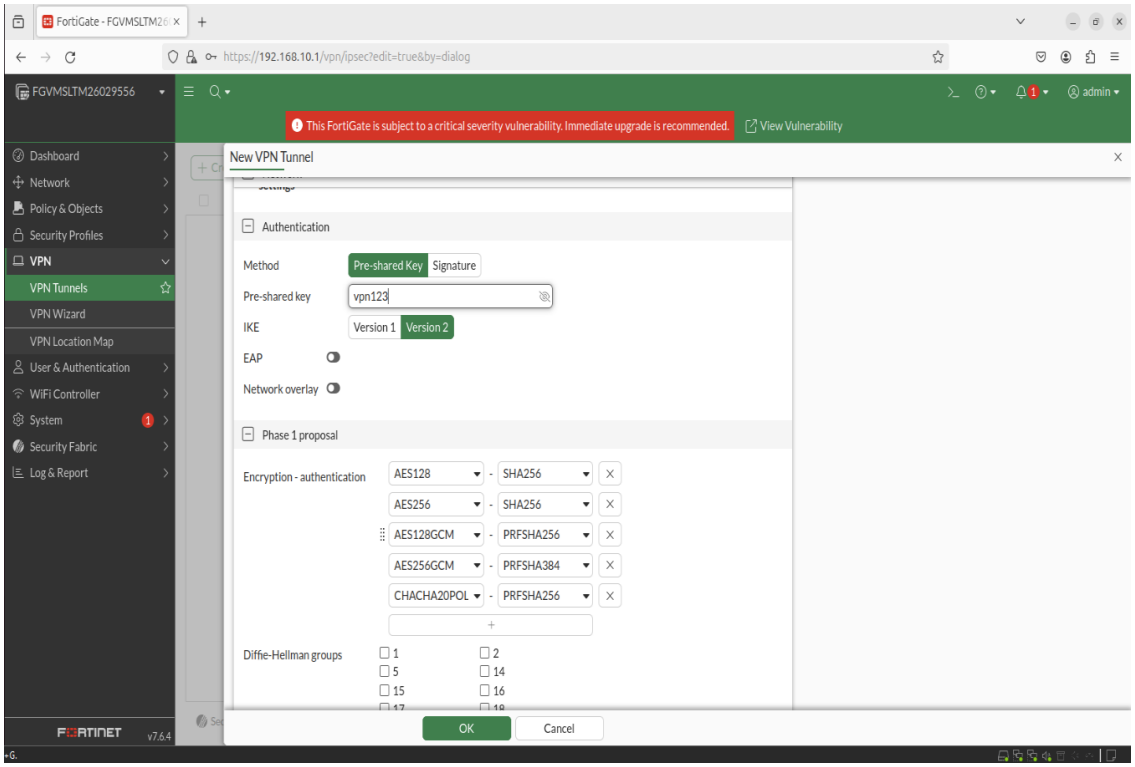


Figura 21 - Especificações de autenticação, chaves e propostas criptográficas espelhadas da Phase 1 IKEv2 no FortiGate SPOKE.

Validada a conformidade do plano de controlo na *Phase 1*, o orquestrador inspeciona os seletores de tráfego (*Proxy-IDs*) parametrizados na *Phase 2*. Os nós GET IP Local (HUB) e GET IP Remote (SPOKE) obtêm do NetBox os blocos legítimos e o nó condicional IF 2 - Phase 2 Mismatch compara-os com os campos proxy_src e proxy_dst devolvidos pela REST API da firewall, confirmando que a gama local 10.0.0.0/24 está mapeada contra a gama remota 192.168.10.0/24. Esta verificação deteta desalinhamentos de sub-rede que provocam falhas silenciosas de negociação, conforme ilustrado nas **Figuras 22 e 23**.

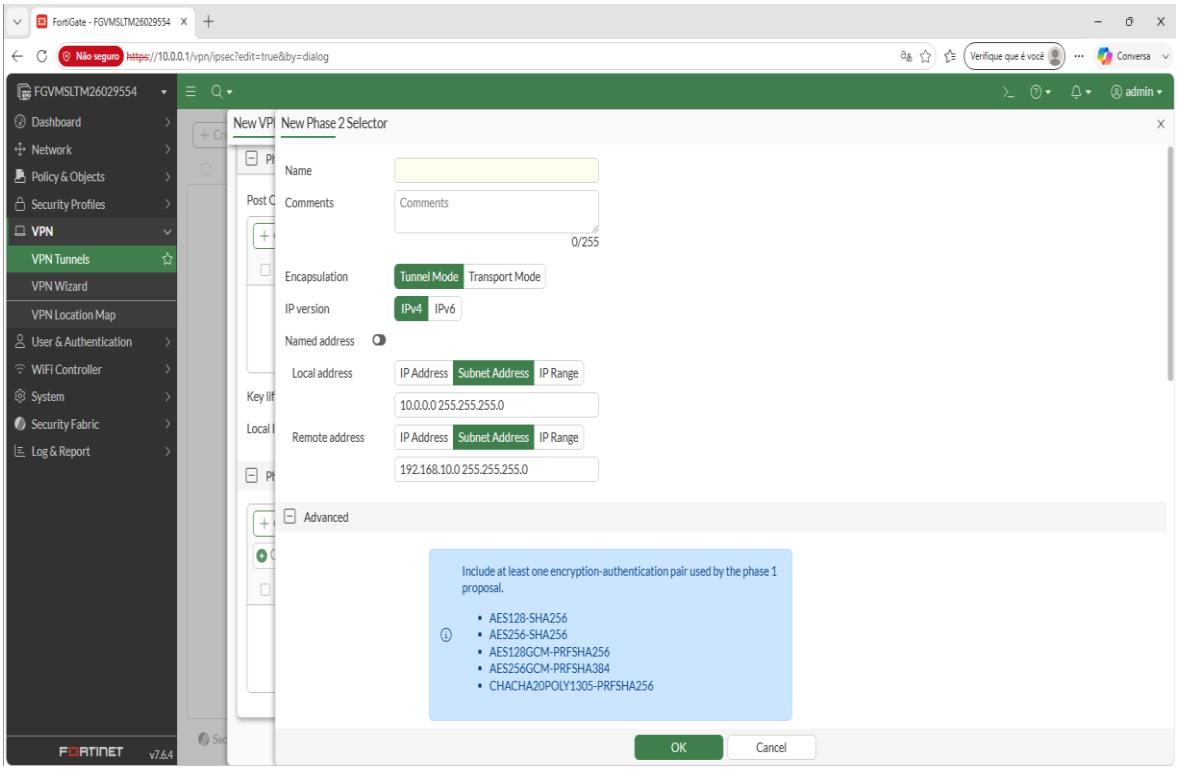


Figura 22 - Janela de criação e revisão dos seletores de tráfego (Proxy-IDs) locais e remotos da Phase 2 no nó HUB (Fase 2).

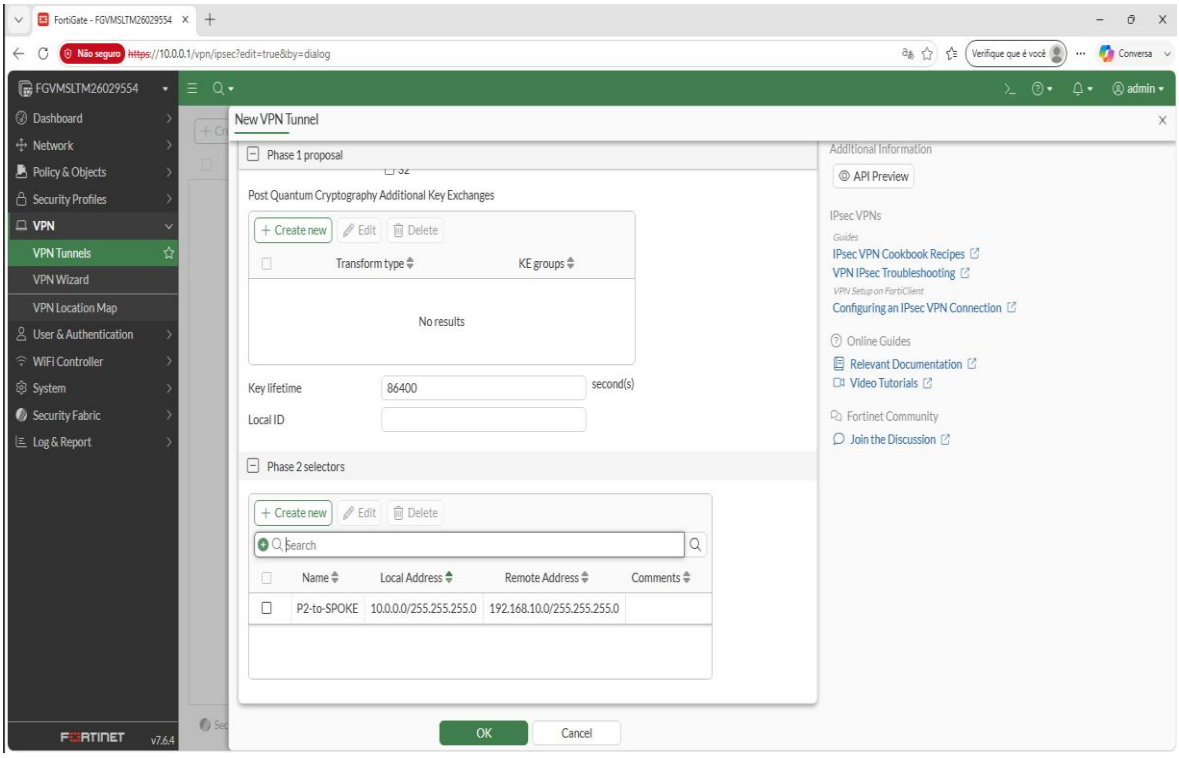


Figura 23 - Mapeamento detalhado dos blocos CIDR legítimos para a rede local e remota na Phase 2 selectors do FortiGate HUB (Fase 2).

O encaminhamento entre as duas delegações assenta em rotas estáticas simétricas configuradas manualmente em ambos os extremos. No HUB, a rota para a LAN remota 192.168.10.0/24 aponta para a interface de túnel *HUB-TO-SPOKE*; no SPOKE, a rota reversa para a LAN central 10.0.0.0/24 aponta para a interface *SPOKE-TO-HUB*. Ambas foram parametrizadas com distância administrativa 10, em correspondência com a **Tabela 3** e conforme documentado nas **Figuras 24 e 25**. Esta configuração constitui um pressuposto do funcionamento do túnel e não é verificada pelo orquestrador, uma vez que a auditoria automática se concentra nos parâmetros mais sujeitos a alteração manual em contexto operacional — o *gateway* remoto, os seletores de *Phase 2* e as regras de filtragem.

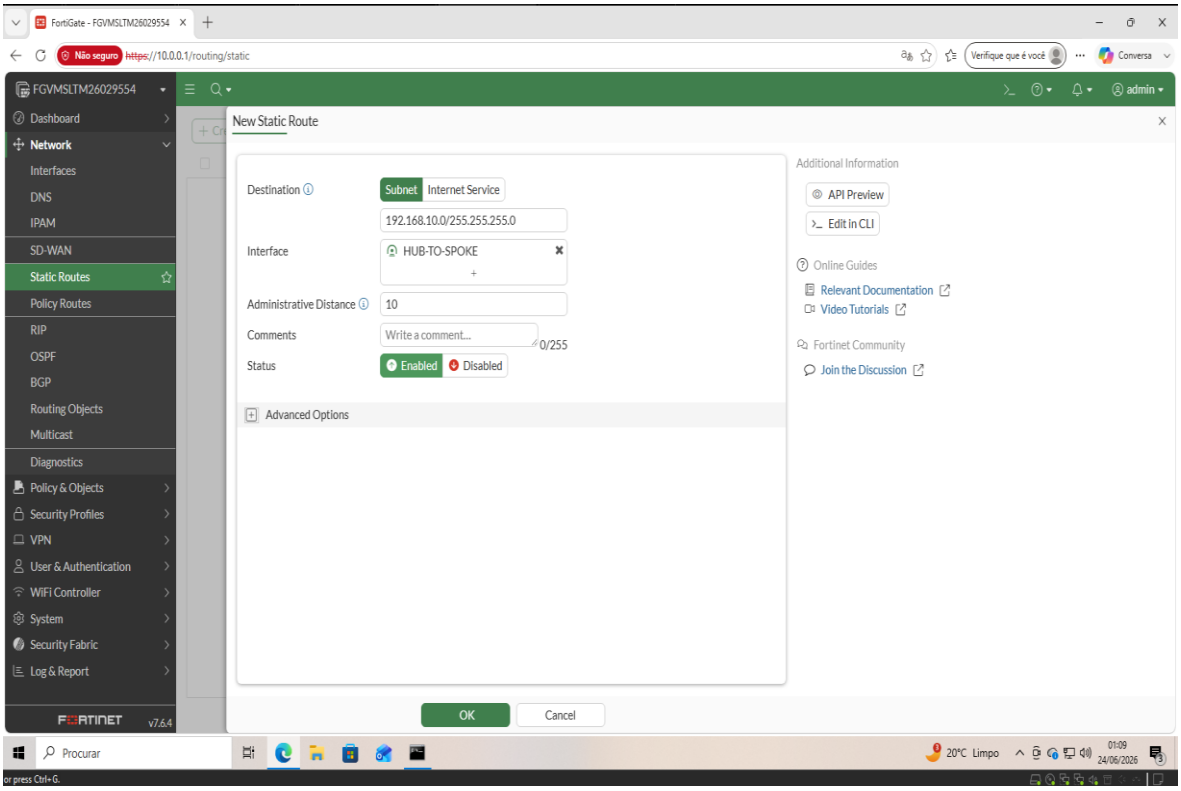


Figura 24 - Configuração da rota estática no FortiGate HUB direcionando pacotes da sub-rede destino para a interface do túnel (Fase 2).

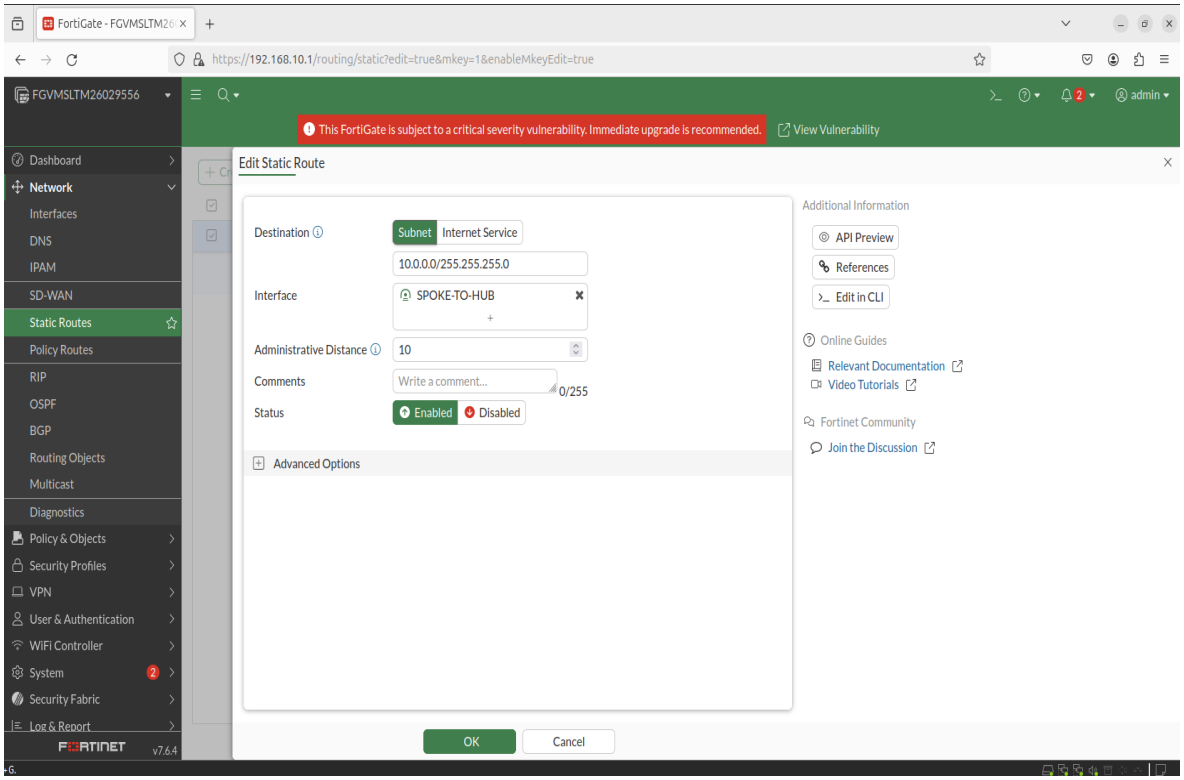


Figura 25 - Configuração da rota estática simétrica no FortiGate SPOKE direcionando pacotes para o gateway central da Sede (Fase 2).

De seguida, o orquestrador audita as regras de filtragem consultando o *endpoint* global */api/v2/cmdb/firewall/policy/*. O nó condicional *IF Policy Disabled* corre uma expressão JavaScript que avalia se a Política ID 1 (LAN-to-SPOKE) ou a Política ID 2 (SPOKE-to-LAN) se encontram inativas no HUB.

As definições de aceitação de tráfego, serviços autorizados e interfaces de trânsito para ambos os ativos estão documentadas sequencialmente nas **Figuras 26, 27, 28, 29, 30, 31, 32 e 33.**

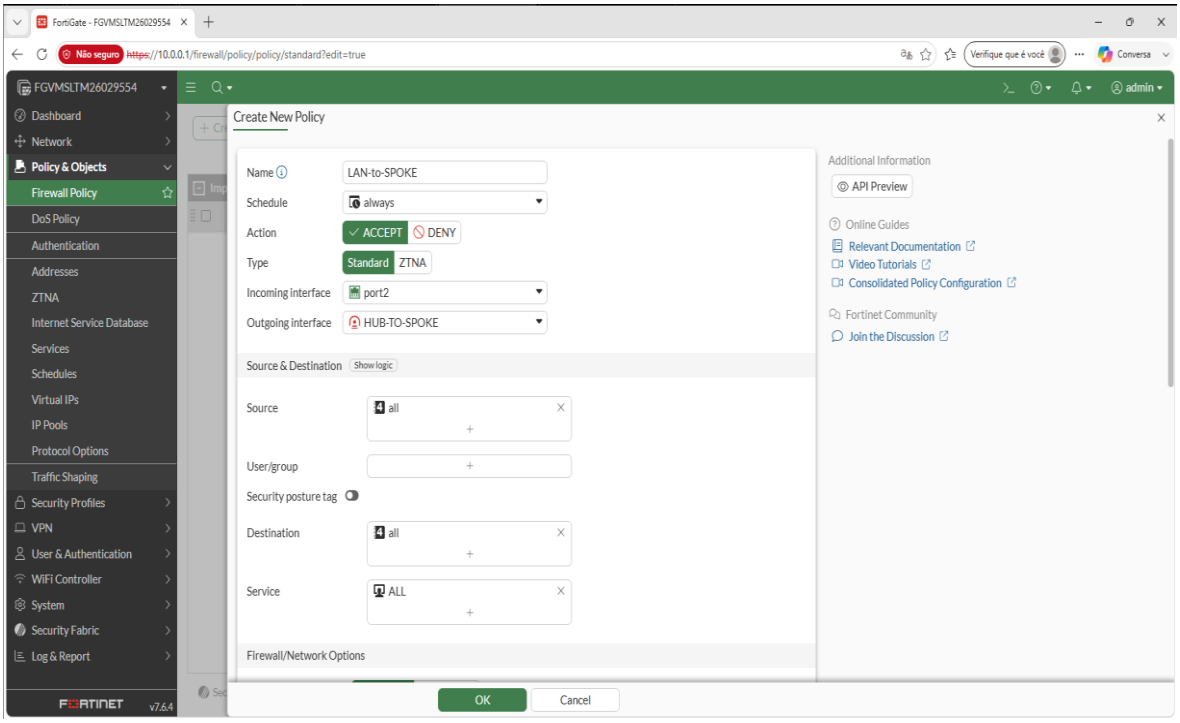


Figura 26 - Parametrização da política de firewall avançada de aceitação de tráfego de saída LAN-to-SPOKE no FortiGate HUB (Fase 2).

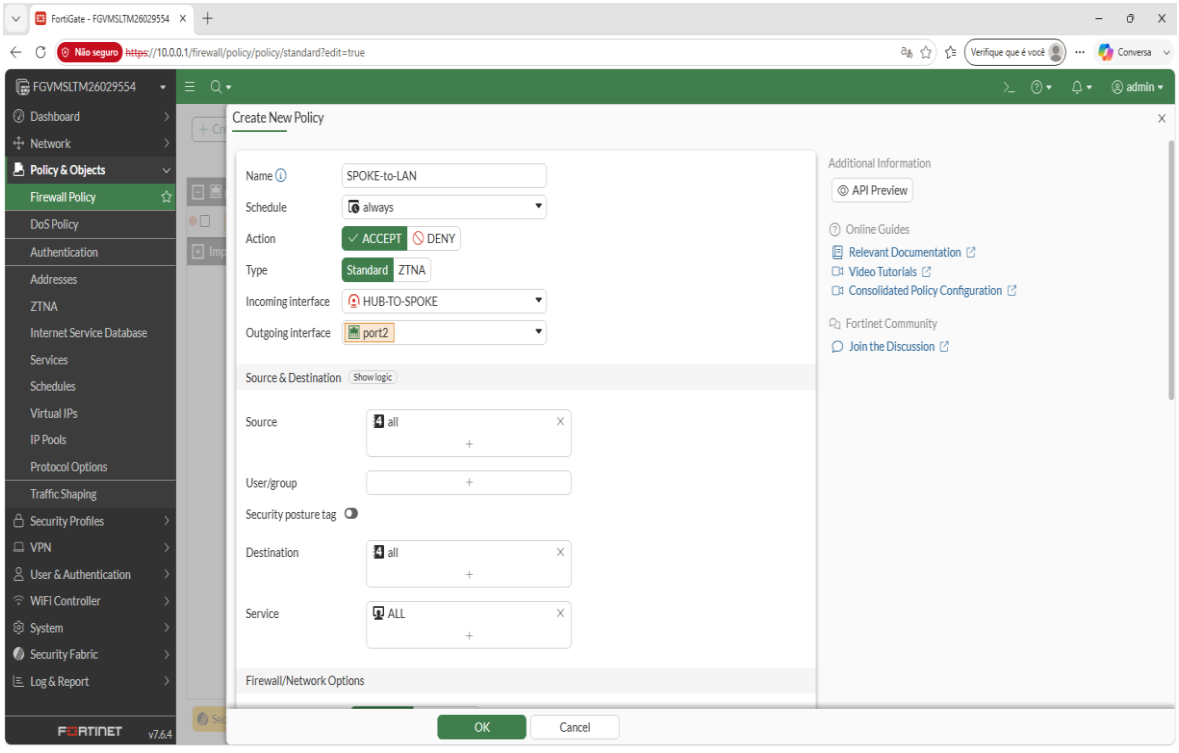


Figura 27 - Parametrização da política de firewall de retorno/entrada SPOKE-to-LAN no FortiGate HUB (Fase 2).

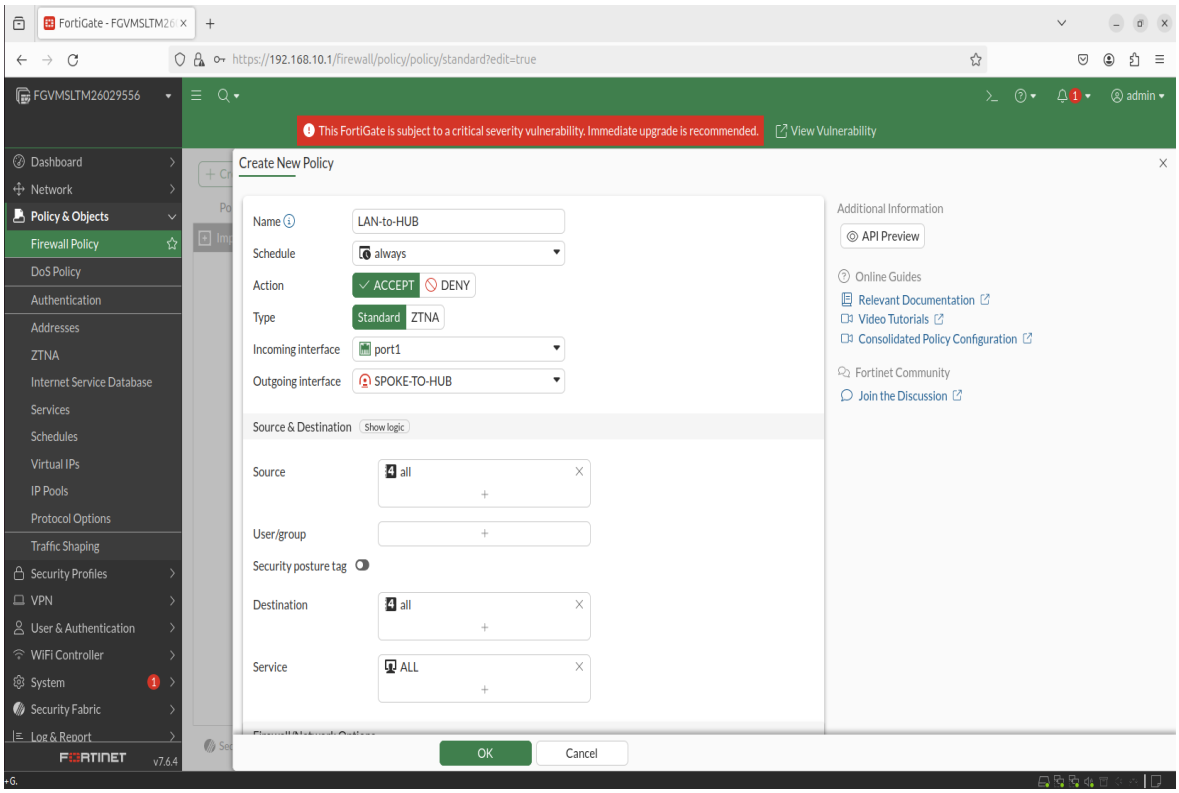


Figura 28 - Parametrização da política de firewall avançada de aceitação de tráfego de saída LAN-to-HUB no FortiGate SPOKE (Fase 2).

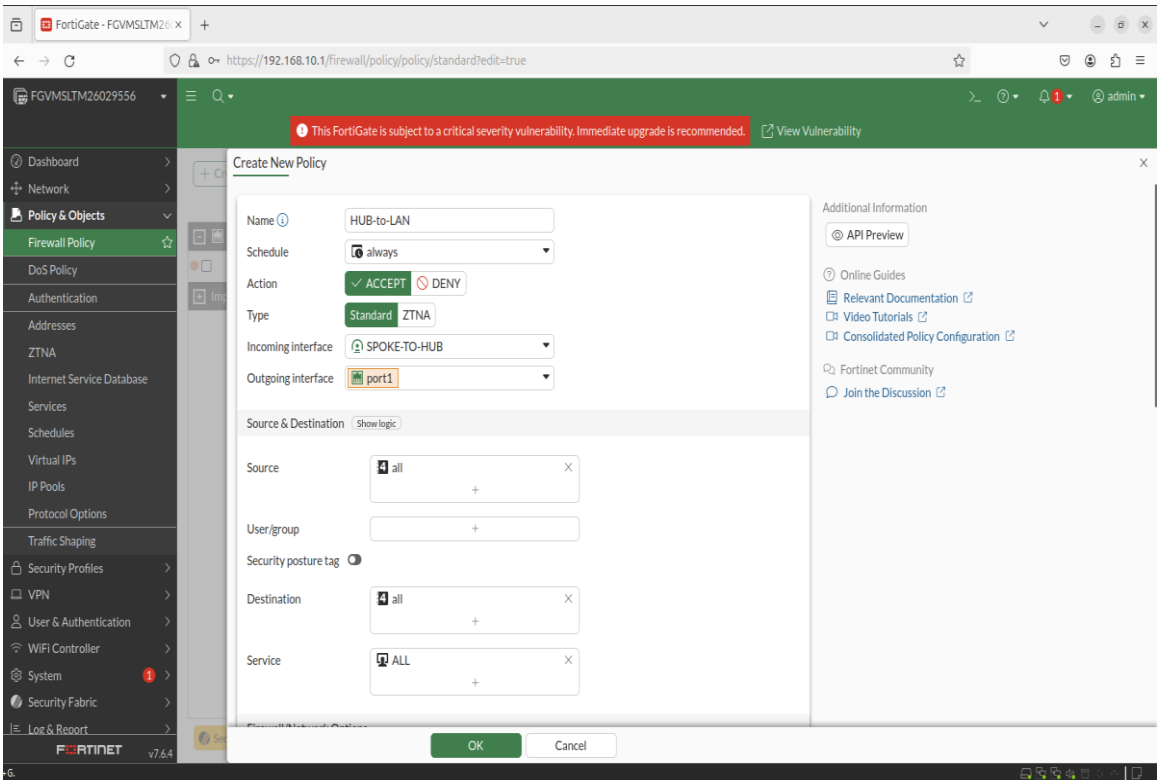


Figura 29 - Parametrização da política de firewall avançada de aceitação de tráfego de saída LAN-to-HUB no FortiGate SPOKE (Fase 2).

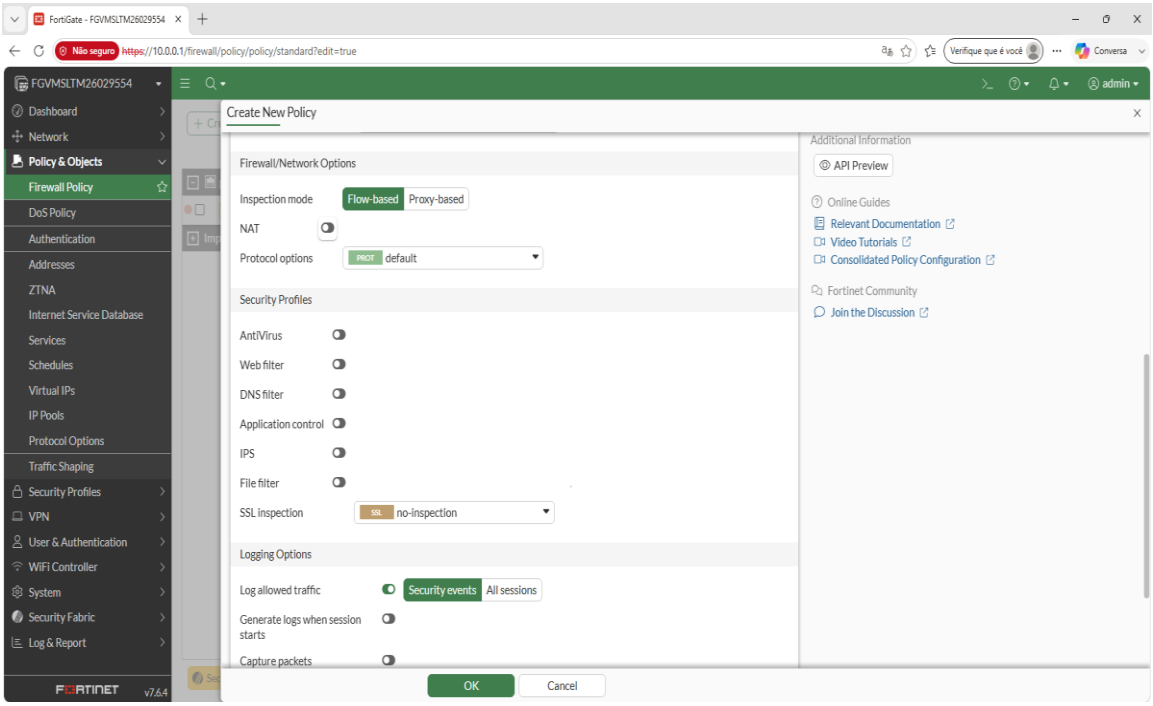


Figura 30 - Detalhe das opções de Firewall/Network Options e Logging de tráfego na política 1 do FortiGate HUB (Fase 2).

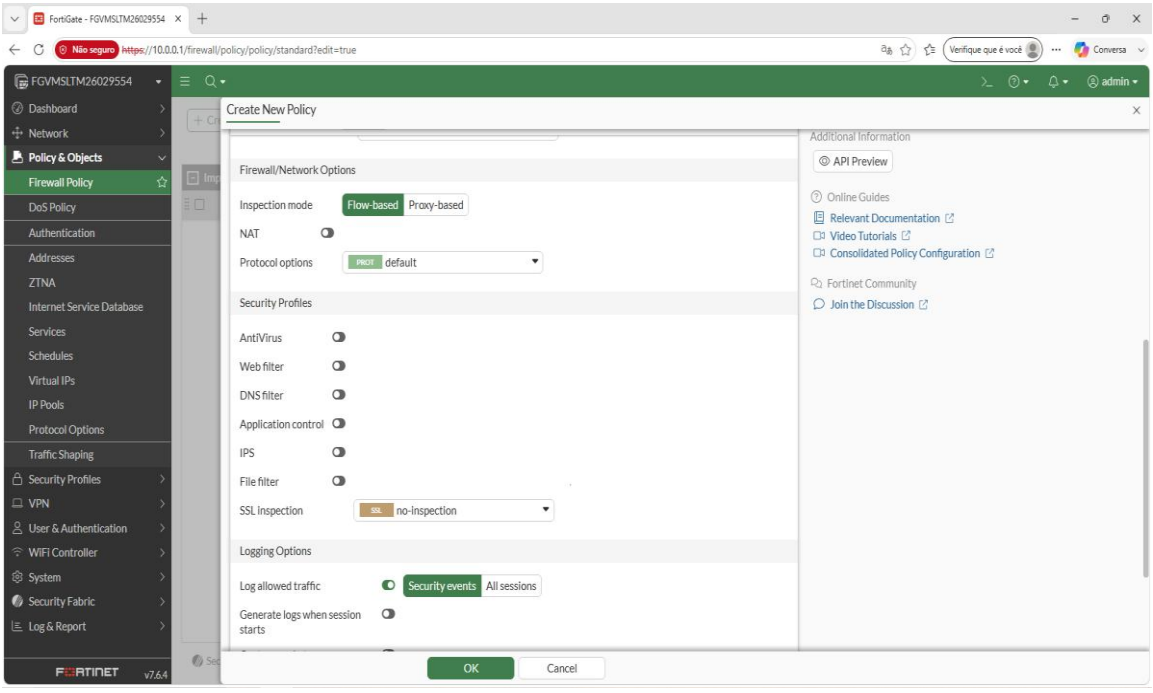


Figura 31 - Detalhe das opções avançadas de inspeção Flow-based e Logging de sessões na política 2 do FortiGate HUB (Fase 2).

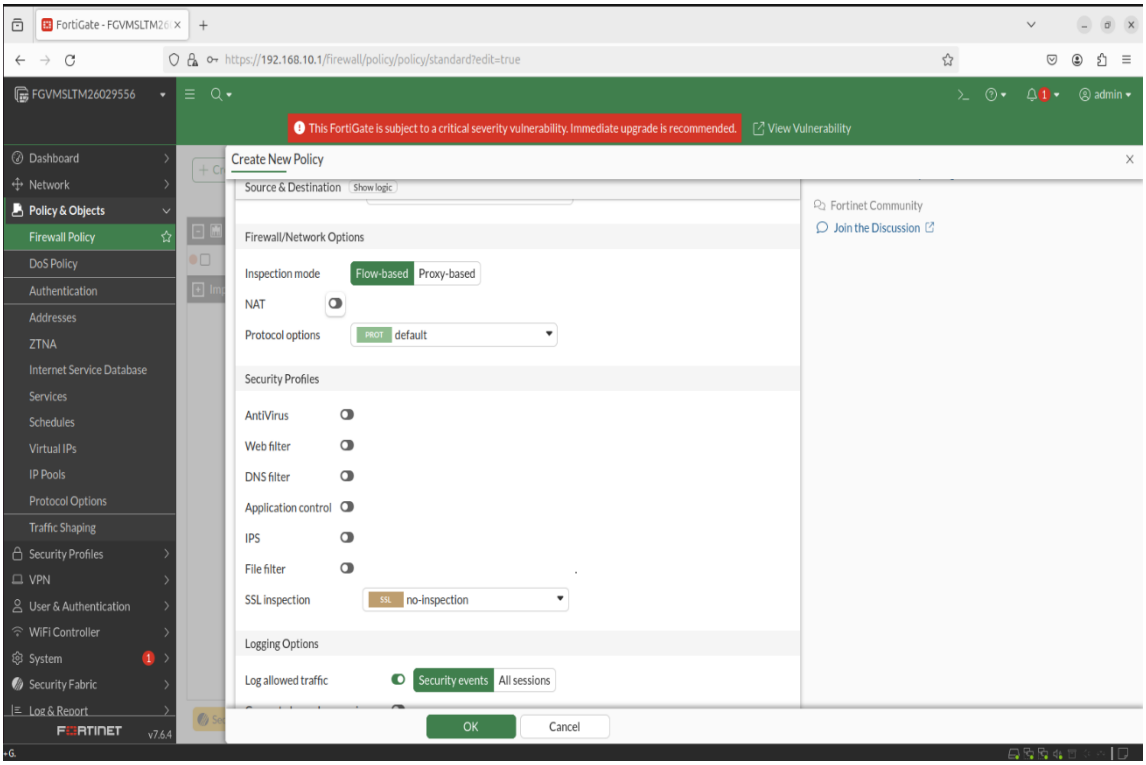


Figura 32 - Detalhe das opções de Firewall/Network Options e Logging de tráfego na política 1 do FortiGate SPOKE (Fase 2).

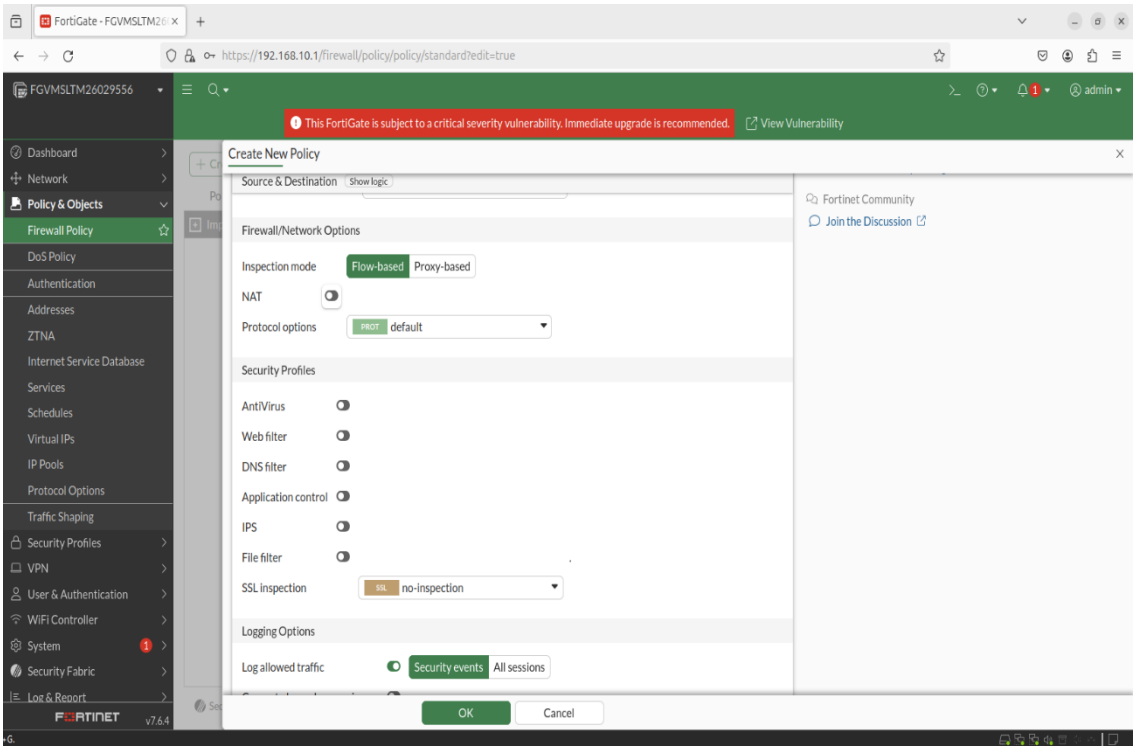


Figura 33 - Detalhe das opções avançadas de inspeção Flow-based e Logging de sessões na política de segurança 2 do FortiGate SPOKE (Fase 2).

O n8n executa requisições HTTP PUT síncronas para reativar as políticas de segurança desativadas, injetando o *payload* JSON *{"status": "enable"}* diretamente na CMDB da firewall sempre que o workflow assim o detectar. Se todos os parâmetros de conformidade física e lógica se encontrarem corretos, mas a VPN persistir inativa, a árvore de decisão classifica a anomalia sob o estado *tunnel_hung* (túnel congelado). Nestas condições, o orquestrador despacha uma requisição HTTP POST operativa que força a imediata renegociação de chaves e o restabelecimento dos seletores de tráfego de *Phase 2*.

Concluída a intervenção, o ciclo fecha com um intervalo técnico de convergência de 35 segundos para estabilização das novas associações de segurança (SAs). O orquestrador valida então o sucesso da operação e persiste o registo do incidente na coleção *incident_runs_enterprise* do MongoDB, garantindo a total auditabilidade da arquitetura NetDevOps [1].

2.3.4 Comparação técnica e operacional entre as fases 1 e 2

A transição metodológica entre o mecanismo de contingência assente em CLI (**Fase 1**) e a integração programática por APIs (**Fase 2**) expõe diferenças profundas ao nível da resiliência, da segurança e do desempenho da infraestrutura da ARTE, I.P. Enquanto o primeiro modelo constituiu uma resposta tática a uma barreira de licenciamento experimental, a segunda etapa consolidou um modelo de automação em ciclo fechado (*Closed-Loop Automation*) alinhado com as melhores práticas de Engenharia de Redes. A **Tabela 5** sistematiza e confronta as propriedades operativas e técnicas de ambos os cenários validados em laboratório.

Tabela 5 - Matriz Comparativa Tecnológica e Operacional (Fase 1 vs Fase 2)

Dimensão Técnica / Operatica	Fase 1: Licença Trial & SSH Fallback	Fase 2: Licença Comercial & REST API Nativa
Protocolo de Transporte	Sessões interativas SSH (Porta TCP 22) com emulação de terminal virtual.	Chamadas síncronas HTTPS (Porta TCP 443) sobre TLS de alta segurança.
Formatação de Payloads	Injeção de strings em texto limpo (sintaxe CLI proprietária do FortiOS).	Payloads JSON puros, normalizados, tipificados e validados pelo fabricante.
Modelo de Execução (n8n)	Baixo código (Low-Code) através do nó *Execute Command* e scripts Python.	Sem código (Zero-Code) assente em nós nativos *HTTP Request* do orquestrador.
Acionador de Automação	*Polling* reativo condicional baseado em amostragem temporal fixa (Janela de 10s).	Webhooks em tempo real acionados pelo PRTG, com resposta síncrona e atómica via API REST.

Eficiência do MTTR	Lenta (Latência de abertura de sessão SSH e processamento sequencial de texto).	Ultra-rápida (Deteção instantânea e execução síncrona na escala dos milissegundos).
Granularidade Forense	Indireta. Avaliação condicional baseada no sintoma (Bytes de tráfego a zero).	Direta. Extração cirúrgica de logs forenses em tempo real direto da memória da firewall.
Segurança das Credenciais	Elevada exposição (Utilizador/Senha administrativos *hardcoded* nos scripts locais).	Minimizada (Tokens de API REST associados a diretivas estritas de Trusted Hosts).
Âmbito de Atuação (Escrita)	Destrutivo (Reset e *flush* total do estado criptográfico da Phase 1/Phase 2).	Configuracional (Alteração pontual de Remote-GW e reativação cirúrgica de políticas).
Persistência de Dados	Coleção comum incident_runs volátil e desprovida de logs do sistema operativo.	Coleção dedicada incident_runs_enterprise sincronizada com o Metabase BI.

A análise comparativa consolidada na **Tabela 5** evidencia a evolução da arquitetura deste sistema. Na **Fase 1**, o sistema dependia de um mecanismo de *polling* estático e a uma lógica puramente baseada no sintoma: se o fluxo de tráfego caísse para zero, a automação assumia de pronto uma anomalia criptográfica e aplicava uma ação invasiva de *flush* (ike gateway clear), interrompendo o túnel sem necessidade quando a verdadeira causa poderia ser um desvio configuracional introduzido por ação humana (tal como uma política de segurança inativa). Sob a perspetiva de segurança, o armazenamento de credenciais de administração em texto limpo nos *scripts* Python do servidor de gestão representaria uma vulnerabilidade de conformidade (compliance) grave face aos requisitos da norma ISO 27001 em ambiente produtivo real.

Por outro lado, a **Fase 2** corporiza o estado da arte das redes autorregenerativas (active self-healing). Ao integrar os alertas em tempo real via webhooks do PRTG diretamente com nós HTTP Request nativos no orquestrador, o Tempo Médio de Resolução (MTTR) é significamente reduzido.

A inteligência operacional do orquestrador foi reforçada. O n8n corre verificações preventivas via REST API, validando os parâmetros operacionais face ao NetBox e auditando as regras de filtragem. Isto confere-lhe a capacidade de restabelecer pontualmente as políticas ID 1 ou ID 2 por meio de chamadas HTTP PUT síncronas.

A postura de cibersegurança foi reforçada ao adotar tokens de autenticação restritos ao endereço IP do servidor Ubuntu (10.0.0.60) através de diretivas de

trusted hosts. No plano analítico, o Metabase foi capacitado para exibir não apenas métricas genéricas de interrupção, mas sim gráficos complexos de causa-raiz baseados na recolha de dados forenses extraídos em tempo real da memória do dispositivo. Enquanto o mecanismo de SSH Fallback da **Fase 1** preserva um papel secundário como plano de contingência para cenários de falha na API Web, a arquitetura da **Fase 2** consolida de forma inequívoca o modelo programático, seguro e auditável exigido para a salvaguarda dos canais de comunicação da Administração Pública.

2.4 Metodologia de injeção de falhas

A validação da arquitetura desenvolvida assentou na injeção controlada de avarias em ambiente laboratorial, segundo os princípios da Engenharia do Caos (Chaos Engineering) [18]. A abordagem consiste em provocar condições de falha num sistema em funcionamento, para observar se os mecanismos de recuperação atuam como projetado e se a triagem distingue corretamente as causas-raiz.

As falhas foram introduzidas por três vias distintas, consoante a camada. As avarias físicas do transporte foram simuladas no hipervisor VMware Workstation Pro, desativando a rede virtual VMnet5 que materializa o circuito MPLS entre o HUB e o SPOKE. As interrupções de alimentação nas delegações remotas foram reproduzidas através da paragem forçada (Power Off) da máquina virtual que aloja o FortiGate SPOKE. Os desvios de configuração foram introduzidos manualmente na interface gráfica do FortiOS, alterando parâmetros que, em contexto operacional, são efetivamente sujeitos a intervenção humana: o endereço do gateway remoto, os seletores de tráfego da Phase 2 e o estado das regras de filtragem.

Foram definidos dez cenários, cinco por cada fase do projeto. Os cenários A a E exercitam o mecanismo de contingência por SSH da **Fase 1**: quebra do *underlay* MPLS (A), desvio de configuração nos seletores da *Phase 2* (B), ausência de tráfego no plano de dados com a configuração conforme (C), desvio no *gateway* remoto (D) e inacessibilidade do equipamento remoto (E). Os cenários F a J submetem a arquitetura **Enterprise** às mesmas classes de avaria, agora com remediação por chamadas REST API nativas: quebra de *underlay* externo (F), inacessibilidade do equipamento remoto (G), desvio no *gateway* remoto (H), desvio nos seletores da *Phase 2* (I) e desativação de regras de filtragem (J) — esta última uma verificação que a Fase 1 não possuía

Esta correspondência entre os dois conjuntos é deliberada: submeter ambas as arquiteturas às mesmas condições permite atribuir as diferenças de desempenho ao mecanismo de atuação, e não à natureza da falha injetada.

Os ensaios realizados, as evidências recolhidas em cada cenário e a análise dos registos do FortiOS são apresentados no **Capítulo 3**, **secção 3.3.**

2.5 Síntese da implementação

A implementação do modelo bidirecional e autónomo estruturado ao longo deste capítulo consolidou, na vertente operacional, a aplicação prática dos conceitos de NetDevOps e os princípios fundamentais de Engenharia de Fiabilidade de Redes (*Site Reliability Engineering* ou SRE) à infraestrutura híbrida da ARTE, I.P. Toda a arquitetura foi agregada sob uma *stack* homogénea de contentores Docker [26], executada num servidor Ubuntu Server 24.04. Esta abordagem simplificou a centralização de rotinas de monitorização, a execução de diagnósticos forenses profundos e a aplicação cirúrgica de ações autocorretivas perante anomalias na camada física ou lógica. Todo o processo de mitigação decorre de forma autónoma e sem a necessidade de intervenção humana direta, assegurando em simultâneo a rastreabilidade integral das modificações efetuadas para fins de conformidade e auditoria regulamentar [18].

No núcleo funcional do protótipo, o orquestrador n8n estabelece a ponte de comunicação entre as diversas plataformas de gestão através da manipulação direta de objetos JSON. O motor processa os eventos de monitorização enviados pelo PRTG, interroga e valida os parâmetros no NetBox (que atua como repositório canónico) e orquestra a injeção de configurações nas firewalls. Estes procedimentos de remediação utilizam interfaces REST API nativas ou sessões SSH administradas programaticamente via Netmiko em ambientes virtuais isolados, salvaguardando a integridade das bibliotecas do sistema em total conformidade com a diretiva PEP 668 [27].

Os ensaios conduzidos segundo a metodologia de Engenharia do Caos confirmaram a eficácia prática e a estabilidade da arquitetura desenvolvida. O protótipo revelou-se altamente eficaz na capacidade de detetar, isolar e reverter anomalias operacionais e físicas — estendendo-se desde cortes de energia locais nas filiais até desalinhamentos de segurança na *Phase 2* e ocorrências de túneis zombies. Ao identificar com precisão a causa-raiz de cada perturbação e executar respostas na escala dos segundos, o sistema reduz o tempo de inatividade operacional a limites mínimos, protegendo de forma proativa os canais digitais da ARTE e salvaguardando os níveis de resiliência exigidos na administração pública [32].

3. Resultados

A migração de uma postura de monitorização puramente reativa para o controlo proativo de infraestruturas críticas de telecomunicações da Administração Pública exige um escrutínio minucioso sobre os dados obtidos na fase experimental. Este capítulo detalha a análise comparativa e a discussão dos resultados obtidos após a implementação da arquitetura bidirecional em ciclo fechado. O foco operacional do sistema sofreu uma alteração profunda. Enquanto a triagem passiva desenvolvida na **Fase 1** se limitava a otimizar o tempo médio de deteção por via de mecanismos tradicionais de alarmística, o protótipo atualizado na **Fase 2** demonstra o seu valor na redução acentuada do tempo médio de resolução através de *playbooks* de autocura automatizados assentes em chamadas REST API nativas.

Os ensaios práticos realizados centram-se na avaliação de métricas de fiabilidade recolhidas de forma contínua. Toda a telemetria gerada pelas ferramentas é persistida sob uma arquitetura de dados estruturada no MongoDB, transitando depois para a camada de *Business Intelligence* através do Metabase para a exposição visual dos indicadores. Com esta abordagem analítica conclui-se com precisão a estabilidade real do ecossistema sob o prisma da engenharia de fiabilidade de redes (SRE) e do movimento NetDevOps, sem deixando para trás, a total transparência e a auditabilidade sobre qualquer ação corretiva executada na infraestrutura híbrida nacional.

3.1 Análise quantitativa e impacto no MTTR

A avaliação quantitativa do desempenho da arquitetura autónoma constitui o pilar fundamental para medir o ganho real de eficiência operacional alcançado no ecossistema da ARTE, I.P. Esta análise foca-se nas métricas temporais que governam o ciclo de vida dos incidentes, confrontando de forma direta a demora e uma possível inconsistência do modelo manual legado com as respostas puramente determinísticas obtidas nas simulações de laboratório.

Para estruturar esta avaliação de forma rigorosa e progressiva, o impacto da automação na contração do MTTR foi segmentado de acordo com as duas etapas evolutivas que caracterizam o desenvolvimento da solução. Em primeiro lugar, examina-se o comportamento do mecanismo de contingência por emulação de terminal, estabelecendo a linha de base (*baseline*) de otimização viável em ambientes de recursos restritos. Na sequência evolutiva, determina-se o ganho de velocidade e segurança transacional proporcionado pela injeção declarativa direta via API corporativa. As subsecções seguintes detalham o comportamento matemático, os limites algorítmicos e os resultados empíricos registados em cada um destes cenários.

3.1.1 Fase 1: Otimização reativa no ambiente Trial (SSH Fallback)

A avaliação do impacto quantitativo gerado pelos ciclos de automação ativa na infraestrutura de serviços públicos torna-se mais claro quando confrontado com o modelo operacional manual legado. Na abordagem manual tradicional, qualquer incidente simples de conectividade na malha de túneis causava quebras na produtividade devido ao desgaste provocado pela constante mudança de contexto (*context switching*). Logo após receber a notificação de uma falha, o operador humano precisava de pensar de onde e porque motivo surgiu tal problema. Este fluxo de trabalho manual resultava em tempos de resposta de aproximadamente 15 a 30 minutos, possivelmente até mais em cenários de elevada complexidade estrutural.

O motor de orquestração assente na plataforma n8n altera este paradigma ao introduzir um fluxo de execução totalmente determinístico. No instante em que ocorre a interrupção de uma adjacência lógica, o sistema dispara e conclui a totalidade das rotinas de pré-validação, diagnóstico diferencial, atuação corretiva e validação em ciclo fechado (*Closed-Loop*) num intervalo que não ultrapassa os 20 segundos. Para fundamentar este ganho sob uma perspetiva científica, o cálculo do tempo médio de resolução ($MTTR$) para um conjunto de N incidentes assume a formulação matemática expressa na **Equação 3.1 - Cálculo do MTTR Autónomo**.

$$MTTR = \frac{\sum_{i=1}^{N}(T_{resolution,i} - T_{detection,i})}{N} \qquad (3.1)$$

Nesta equação, $T_{\text{resolution},i}$ e $T_{\text{detection},i}$ definem a janela temporal do evento, em que $T_{\text{detection},i}$ representa o ***timestamp* exato** em que o Paessler PRTG Network Monitor regista a falha lógica do túnel IPsec no incidente i e despacha o respetivo *webhook*. Em contrapartida, $T_{resolution,i}$ define o momento preciso em que o orquestrador valida o restabelecimento estável do túnel em conformidade com a Fonte Única de Verdade (SSoT). No modelo reativo anterior, a latência de intervenção humana condicionava diretamente o tempo total de indisponibilidade operacional (T_{total}) de um determinado canal, conforme descrito na **Equação 3.2 - Tempo de Inoperabilidade Operacional Total**:

$$T_{total} = T_{deteção} + T_{triagem} + T_{remediação} \qquad (3.2)$$

A introdução da lógica bidirecional em ciclo fechado altera radicalmente este cenário. No ambiente de contingência construído para a **Fase 1 (*SSH Fallback*)**, o tempo de triagem foi mitigado de forma algorítmica ($T_{triagem,i} \rightarrow 0$), fazendo com que o intervalo de remediação ($T_{remediação}$) ficasse circunscrito à latência física de abertura da sessão interativa de terminal e ao processamento de texto via Netmiko, situando-se numa escala previsível de 3 a 5 segundos.

3.1.2 Fase 2: otimização proativa no ambiente Enterprise (REST API Nativa)

A evolução tecnológica para a **Fase 2 (REST API Nativa)** otimizou de forma significativa este limite operativo, reduzindo a variável $T_{\text{remediação}}$ à latência física de processamento de um pacote HTTP síncrono pelo *kernel* do FortiOS, expressa na escala dos milissegundos. No plano da computação autónoma, a taxa de eficiência transacional ($\mathcal{R}_{\text{trans}}$) obtida através do confronto direto entre a execução por linha de comandos da Fase 1 e a injeção síncrona na base de dados de configuração (CMDB) da Fase 2 pode ser modelada analiticamente por intermédio da **Equação 3.3 - Rácio de Eficiência Transacional**.

$$\mathcal{R}_{trans} = \left(1 - \frac{T_{REST_API}}{T_{SSH_Fallback}}\right) \times 100\% \tag{3.3}$$

Os ensaios empíricos realizados em laboratório comprovaram que a Fase 2 atinge de forma consistente uma taxa de eficiência transacional que obedece a $\mathcal{R}_{\text{trans}} \geq 90\%$, operando a reconfiguração de *playbooks* configuracionais complexos num intervalo inferior a 2 segundos. Sob este enquadramento de elevado desempenho, a otimização global do tempo de resposta passa a ser delimitada pelo limite estabelecido na **Equação 3.4 - Limite de Eficiência Algorítmica (Closed-Loop)**:

$$\lim_{T_{triagem}, T_{remediação} \to 0} MTTR \approx T_{deteção} + T_{convergência} \tag{3.4}$$

Esta diminuição no tempo de indisponibilidade operacional ultrapassa os 98% perante falhas lógicas e de configuração, o que valida a eficácia prática e a viabilidade da aplicação de arquiteturas autorregenerativas no contexto do setor público. Apesar do impacto métrico, o principal benefício qualitativo do sistema reside no alívio substancial da carga cognitiva imposta à equipa de comunicações da ARTE, I.P. O operador deixa de consumir tempo útil e foco em rotinas repetitivas de *troubleshooting* técnico e assume a posição estratégica de auditor de processos automatizados de alta fidelidade.

A **Tabela 6** apresenta a comparação detalhada das métricas operacionais recolhidas e validadas no ambiente laboratorial, confrontando o modelo manual legado com os resultados práticos obtidos em ambas as fases de automação.

Tabela 6 - Comparação de métricas temporais e operacionais entre o paradigma manual e o sistema autónomo.

Métrica / Dimensão Operacional	Paradigma Manual Legado	Paradigma NetDevOps: Fase 1 (SSH)	Paradigma NetDevOps: Fase 2 (REST API)	Melhoria Percentual / Impacto
Tempo de Diagnóstico (MTTD)	1 e 5 minutos (dependente do operador)	< 10 segundos (PRTG Webhook)	< 1 segundo (Event-Driven Stitch)	Redução > 99% na deteção de falhas.
Tempo de Resolução (MTTR)	15 a 30 minutos (casos simples)	< 15 segundos (Execução script CLI)	< 1 segundo (Requisição PUT JSON)	Redução > 99.5% em incidentes simples.
Tempo de Resolução (Complexo)	Até 60 minutos (Configuration Drift)	< 20 segundos (Correção via SSH Netmiko)	< 2 segundos (Injeção direta na CMDB)	Mitigação instantânea de drifts de configuração.
Carga Cognitiva das Equipas	Elevada (*context switching* e stress)	Moderada (Atuação automática por scripts)	Nula (Operação invisível e supervisionada)	Eliminação total do erro humano no processo.
Consistência do Diagnóstico	Variável (sujeito a erro interpretativo)	Absoluta (Triagem determinística por código)	Absoluta (Completamente sequencial da CMDB)	Garantia de conformidade contínua e linear.
Rastreabilidade de Alterações	Sem informação.	Parcial (Inserções básicas na coleção comum)	Total (Metadados forenses e logs de memória)	Auditabilidade total para normas ISO 27001.

3.2 Validação dos cenários de Chaos Engineering e análise de logs

A homologação experimental da arquitetura de autocura ativa assentou na simulação sistemática de anomalias operacionais, seguindo estritamente os princípios fundamentais da Engenharia do Caos (*Chaos Engineering*) [11]. O laboratório experimental recorreu à flexibilidade do hipervisor VMware Workstation Pro para emular com fidelidade a topologia da infraestrutura híbrida nacional, integrando instâncias virtuais das Firewalls de Nova Geração FortiGate sob o sistema operativo FortiOS v7.6.4. Esta metodologia estruturada viabilizou a análise empírica do comportamento do ecossistema e a recolha de evidências forenses, dividindo metodologicamente os ensaios de injeção de falhas entre o mecanismo

de contingência herdado, característico da **Fase 1**, e o modelo programático nativo desenvolvido para a **Fase 2**.

3.2.1 Ensaios da Fase 1: Mecanismos de Contingência (Ambiente Trial/SSH)

Cenário A: Simulação de Quebra de Underlay MPLS

Este primeiro cenário recriou a interrupção física do circuito dedicado de transporte WAN (*underlay* MPLS) que interliga a interface *port3* do FortiGate HUB (172.16.0.1) à interface *port2* do FortiGate SPOKE (172.16.0.2). A falha foi injetada desativando administrativamente a rede virtual VMnet5 no hipervisor, o que provocou a queda das associações lógicas do túnel IPsec. No ciclo de consulta seguinte, o n8n interrogou o PRTG Network Monitor e verificou que o sensor do túnel se encontrava fora do estado OK.

Confirmada a queda, o orquestrador consultou a API REST do NetBox, que desempenha o papel de Fonte Única de Verdade (SSoT), para obter o endereço IP do circuito MPLS associado ao dispositivo afetado. Com esse endereço, o fluxo interrogou novamente a API do PRTG, desta vez para o sensor da interface de transporte, que devolveu igualmente um estado diferente de OK. Ao concluir que a indisponibilidade tinha origem na camada física, o n8n bloqueou a execução de qualquer *playbook* de remediação nas firewalls, evitando chamadas de escrita que eventualmente falhariam. Em seguida, enviou um alerta por correio eletrónico via SMTP para o suporte técnico do operador de telecomunicações e gravou os metadados do evento na coleção *incident_runs* do MongoDB com a etiqueta *escalated_to_provider*.

A **Figura 34** documenta esta evidência, ilustrando a interceção do e-mail na plataforma de testes Mailtrap. Este mecanismo evitou a geração de alarmes de configuração quando a origem do problema reside na infraestrutura física do operador externo.

.

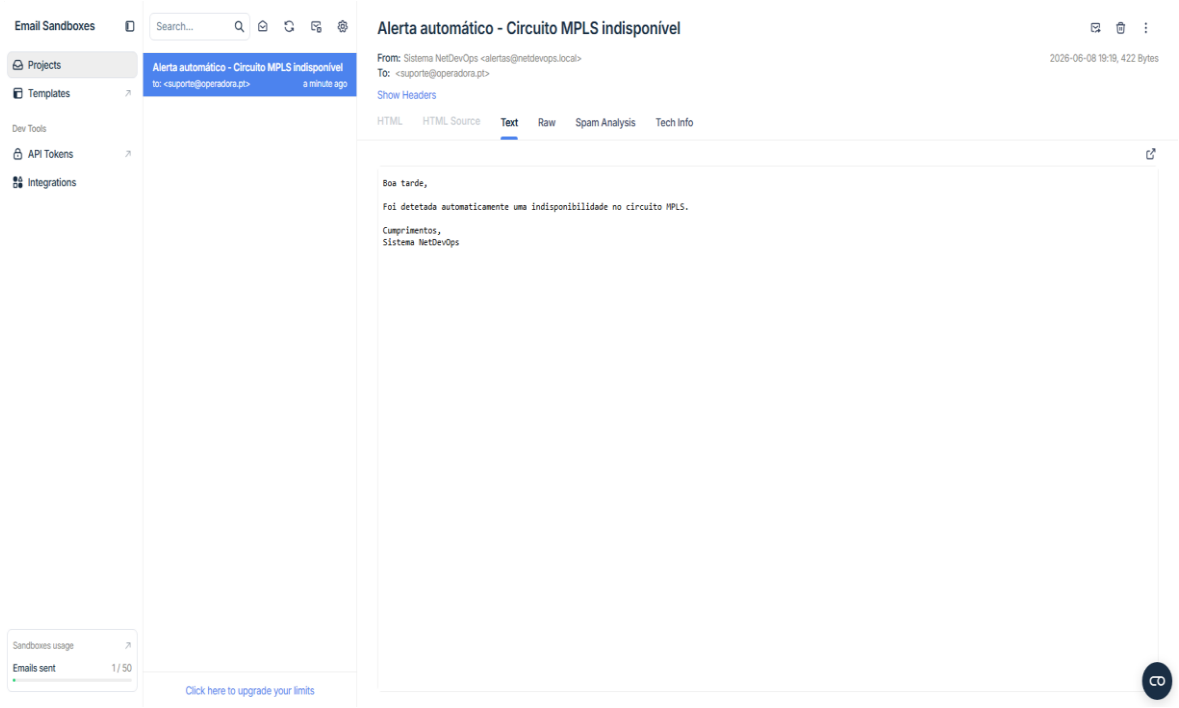


Figura 34 - Notificação automática de incidente via SMTP enviada pelo orquestrador e interceptada em ambiente de sandbox (Mailtrap) após a simulação de falha física no circuito MPLS (Underlay).

Cenário B: Simulação de Configuration Drift na Phase 2

Este ensaio experimental, alinhado com o paradigma de *Chaos Engineering*, simulou a corrupção intencional das definições de segurança lógica na *Phase 2* do protocolo IPsec no *Spoke*, reproduzindo um erro operacional humano comum. Alterou-se manualmente os seletores de tráfego *Proxy-ID* no FortiGate SPOKE remoto, introduzindo gamas de sub-redes incompatíveis com as parametrizações do HUB. Especificamente, a rede do SPOKE passou a apontar para a gama *192.168.99.0/24*, contrastando com o bloco *192.168.10.0/24* configurado no nó central, desalinhamento este que causou a quebra imediata do canal lógico da VPN. O PRTG Network Monitor identificou a perda de adjacência e encaminhou o alerta em formato JSON via *webhook* para o orquestrador n8n.

A rotina de automação iniciou a triagem pelo mecanismo de pré-verificação física (*Pre-Check*), disparando um comando de Ping ICMP da *shell* do Ubuntu Server em direção ao IP público da WAN da filial (172.16.0.2). O teste concluiu com sucesso, registando *exitCode == 0*, o que atestou de forma empírica que a firewall remota se encontrava ligada e operacional na rede de transporte.

Logo após, o n8n efetuou uma requisição de leitura através da API REST do FortiGate HUB, cujo processamento do *payload* JSON indicou que a *Phase 1* permanecia estabelecida, mas os seletores de tráfego ativos divergiam do inventário documentado no NetBox. Caracterizado o desvio estrutural do tipo *phase2_mismatch*, o motor do orquestrador isolou a causa-raiz e encaminhou o

fluxo para o bloco IaC específico, tal como ilustra o rasto de execução ativo no canvas da **Figura 35**.

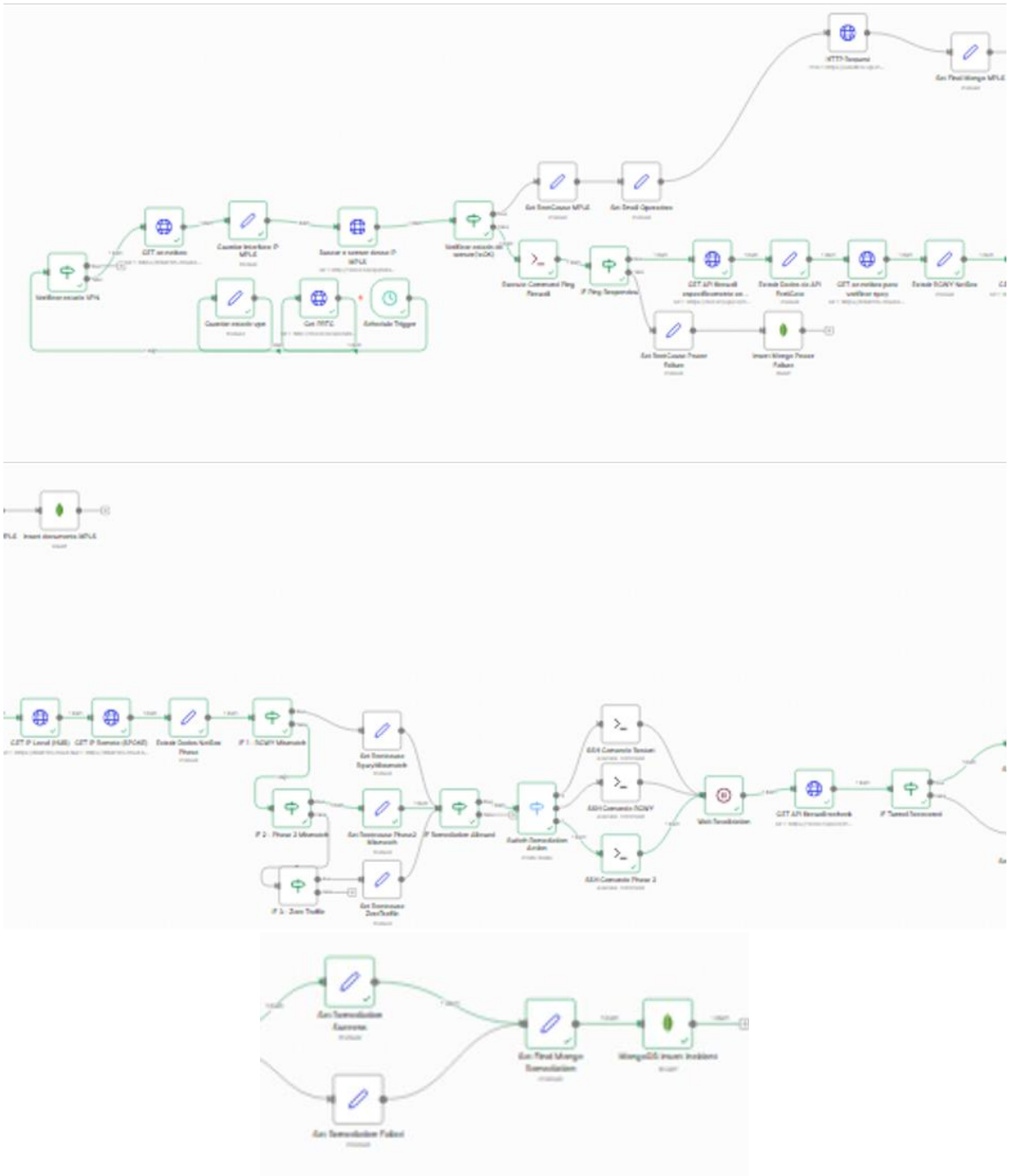


Figura 35 - Fluxo do n8n (Fase 1): Execução ativa do ramo de automação para a causa phase2_mismatch.

O fluxo despoletou a execução local do script *remediate_phase2.py* no ambiente virtual Python (venv) isolado, garantindo a conformidade estrita com a diretiva PEP 668. A ferramenta utilizou as primitivas da biblioteca Netmiko para autenticar a sessão SSH na CLI do FortiGate HUB, injetando as novas diretivas de sub-redes (set src-subnet) e set dst-subnet) extraídas do NetBox. Estas alterações

foram aplicadas sob o bloco *config vpn ipsec phase2-interface*, restabelecendo o alinhamento configuracional da infraestrutura.

A análise de data/hora gerados no *syslog* do FortiOS do FortiGate HUB, detalhados na **Figura 36** com leitura cronológica ascendente, valida o processo de depuração e reinstanciação das associações de segurança (SAs). O alarme inicial de falha na *Phase 2* ocorreu às 15:57:41, sendo que a execução programática do *playbook* IaC se concluiu na CLI às 15:58:23. As instruções enviadas removeram as SAs antigas (delete_phase1_sa), forçando uma descida temporária do túnel (tunnel-down). No segundo seguinte, às 15:58:24, o protocolo IKE iniciou a renegociação automática, processo que resultou às 15:58:30 no restabelecimento do plano de dados, documentado pelos registos sequenciais de *install_sa, phase2-up* e *tunnel-up.*

Os *syslogs* registaram ainda a interceção de um erro de encapsulamento (*esp_error*) no instante da rotação de chaves, um comportamento que evidência claramente a transição ativa no plano de dados sob o comando direto da automação.

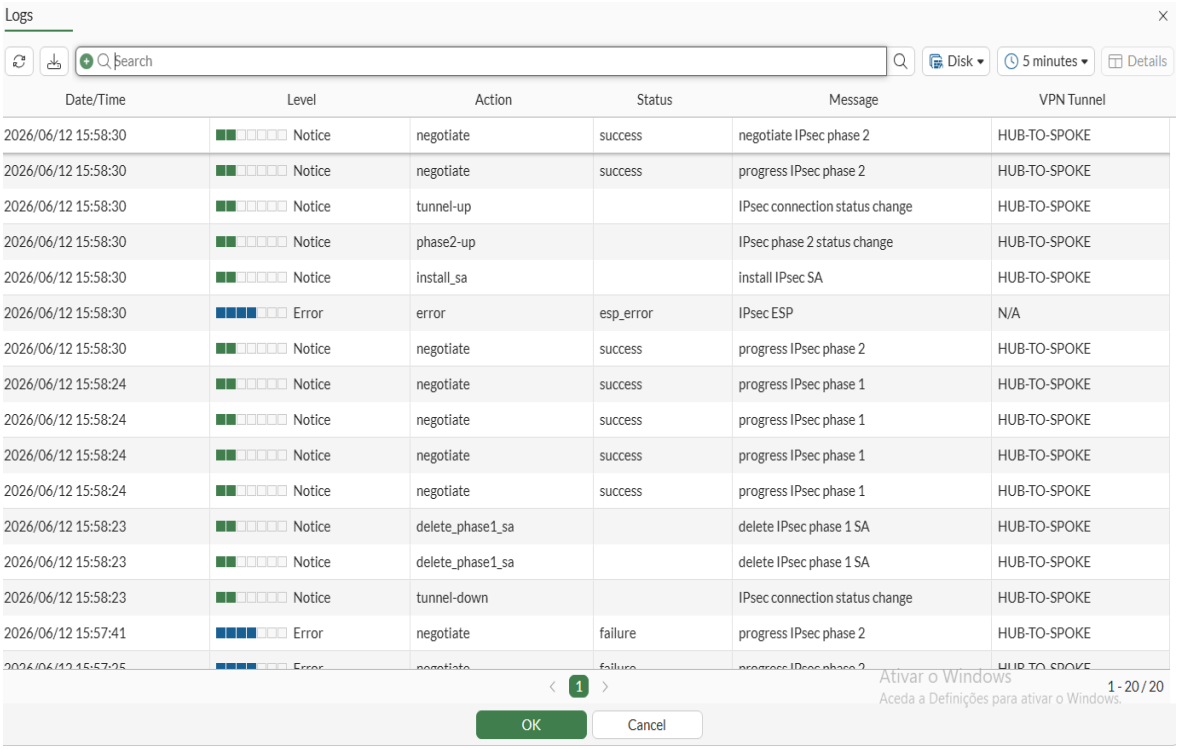


Figura 36 - *Syslogs* do FortiGate HUB (Fase 1): Correção de sub-redes da Phase 2 via Netmiko e restabelecimento do túnel.

Cenário C: Simulação de Quebra Criptográfica Silenciosa (Zero Traffic)

Este ensaio exercitou o ramo terminal da cascata de exclusão. Ao contrário dos cenários anteriores, o estado zero_traffic não corresponde a uma causa-raiz identificável por comparação com a *SSoT*: resulta do esgotamento das verificações anteriores. Quando o IF 1 confirma o gateway remoto e o IF 2 confirma

os seletores da Phase 2, mas os contadores incoming_bytes e outgoing_bytes devolvidos pela API do FortiGate permanecem a zero, o orquestrador conclui que a configuração está conforme e que a falha reside no estado criptográfico da sessão. Esta condição é habitualmente designada por "túnel zombie": a Phase 1 do IKE figura ativa nos painéis de monitorização, mas o plano de dados não transporta tráfego útil.

Para exercitar este ramo, provocou-se a interrupção do tráfego mantendo a configuração conforme no NetBox. No ciclo de consulta seguinte, o n8n verificou o estado do sensor no PRTG, confirmou que o circuito de underlay se mantinha operacional e que o Pre-Check ICMP devolvia exitCode == 0, e percorreu os dois primeiros níveis de triagem sem detetar divergências. Chegado ao IF 3, com ambos os contadores a zero, o fluxo ativou o nó Set Rootcause ZeroTraffic e encaminhou a execução para o bloco de remediação. O script remediate_tunnel.py estabeleceu uma sessão SSH com a CLI do FortiGate HUB através da biblioteca Netmiko e executou diagnose vpn ike gateway clear name HUB-TO-SPOKE, limpando as associações de segurança para forçar nova negociação.

Os registos do FortiOS documentam a sequência. Às 15:40:43 a CLI regista a purga das SAs antigas (delete_phase1_sa) e a descida do túnel (tunnel-down). Às 15:40:44 o IKE inicia a renegociação da Phase 1, concluída às 15:40:50 com o registo sequencial de install_sa, phase2-up e tunnel-up. O erro transitório de encapsulamento (esp_error) registado no momento da rotação de chaves é consistente com a transição do plano de dados.

Este ensaio valida o encaminhamento da decisão e a execução da ação corretiva, mas não isola uma causa-raiz específica: o estado zero_traffic agrega todas as falhas de plano de dados não atribuíveis a desvio de configuração. A discriminação destes casos exigiria a leitura dos registos de negociação IKE do FortiOS — capacidade introduzida apenas na Fase 2

Cenário D: Simulação de Desvio de Configuração de Gateway Remoto

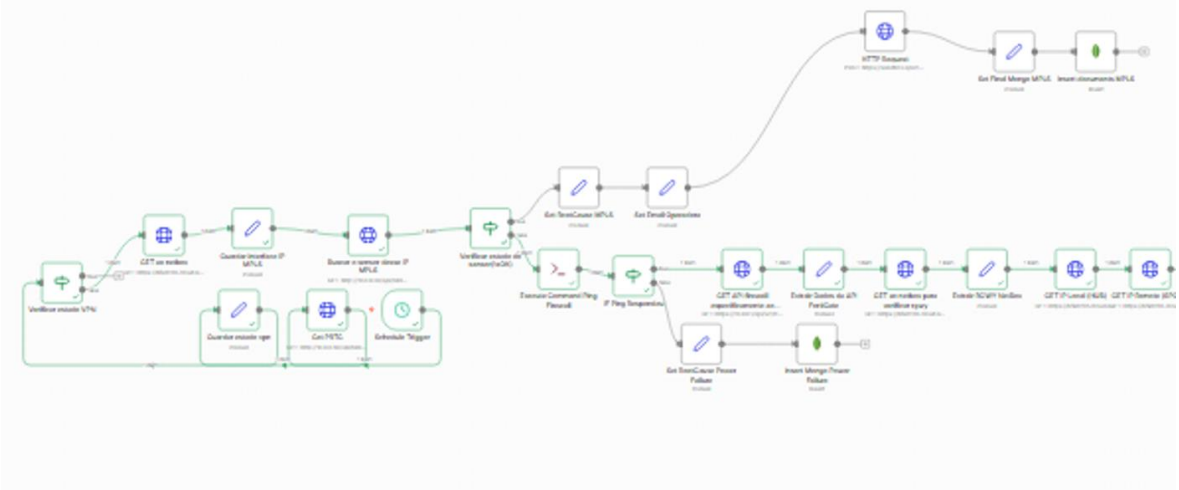
Neste ensaio focado em falhas administrativas, o plano de testes introduziu manualmente um desvio de configuração (*configuration drift*) no endereço IP do gateway remoto (`remote-gw`) configurado no FortiGate HUB. A inserção do endereço IP inválido *1.1.1.1* na *Phase 1* do túnel impediu o estabelecimento das comunicações IKE, o que quebrou de imediato o canal lógico. Assim que o PRTG desencadeou o diagnóstico, o orquestrador n8n interrogou o NetBox e executou a sonda preventiva de conectividade física (*Pre-Check* ICMP) contra o IP público do Spoke (172.16.0.2). O teste obteve sucesso com a devolução do indicador *$exitCode == 0$,* confirmando que a appliance remota se encontrava ligada à alimentação elétrica e operacional na rede WAN.

O n8n recolheu a configuração através da API REST do FortiGate HUB e comparou os dados com as definições estruturadas no inventário centralizado do NetBox. A árvore de decisão identificou que o parâmetro *remote-gw* ativo *(1.1.1.1*)

divergia do valor ideal documentado (*172.16.0.2*), classificando a ocorrência sob a condição *rgwy_mismatch*.

De imediato, o orquestrador acionou o script remediate_config.py no ambiente virtual Python (venv) isolado para cumprimento do PEP 668. Este componente estabeleceu uma sessão SSH programática via Netmiko com a CLI da firewall HUB para aplicar a retificação do IP diretamente no ramo hierárquico da interface IPsec. A alteração na configuração forçou a destruição das associações lógicas antigas, gerando o *reset* automático do contador de *Uptime* do túnel. O fluxo aplicou uma pausa técnica de 12 segundos (nó *Wait*) para a estabilização da troca de chaves. Após validar o sucesso operacional via API REST, o fluxo armazenou os metadados do incidente como *auto_recovered* no MongoDB, conforme apresenta a **Figura 37**.

A análise forense das evidências operacionais capturadas em tempo real na consola de *syslogs* do FortiOS, visível na **Figura 38** com leitura cronológica ascendente, comprova a eficácia do ciclo fechado. Às 15:47:55, o sistema intercetou e registou o estado de bloqueio absoluto com um erro crítico na base do túnel (*progress IPsec phase 1: negotiate failure*), devido à presença do IP inválido. No segundo exato em que o script em IaC aplicou a correção na CLI da firewall HUB, o protocolo IKE despoletou o processo de renegociação *Out-of-Band*. O sistema registou o progresso bem-sucedido da *Phase 1* às 15:47:56, e o sucesso do alinhamento autónomo consolidou-se às 15:47:59, momento em que a appliance HUB registou sequencialmente a instalação estável das chaves (*install_sa*), a subida da *Phase 2* (*phase2-up*) e a convergência do plano de dados (*tunnel-up*). A conectividade ficou plenamente restaurada sem qualquer interação manual humana.



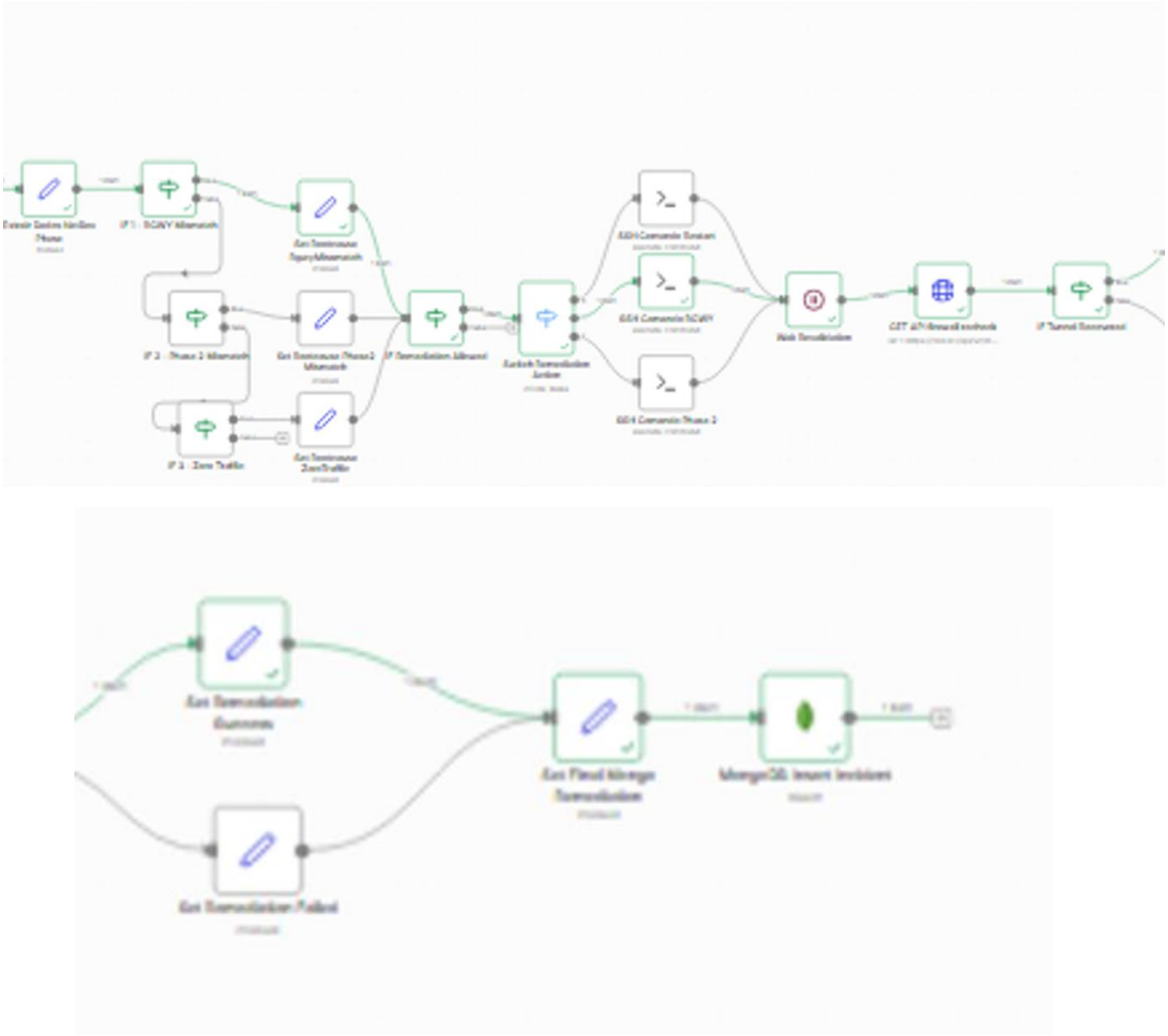


Figura 37 - Fluxo do n8n (Fase 1): Execução ativa do ramo de automação para a causa rgwy_mismatch.

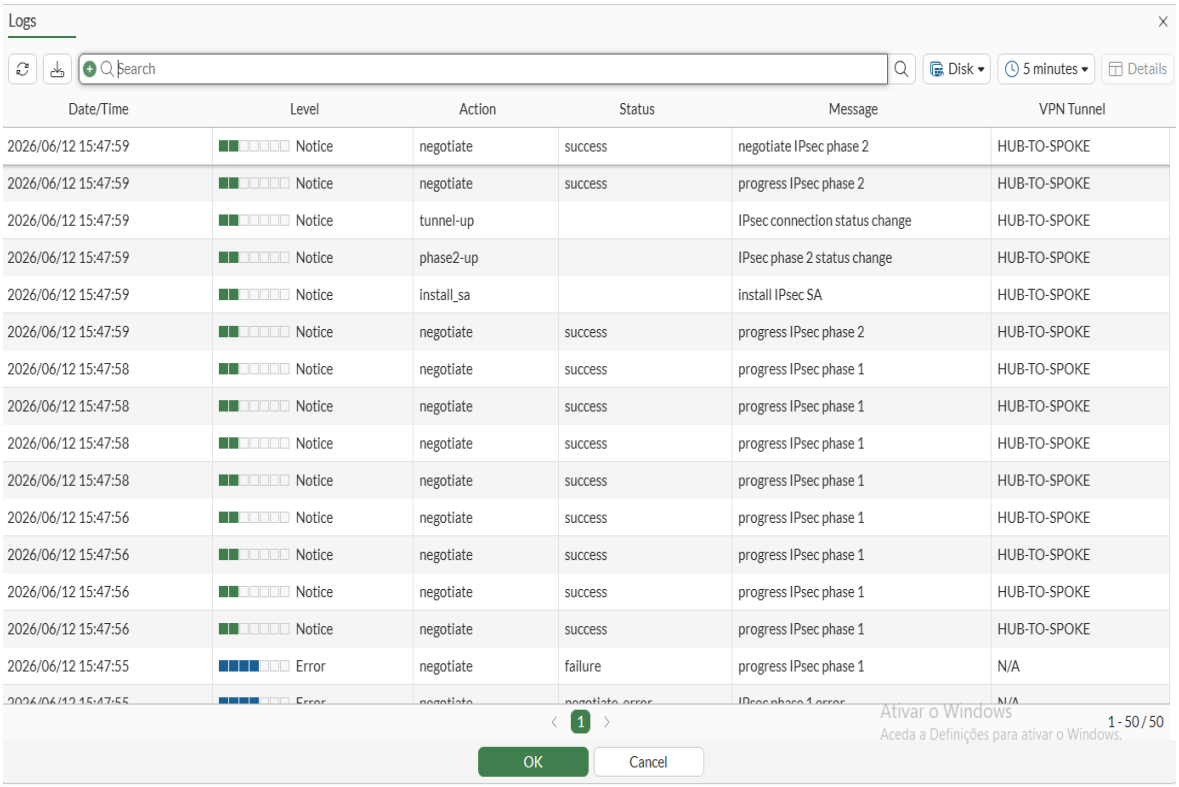


Figura 38 - *Syslogs* do FortiGate HUB (Fase 1): Transição automática de falha na Phase 1 (negotiate failure) para restabelecimento do túnel (tunnel-up).

Cenário E: Inacessibilidade Física do Equipamento Remoto

O teste de robustez e segurança lógica do sistema perante falhas catastróficas na infraestrutura, simulando uma interrupção elétrica total numa das filiais. Para alcançar este cenário, a equipa realizou o encerramento forçado (*Power Off*) da máquina virtual que alojava a appliance FortiGate SPOKE dentro do ambiente VMware Workstation Pro. A perda repentina de conectividade foi identificada pelo PRTG, e a plataforma de monitorização registou a inoperabilidade lógica do túnel, encaminhando o alerta para o orquestrador n8n

Assim que com a ajuda do NetBox validou que o circuito de transporte WAN MPLS mantinha o seu estado administrativamente ativo, o orquestrador n8n acionou a rotina preventiva de conectividade física. O comando local de Ping, disparado a partir da *shell* do Ubuntu Server em direção ao endereço IP público da WAN da filial, falhou de forma sistemática. O sistema operativo devolveu um código de erro diferente de zero, fixando a variável no estado $exitCode == 1$.

A **Figura 39** exibe o rasto de execução ativa no canvas do n8n, assinalado pelo percurso de execução assinalado a verde, servindo como evidência forense da resposta analítica do orquestrador durante o colapso físico. Fica claro que, após certificar a conformidade da infraestrutura MPLS externa, o fluxo avançou de forma síncrona em direção ao nó *Execute Command Ping Firewall*. Perante a ausência de resposta ICMP por parte da filial, a condição lógica do nó *IF Ping Respondeu* classificou o resultado como falso. Esta triagem contornou o bloco convencional da

REST API e desviou a orquestração diretamente para o nó *Set RootCause Power Failure*, continuando o registo do incidente para a coleção do MongoDB através do nó *Insert Mongo Power Failure* sob a etiqueta *power_or_hardware_failure*. Este mecanismo evitou a ocorrência de *timeouts* dispendiosos e eliminou o consumo desnecessário de CPU no equipamento de gestão.

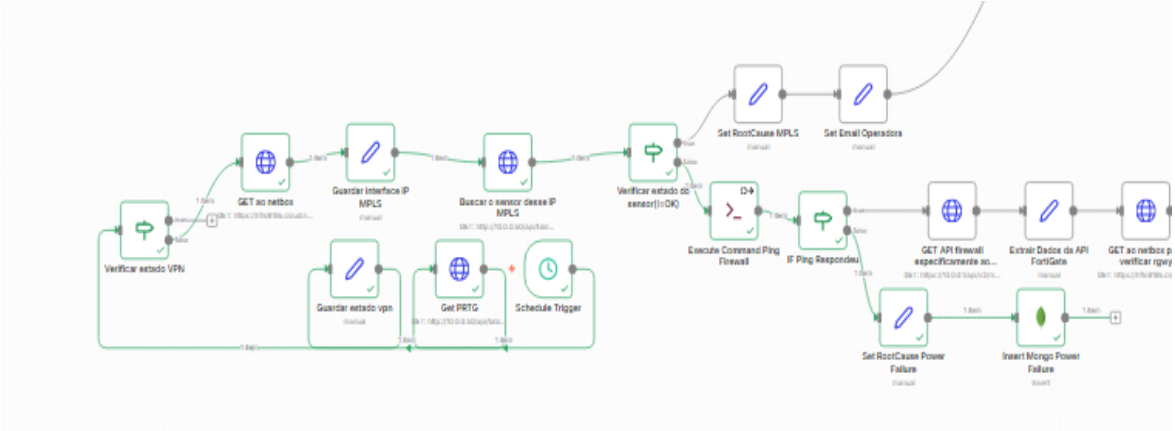


Figura 39 - Execução do workflow n8n no desvio lógico para falha elétrica (Fase 1).

3.2.2 Ensaios da fase 2: integração de APIs Nativas (Ambiente Enterprise)

De forma a validar a nova árvore lógica baseada em APIs e testar os *playbooks Zero-Code* desenvolvidos após a introdução das licenças comerciais da ARTE, I.P., submeteu-se o laboratório a um conjunto completo de ensaios de *Chaos Engineering*.

Cenário F: Falha na Camada de Transporte Externo (Underlay MPLS Down)

Este ensaio recriou-se a quebra física do circuito de transporte MPLS. Assim que o *webhook* do PRTG notificou a queda do sensor, o n8n consultou síncronamente o sensor de transporte ID 2072. O bloco *Verificar estado do sensor (!=OK)* avaliou o estado como verdadeiro, desviando imediatamente o fluxo para o ramo superior de isolamento físico. A linha de validação ativa deste cenário encontra-se ilustrada na **Figura 40**, demonstrando o bloqueio preventivo de escrita na firewall e o envio do alerta SMTP direto para o Mailtrap da operadora.

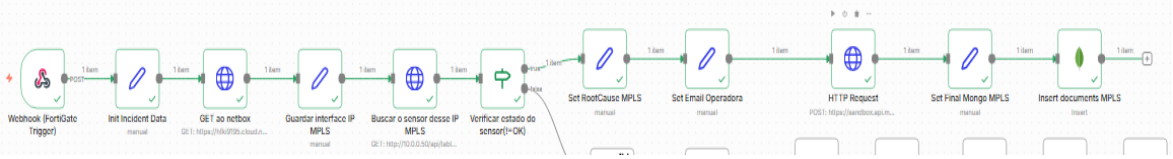


Figura 40 - Rasto de execução ativa no n8n (Fase 2) evidenciando o desvio imediato para o ramo de falha de transporte externo.

Cenário G: Colapso por Ausência de Acessibilidade Física de Equipamento na SPOKE

Para testar o isolamento de hardware, provocou-se um corte de energia ao desligar a VM do SPOKE. O n8n verificou que a rede MPLS estava operacional e disparou o nó *Execute Command Ping Firewall*.

Face à ausência de resposta ICMP, o nó *IF Ping Respondeu* devolveu falso, desviando o percurso de execução assinalado a verde para o bloco inferior de falha elétrica, não precisando assim de conexões API desnecessárias e disparando o alerta para o *bot* do Telegram, conforme documentado na **Figura 41** e na **Figura 42**, onde a notificação crítica de inacessibilidade na filial é recebida instantaneamente no grupo de Telegram.

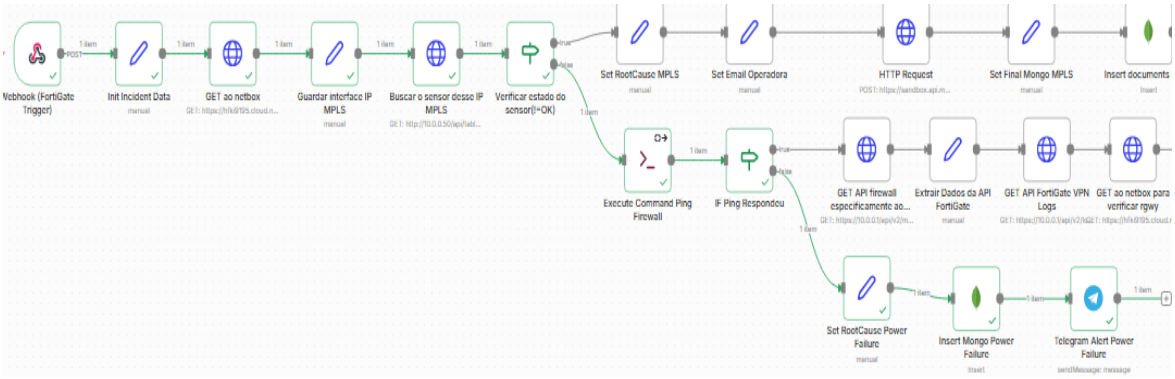


Figura 41 - Rasto de execução ativa no n8n (Fase 2) demonstrando a triagem preventiva de interrupção de energia local.

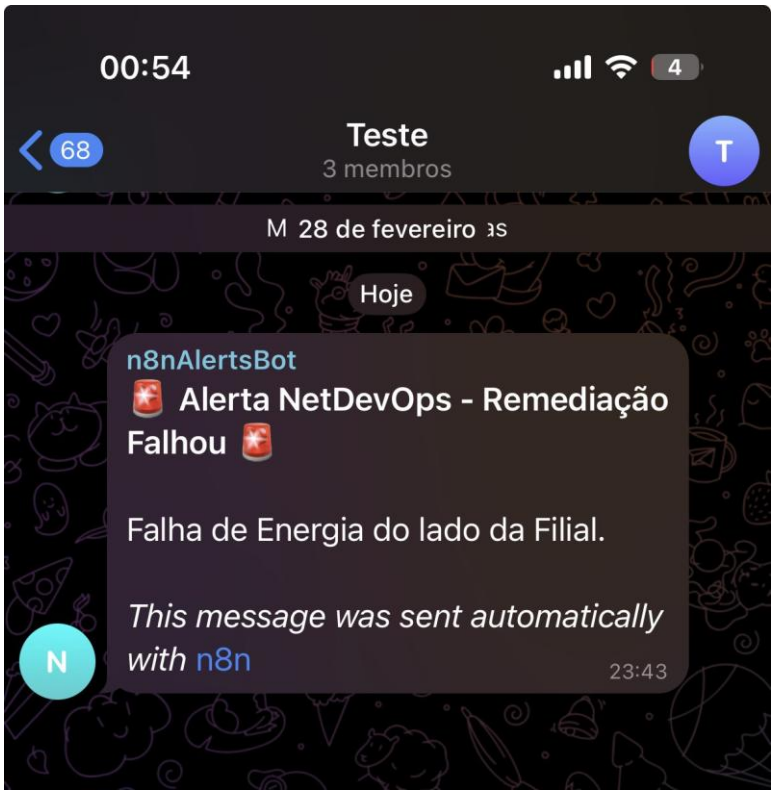


Figura 42 - Notificação crítica de inacessibilidade na filial recebida no grupo de Telegram (Fase 2).

Cenário H: Desvio de Configuração no Plano de Controlo (Phase 1 Remote Gateway Mismatch)

Neste ensaio focado em erros administrativos, alterou-se o IP do Remote Gateway no HUB para a gama inválida *1.1.1.1*. O n8n recolheu as configurações da CMDB da firewall e cruzou-as com a Fonte de Verdade do NetBox.O nó *IF 1 - RGWY Mismatch* validou a desconformidade como verdadeira, isolando a causa-raiz e ativando o nó de remediação *API Update RGWY* para reescrever o IP correto via método HTTP PUT, como exibe o percurso de execução assinalado a verde na **Figura 43**.

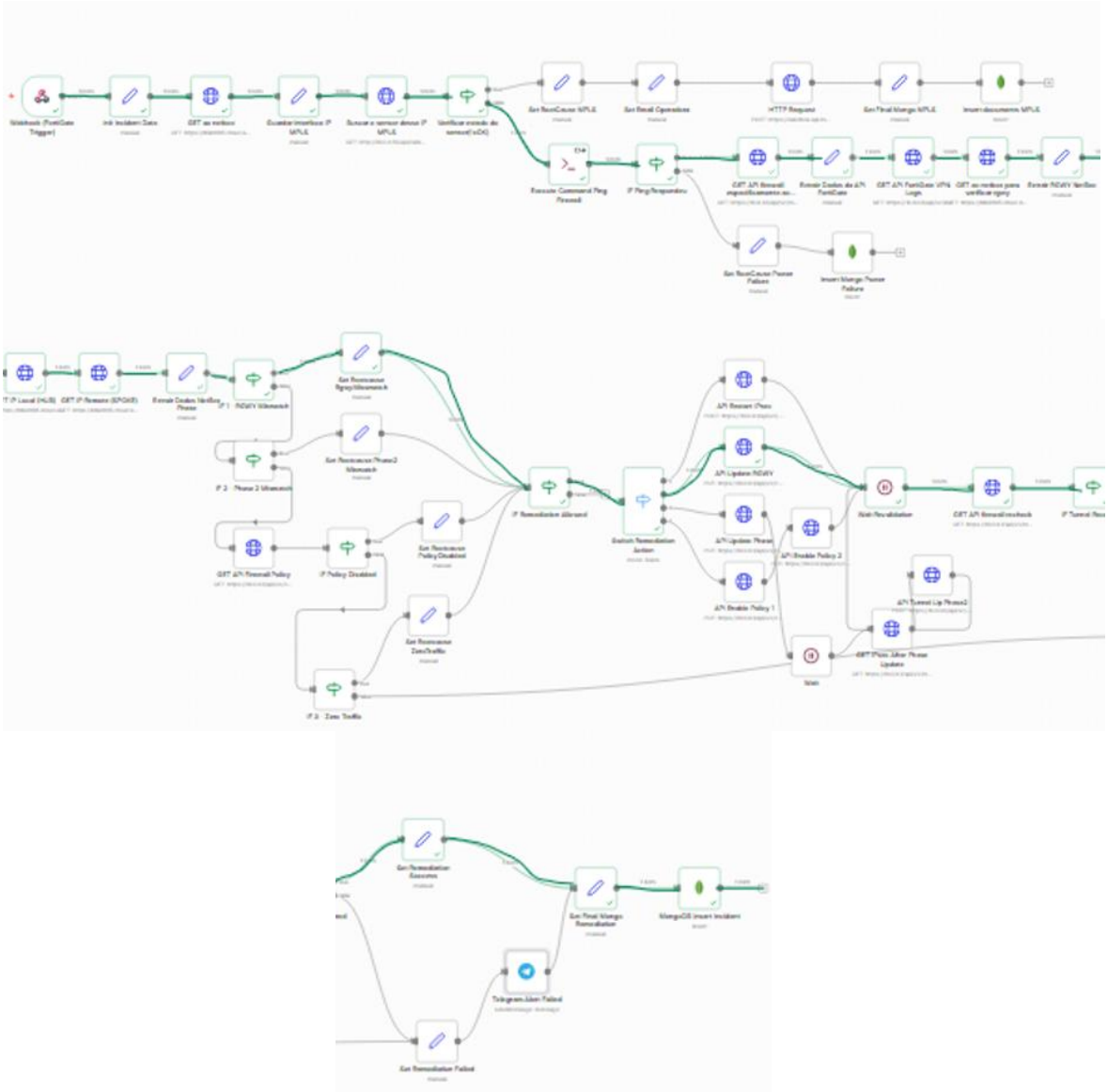


Figura 43 - Rasto de execução ativa no n8n (Fase 2) demonstrando o isolamento e retificação automática de drift no Gateway Remoto.

Cenário I: Incompatibilidade de Seletores de Tráfego no Plano de Dados (Phase 2 Mismatch)

Simulou-se a corrupção de sub-redes alterando os seletores da *Phase 2* no SPOKE. O n8n validou tudo o que estava atras como correto, mas o nó *IF 2 - Phase 2 Mismatch* intercetou o desalinhamento de blocos CIDR face ao NetBox.

O fluxo ativou a causa *phase2_mismatch* e executou o nó *API Update Phase2*, abrindo um sub-ciclo síncrono que reconfigura as sub-redes na CMDB e força o levantamento da *Phase 2*, percurso este documentado no fluxo de validação da **Figura 44**.

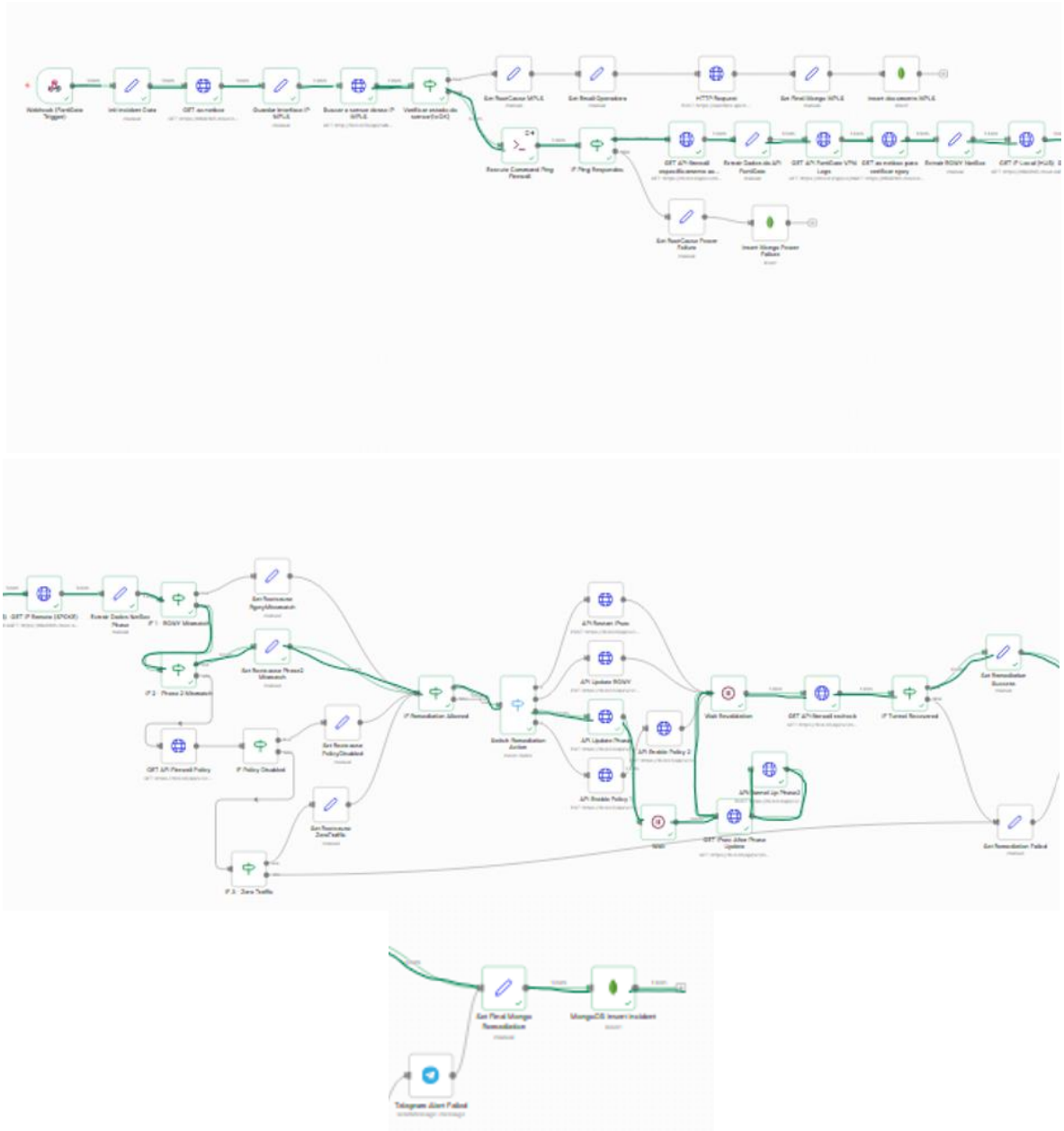


Figura 44 - Rasto de execução ativa no n8n (Fase 2) evidenciando o percurso do ramo de correção e subida da Phase 2.

Cenário J: Desativação Administrativa de Regras de Acesso (Firewall Policy Disabled)

Neste ensaio, desativou-se a política de firewall ID 2 *(SPOKE-to-LAN*) na interface gráfica do FortiOS. O n8n executou o varrimento de conformidade e detetou que os parâmetros físicos e criptográficos estavam corretos, mas a regra de tráfego estava inativa.

O nó *IF Policy Disabled* devolveu verdadeiro, ativando a causa *policy_disabled*. O orquestrador encaminhou o fluxo para os nós *API Enable Policy 1* e *API Enable Policy 2,* reativando as duas regras em simultâneo via métodos PUT, conforme ilustrado na **Figura 45**.

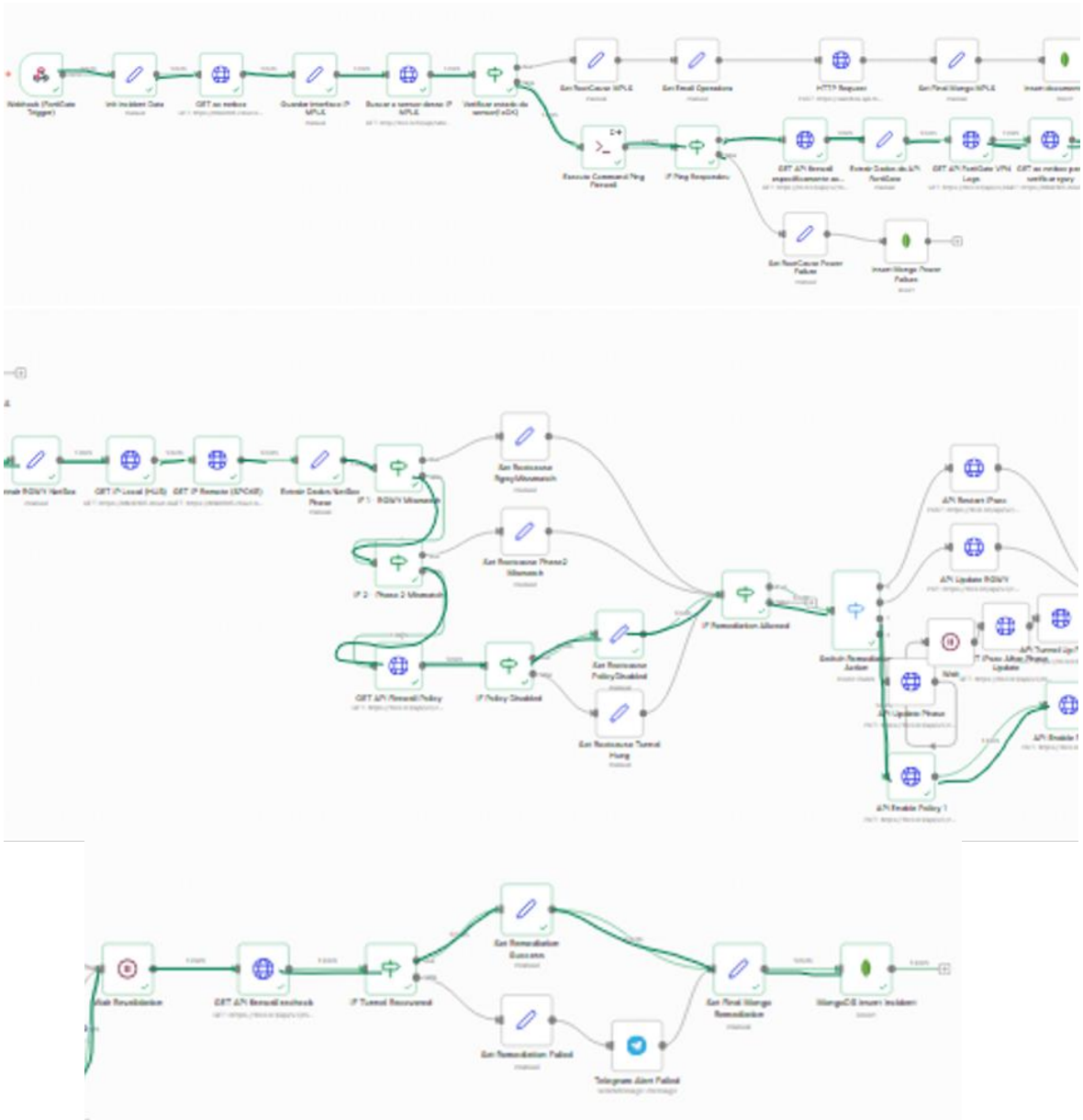


Figura 45 - Rasto de execução ativa no n8n (Fase 2) consolidando o "Combo Duplo" de reativação de políticas de firewall via API.

Para consolidar e organizar o mapeamento de todas as regras operacionais, condições de infraestrutura e *playbooks* de automação descritos ao longo deste capítulo, estruturou-se a Matriz de Decisão, Atuação e Auditoria global apresentada na **Tabela 7**.

Tabela 7 - Matriz de decisão, atuação e auditoria dos cenários de Chaos Engineering (Fases 1 e 2).

Cenário de Teste	Causa-Raiz Lógica Detetada	Mecanismo de Atuação (Ação Corretiva)	Estado de Fecho / Alerta
A: Quebra Underlay	mpls_down (Sensor DOWN no PRTG)	Aborto lógico preventivo; Notificação SMTP ao operador	escalated_to_provider
B: Drift Phase 2 (F1)	phase2_mismatch (Proxy-IDs incorretos)	**SSH / CLI Netmiko:** Correção de sub-redes na firewall	auto_recovered
C: Ausência de Tráfego (F1)	zero_traffic (configuração conforme, contadores a zero)	**SSH / CLI Netmiko:** Limpeza das SAs via ike gateway clear	auto_recovered
D: Drift Gateway (F1)	rgwy_mismatch (Remote-Gateway inválido)	**SSH / CLI Netmiko:** Retificação do IP de remote-gw	auto_recovered
E: Equipamento Inacessível (F1)	power_or_hardware_failure (Ping timeout)	Aborto preventivo de conexões SSH (anti-timeout)	power_or_hardware_failure
F: Quebra de Underlay Externo (F2)	mpls_down (sensor do circuito MPLS fora do estado *OK* no PRTG)	REST API: Aborto preventivo da escrita na CMDB; notificação SMTP ao operador via Mailtrap	escalated_to_provider
G: Equipamento Inacessível (F2)	power_or_hardware_failure (*Pre-Check* ICMP com exitCode ≠ 0)	Aborto preventivo das chamadas à API; alerta via Telegram	power_or_hardware_failure
H: Drift Gateway (F2)	rgwy_mismatch (Remote-Gateway inválido)	**REST API:** Chamada PUT CMDB (Gateway correto)	auto_recovered (Enterprise)

I: Drift Phase 2 (F2)	phase2_mismatch (Proxy-IDs incorretos)	**REST API:** Chamada PUT CMDB e bring-up de P2	auto_recovered (Enterprise)
J: Policy Disabled (F2)	policy_disabled (ID 1 ou 2 desativados)	**REST API:** Chamadas PUT {"status": "enable"}	auto_recovered (Enterprise)

3.3 Visibilidade analítica e indicadores BI (Metabase)

Os indicadores apresentados nesta secção resultam da agregação dos incidentes gerados pelos ensaios descritos na secção anterior, persistidos no MongoDB e consumidos pelo Metabase**.** A tradução dos registos operativos em indicadores de governação é necessária para assegurar a visibilidade e a auditabilidade contínua da infraestrutura de comunicações da ARTE, I.P. Na arquitetura desenvolvida, a camada de *Business Intelligence* (BI), assente na plataforma Metabase, consolida e torna legíveis os dados armazenados no MongoDB. Através da modelação de painéis dinâmicos, esta camada faculta às equipas de engenharia e auditoria uma interpretação clara do comportamento dos fluxos de autorremediação.

Para enquadrar a evolução desta capacidade analítica de forma estruturada, a apresentação dos indicadores de monitorização foi dividida em dois patamares cronológicos e lógicos. Numa fase inicial, detalha-se a linha de base (*baseline*) focada no isolamento de quatro causas-raiz clássicas observadas no ambiente *Trial*, validando a consistência do motor de triagem. Em seguida, demonstra-se o mapeamento alcançado com a introdução da coleção Enterprise, em que o resultado de esquemas JSON pode viabilizar uma tabela forense. As subsecções seguintes expõem as métricas volumétricas, os rácios de eficiência e os painéis gráficos gerados em cada um dos cenários.

3.3.1 Fase 1: baseline e indicadores da coleção diagnóstica

A visibilidade de gestão e a verificação contínua da conformidade dos fluxos de autocura ativa assentam numa arquitetura de dados plenamente integrada. O orquestrador n8n audita a atividade operacional através da inserção de metadados na coleção *incident_runs ou incidente_runs_enterprise* se forem dados provenientes da fase 2 do projeto numa base de dados não relacional MongoDB. Para sustentar estes microsserviços, a implementação tirou partido de virtualização baseada em contentores Docker. A configuração da infraestrutura de rede e de todos os volumes de persistência necessários foi desenhada de forma declarativa através de ficheiros YAML, respeitando os princípios de Infraestrutura como Código (IaC). Esta decisão confere uma elevada portabilidade ao ambiente de dados, viabilizando a replicação imediata de toda a estrutura em qualquer servidor anfitrião baseado em Ubuntu Server.

A interface gráfica do MongoDB Compass, disponível na rede de gestão e representada na **Figura 46**, faculta o acesso direto à configuração e integridade desses registos estruturados em documentos JSON independentes. O analista consegue, assim, inspecionar e validar o rasto forense deixado pelo sistema autónomo em tempo real. A ligação segura ao cluster em contentores Docker tira partido de credenciais de super-administrador, direcionando o cliente de gestão para o endpoint de testes *mongodb://root:RootMongo2026!@10.0.0.60:27017/*. Ao interrogar a coleção, recupera-se o historial descritivo de cada vetor de mitigação, cujo modelo estrutural segue a estrutura-tipo em formato JSON.


```
{
  "_id": {...},
  "timestamp": "2026-06-12T20:02:59.313-04:00",
  "rootCause": "power_or_hardware_failure",
  "severity": "critical",
  "remediationAllowed": "false",
  "remediationResult": "not_attempted",
  "finalState": "manual_intervention_required",
  "tunnelName": "HUB-TO-SPOKE"
}
```

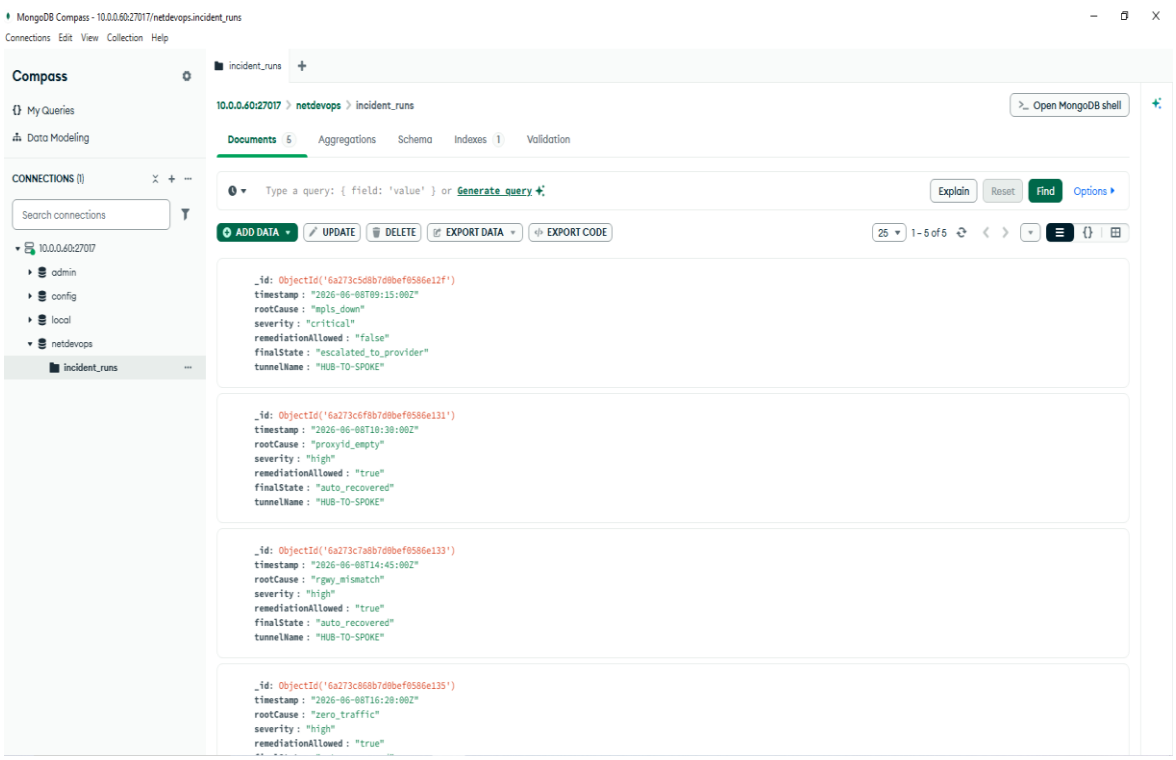



Figura 46 - Interface gráfica do MongoDB Compass no ambiente de gestão (Windows), exibindo as instâncias de incidentes estruturadas em documentos JSON e armazenadas na coleção incident_runs no IP de produção 10.0.0.60 para rastreabilidade operativa (Fase 1).

Selecionou-se a plataforma Metabase para estruturar a camada de Business Intelligence, por consumir de forma facilitada os registos NoSQL armazenados no MongoDB. O painel de visualização expõe a consolidação dos ensaios de Engenharia do Caos executados sobre a topologia híbrida. A linha de base

(baseline) reuniu 13 incidentes, resultantes da injeção repetida das cinco tipologias de falha que o motor de triagem isola no ambiente Trial. A **Figura 47** apresenta a frequência absoluta por causa-raiz: *mpls_down* com 4 ocorrências, *rgwy_mismatch* com 3, e phase2_mismatch*, power_or_hardware_failure* e *zero_traffic* com 2 cada. A **Figura 48** traduz a mesma distribuição em termos proporcionais, evidenciando que as quebras de transporte representam 30,8 % do total e os desvios de gateway 23,1 %, cabendo às restantes três tipologias 15,4 % cada. A repetição de cada tipologia permitiu confirmar que o motor de triagem atribui de forma consistente a mesma causa-raiz a avarias da mesma natureza. A distribuição observada reflete a composição do plano de ensaios e não a frequência relativa das avarias em ambiente de produção.

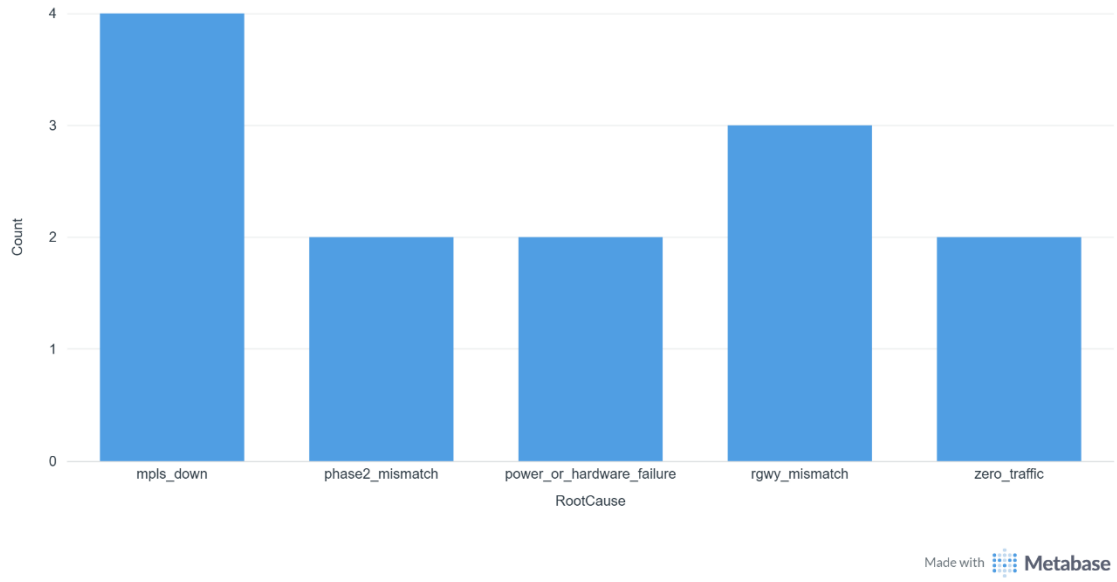


Figura 47 - Gráfico de barras verticais no Metabase para Análise de Causa-Raiz, contabilizando a frequência absoluta das tipologias de falha isoladas pelo orquestrador ao longo dos 13 incidentes da linha de base (Fase 1).

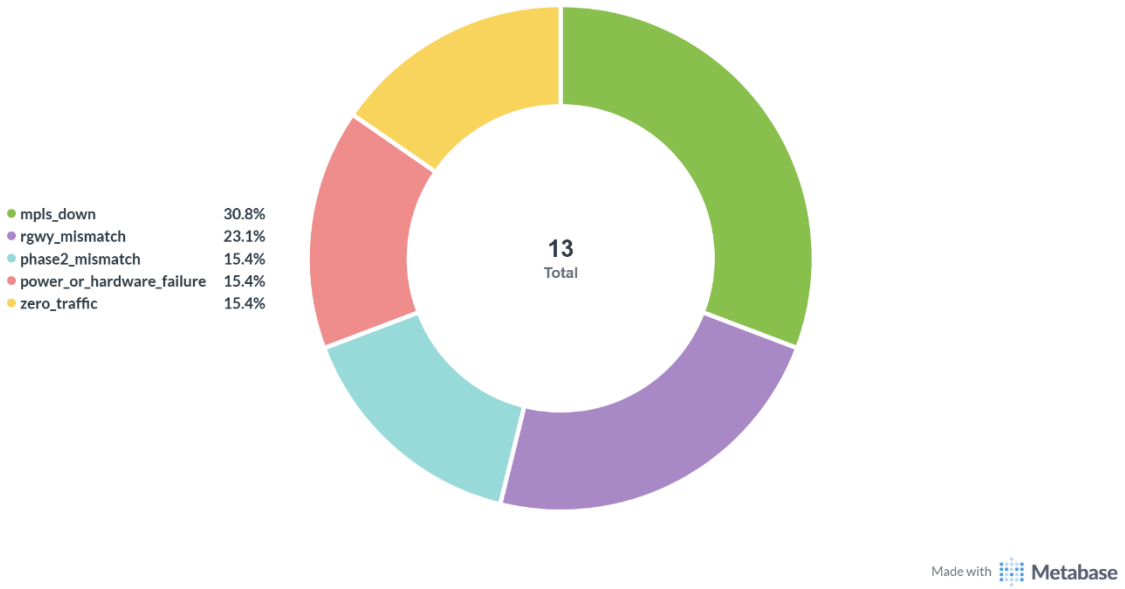


Figura 48 - Gráfico circular do tipo donut no Metabase, evidenciando a distribuição percentual proporcional das tipologias de falha isoladas ao longo dos 13 incidentes da linha de base (Fase 1).

Ao cruzar os mesmos dados sob a ótica do impacto organizacional, o Metabase classifica as ocorrências segundo o nível de severidade que a equipa atribuiu no orquestrador. Conforme apresentado no gráfico circular do tipo da **Figura 49**, os 13 incidentes distribuem-se entre falhas de infraestrutura e desalinhamentos de configuração. Os sete incidentes associados a anomalias lógicas na firewall — correspondentes às causas *zero_traffic*, *rgwy_mismatch* e *phase2_mismatch* — representam 53,8 % do total e recebem a severidade *high*. As seis ocorrências que traduzem interrupções físicas na camada de transporte ou no fornecimento de energia na filial — identificadas pelas marcas *mpls_down* e *power_or_hardware_failure* — perfazem os restantes 46,2 % e são classificadas como *critical*. A atribuição desta magnitude permite assim distinguir, no mesmo painel, as avarias que o sistema resolve autonomamente daquelas que exigem intervenção de terceiros.

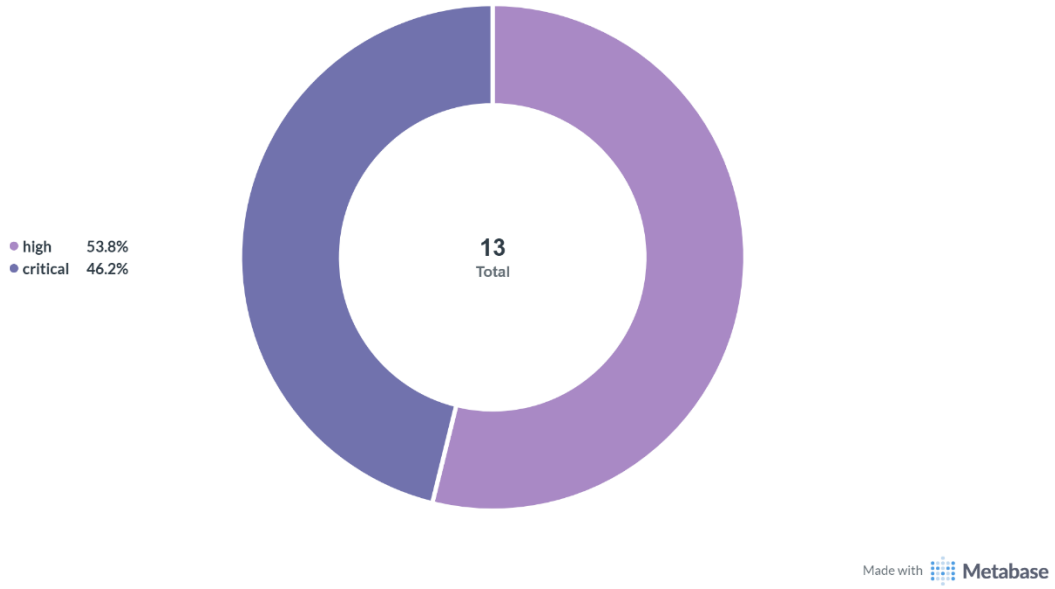


Figura 49 - Gráfico circular do tipo donut no Metabase, evidenciando a distribuição percentual proporcional das tipologias de falha isoladas ao longo dos 13 incidentes da linha de base (Fase 1).

Numa outra perspetiva, o painel segmenta as ocorrências com base na permissão de execução de rotinas ativas. Conforme apresentado no gráfico circular da **Figura 50**, dos 13 incidentes acumulados, 53,8 % obtiveram a variável *remediationAllowed* com o valor verdadeiro (true). Correspondem aos sete casos em que a causa-raiz foi atribuída a uma falha lógica de configuração e em que o acionamento dos *scripts* via Netmiko/SSH era, por isso, legítimo. Os restantes 46,2 % registaram o indicador como falso (false): são os seis incidentes de origem física ou externa, em que o orquestrador abortou a execução antes de qualquer escrita na firewall.

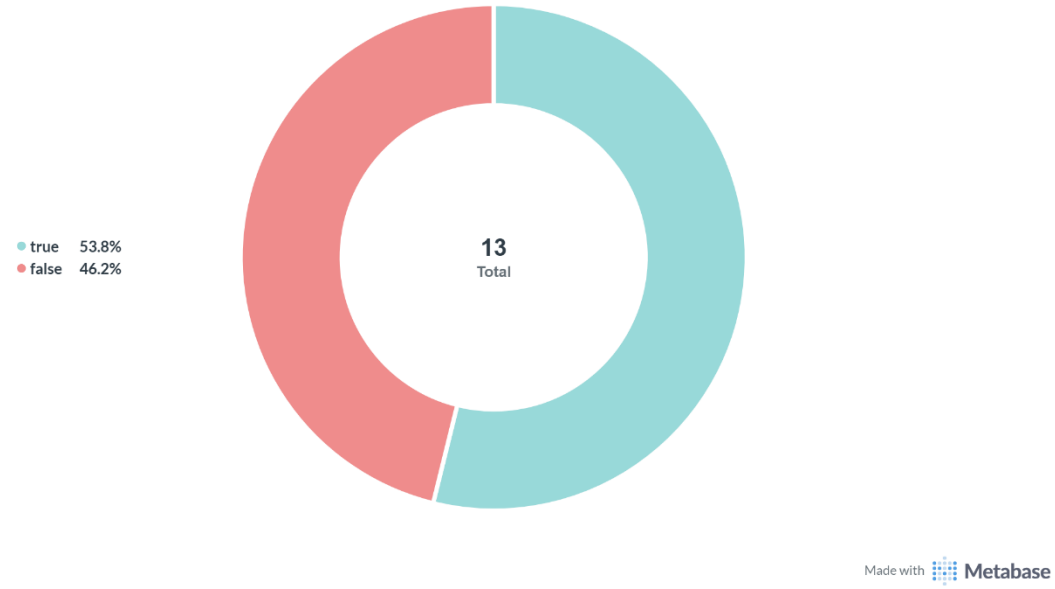


Figura 50 - Distribuição dos 13 incidentes da linha de base segundo a variável remediationAllowed, evidenciando a separação entre as ocorrências com remediação autorizada (true) e as bloqueadas por origem física ou externa (false) (Fase 1).

Numa análise complementar, o Metabase cruza estas permissões com o desfecho das ações executadas. O indicador da **Figura 51**, focado no parâmetro *remediationResult*, mostra que 53,8 % das ocorrências terminaram em sucesso (Success) — os sete casos em que os scripts *remediate_tunnel.py*, *remediate_phase2.py* e *remediate_config.py* restabeleceram a ligação. Os restantes 46,2 % foram classificados como não tentados (not_attempted), correspondendo às falhas de transporte ou de inacessibilidade à filial remota que o orquestrador bloqueou antes de qualquer tentativa de escrita. Lendo os dois indicadores em conjunto, os sete sucessos correspondem à totalidade das ocorrências com *remediationAllowed*: true, enquanto as seis restantes nunca chegaram à fase de escrita, por decisão do próprio fluxo.

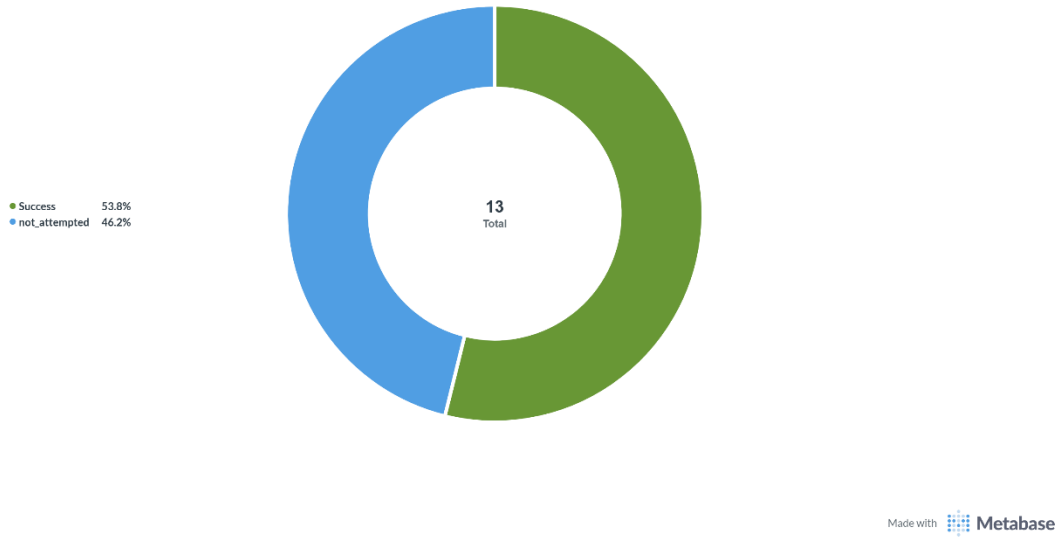


Figura 51 - Distribuição do parâmetro remediationResult nos 13 incidentes registados, com 53,8 % de remediações bem-sucedidas e 46,2 % não tentadas.

O gráfico circular da **Figura 52** apresenta a distribuição dos estados finais ao longo dos 13 incidentes acumulados. A parcela mais expressiva corresponde às ocorrências resolvidas pelo orquestrador sem intervenção humana, sob o estado *auto_recovered*, com 53,8 % do total. Seguem-se os eventos de origem externa encaminhados para o suporte do operador, classificados como *escalated_to_provider*, com 30,8 %, e as situações de inacessibilidade do equipamento remoto, sob a etiqueta *power_or_hardware_failure*, com 15,4 %. A **Figura 53** apresenta os mesmos dados em valores absolutos: sete, quatro e dois incidentes, respetivamente. Lidos em conjunto com o indicador anterior, estes valores mostram que pouco mais de metade dos incidentes foi resolvida em ciclo fechado, enquanto os restantes foram corretamente classificados na origem: encaminhados para o operador, quando a causa era externa ao perímetro gerido, ou registados como falha de energia ou de hardware, quando o diagnóstico apontou para o equipamento ou para a alimentação do local.

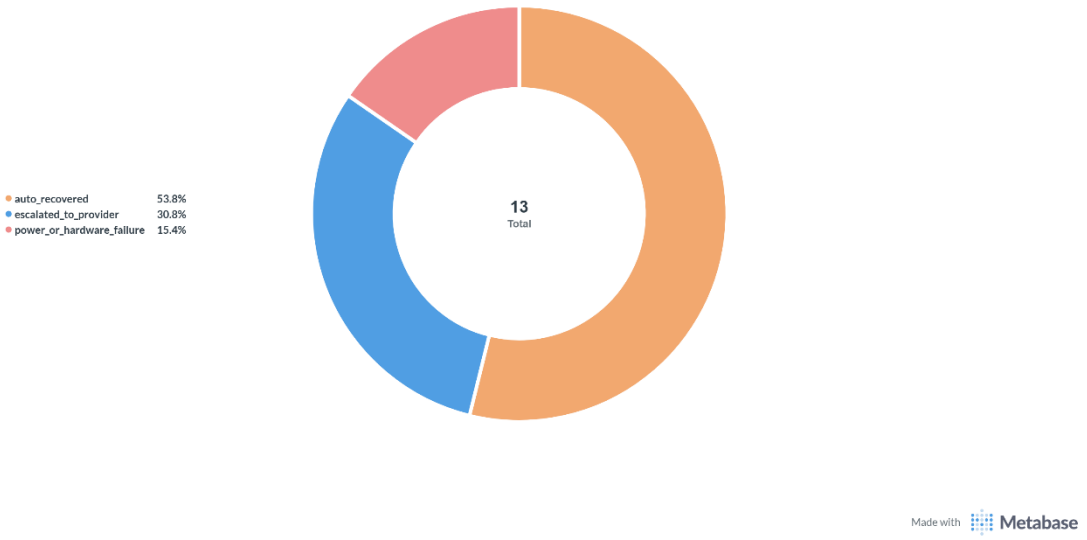


Figura 52 - Distribuição percentual dos estados finais (FinalState) dos 13 incidentes registados.

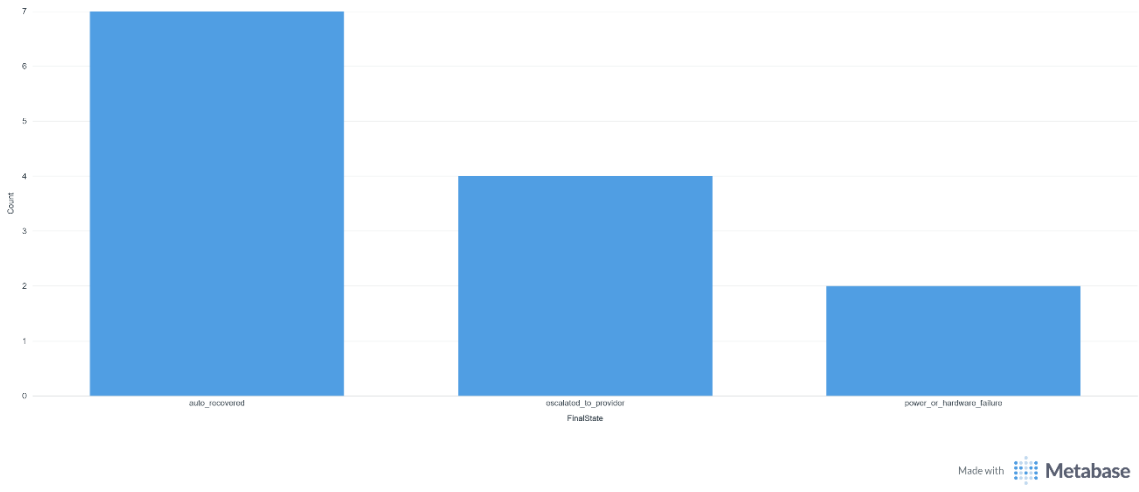


Figura 53 - Contagem absoluta de incidentes por estado final (FinalState).

3.3.2 Fase 2: Mapeamento de indicadores Enterprise

A migração para a arquitetura licenciada permitiu estruturar um conjunto de dados mais rico. O orquestrador passou a enviar os metadados para a coleção *incident_runs_enterprise* do MongoDB, com uma estrutura de documento distinta da utilizada na linha de base. Após a sincronização de esquemas no Metabase, o conjunto reuniu 11 incidentes acumulados.

A **Figura 54** apresenta a distribuição da variável *remediationResult*. Das 11 ocorrências, 72,7 % terminaram em sucesso (Success), correspondendo à totalidade dos casos em que a remediação chegou a ser tentada. Os restantes 27,3 % foram classificados como não tentadas (not_attempted): o orquestrador bloqueou a escrita na firewall sempre que a causa-raiz foi isolada como falha física ou externa, sob responsabilidade de terceiros.

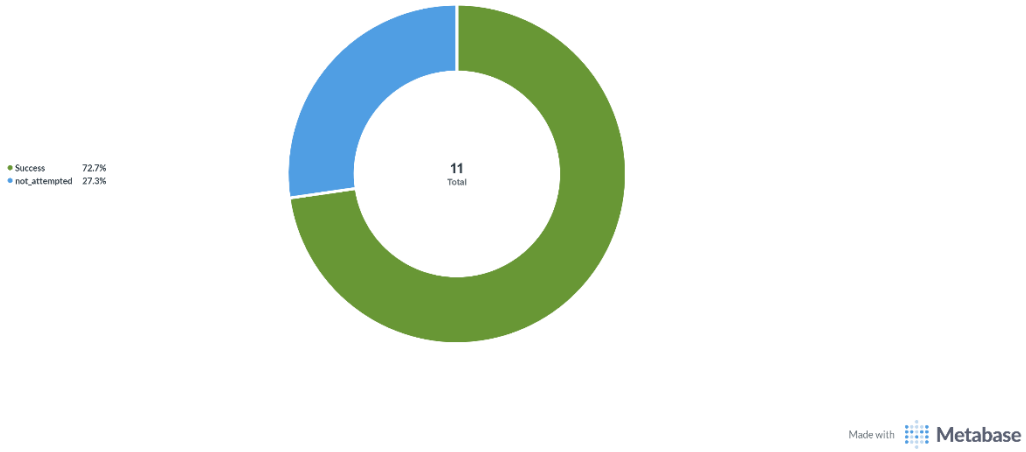


Figura 54 - Distribuição dos desfechos de remediação (remediationResult) ao longo dos 11 incidentes da coleção Enterprise, no Metabase (Fase 2).

Cruzando estes desfechos com a tipologia das avarias, a Figura 55 apresenta o gráfico circular para Análise de Causa-Raiz na coleção *Enterprise*. A distribuição das 11 ocorrências reparte-se por cinco tipologias.

Os estados *policy_disabled* e *rgwy_mismatch* surgem em maior destaque, com 27,27 % cada, correspondendo a 3 ocorrências por tipologia. O primeiro decorre do novo mecanismo de auditoria às regras de filtragem, ausente na **Fase 1**, que deteta e corrige desativações inadvertidas; o segundo resulta de desvios no endereço do *gateway* remoto. Seguem-se *phase2_mismatch* e *power_or_hardware_failure*, ambos com 18,18 % e 2 ocorrências: o primeiro corresponde a desalinhamentos nos seletores de tráfego, corrigidos por chamada PUT à CMDB; o segundo isola o conjunto de falhas que podem ocorrer no local que torna o equipamento inacessível

, detetados pelo *Pre-Check* ICMP. Por fim, *mpls_down* regista 9,09 %, com 1 falha de circuito sob responsabilidade do operador externo.

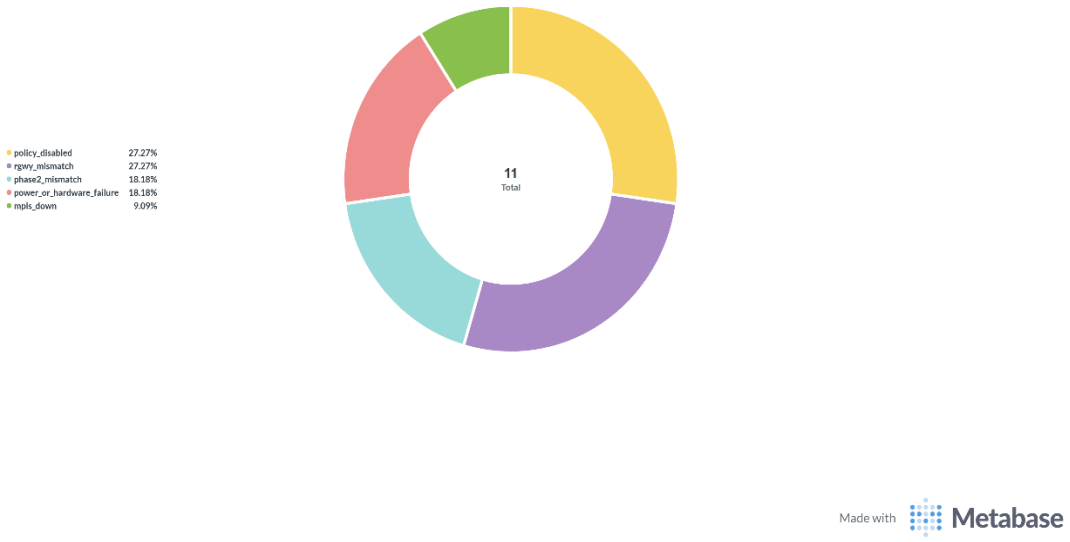
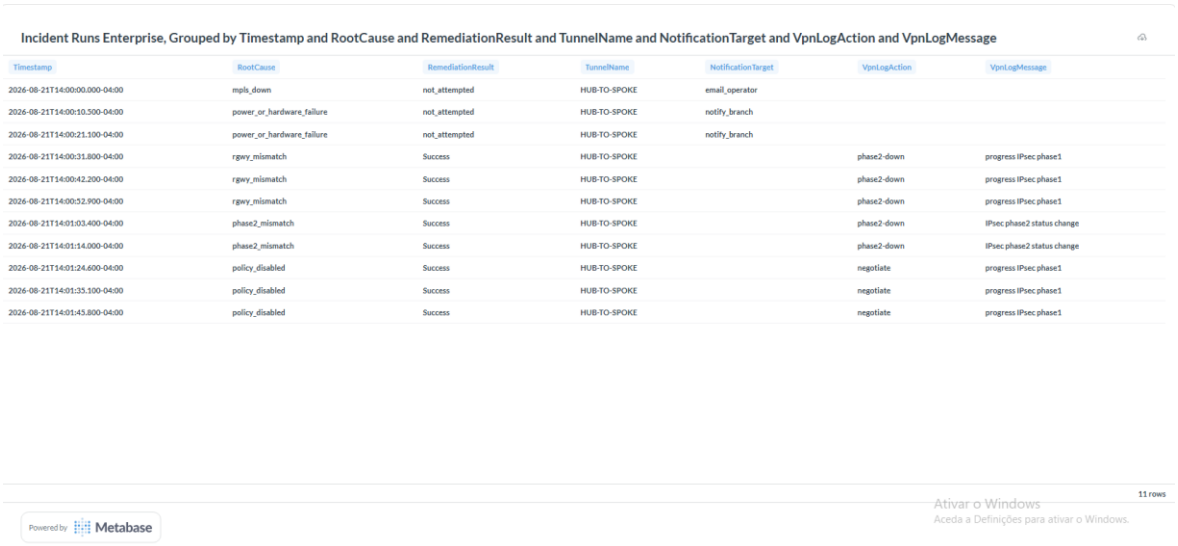


Figura 55 - Distribuição percentual das causas-raiz isoladas ao longo dos 11 incidentes da coleção Enterprise, no Metabase (Fase 2).

Numa perspetiva de auditoria, a extração de dados apresentada na **Figura 56** documenta o registo detalhado de cada avaria. As variáveis trabalhadas antes da inserção no MongoDB permitem ao Metabase apresentar, em colunas distintas, o timestamp do evento com precisão de milissegundos, a tipologia da falha, o desfecho da remediação, o destinatário da notificação e as sequências de texto extraídas dos registos do FortiOS — como *progress IPsec phase1* e *IPsec phase2 status change*. A tabela reúne os 11 incidentes da coleção *Enterprise* e permite acompanhar, registo a registo, a correspondência entre a causa-raiz isolada, a ação tomada e a evidência recolhida no equipamento.



Incident Runs Enterprise, Grouped by Timestamp and RootCause and RemediationResult and TunnelName and NotificationTarget and VpnLogAction and VpnLogMessage

Timestamp	RootCause	RemediationResult	TunnelName	NotificationTarget	VpnLogAction	VpnLogMessage
2026-08-21T14:00:00.000-04:00	mpls_down	not_attempted	HUB-TO-SPOKE	email_operator		
2026-08-21T14:00:10.500-04:00	power_or_hardware_failure	not_attempted	HUB-TO-SPOKE	notify_branch		
2026-08-21T14:00:21.100-04:00	power_or_hardware_failure	not_attempted	HUB-TO-SPOKE	notify_branch		
2026-08-21T14:00:31.800-04:00	rgwy_mismatch	Success	HUB-TO-SPOKE		phase2-down	progress IPsec phase1
2026-08-21T14:00:42.200-04:00	rgwy_mismatch	Success	HUB-TO-SPOKE		phase2-down	progress IPsec phase1
2026-08-21T14:00:52.900-04:00	rgwy_mismatch	Success	HUB-TO-SPOKE		phase2-down	progress IPsec phase1
2026-08-21T14:01:03.400-04:00	phase2_mismatch	Success	HUB-TO-SPOKE		phase2-down	IPsec phase2 status change
2026-08-21T14:01:14.000-04:00	phase2_mismatch	Success	HUB-TO-SPOKE		phase2-down	IPsec phase2 status change
2026-08-21T14:01:24.600-04:00	policy_disabled	Success	HUB-TO-SPOKE		negotiate	progress IPsec phase1
2026-08-21T14:01:35.100-04:00	policy_disabled	Success	HUB-TO-SPOKE		negotiate	progress IPsec phase1
2026-08-21T14:01:45.800-04:00	policy_disabled	Success	HUB-TO-SPOKE		negotiate	progress IPsec phase1

11 rows

Powered by Metabase

Ativar o Windows
Aceda a Definições para ativar o Windows.

Figura 56 - Tabela de auditoria no Metabase com o detalhe dos 11 incidentes da coleção Enterprise, evidenciando o cruzamento entre causa-raiz, desfecho da remediação e registos extraídos do FortiOS (Fase 2).

Em síntese, a camada de *Business Intelligence* cumpriu a função para que foi concebida: traduzir registos operacionais dispersos em indicadores legíveis por equipas de engenharia e de auditoria. Os painéis apresentados permitem responder, sem consulta direta à base de dados, a quatro perguntas distintas — que avarias ocorreram, com que gravidade, se a remediação foi autorizada e qual o seu desfecho.

A comparação entre as duas coleções evidencia a evolução entre fases. A coleção *Enterprise* regista uma proporção mais elevada de remediações bem-sucedidas e introduz uma tipologia de avaria que a **Fase 1** não detetava — as desativações inadvertidas de regras de filtragem —, além de acrescentar o cruzamento com os registos do FortiOS. Importa contudo sublinhar que os dois conjuntos têm composições distintas de causas-raiz.

A dimensão das amostras — 24 incidentes — permite validar o funcionamento do sistema e a coerência da instrumentação, mas não sustenta conclusões sobre a frequência relativa das avarias nem sobre taxas de eficácia esperadas em produção. Essa caracterização exigiria um período de observação prolongado sobre a infraestrutura real, tal como se discute na secção de limitações

4. Conclusão

O desfecho deste projeto de engenharia informática consolida um esforço focado na modernização operacional das arquiteturas de comunicações da ARTE, I.P. O **Projeto II** introduziu uma abordagem que concretizou a transição de um modelo de gestão puramente reativo para um plano de computação autónoma, assente nos pilares de NetDevOps e de Engenharia de Fiabilidade de Redes (SRE). O rumo metodológico aqui traçado alterou profundamente o cenário herdado da fase anterior. Se o âmbito do **Projeto I** se limitava ao diagnóstico passivo da infraestrutura, a nova implementação desenvolvida no **Projeto II** fixou um ciclo de autorremediação ativa (*Active Self-Healing*) e automação em ciclo fechado (*Closed-Loop Automation*). Esta evolução é sem dúvida uma mais valia para os canais de comunicação críticos da ARTE — que interligam serviços públicos estruturais, como as Lojas do Cidadão. Em última análise, a arquitetura reduz a dependência de intervenção externa, salvaguardando a continuidade de outros processos essenciais face a instabilidades de natureza física ou lógica.

4.1 Principais conclusões

A automação em ciclo fechado desenvolvida no Projeto II reduziu o MTTR da ordem das dezenas de minutos, no modelo manual, para a ordem dos segundos. Os registos do FortiOS documentam ciclos completos de deteção, correção e convergência entre 4 e 7 segundos nos ensaios da Fase 1, e a arquitetura Enterprise elimina ainda a latência de *polling* de 60 segundos ao substituir a consulta periódica por notificação via *webhook*. A **Equação 4.1 - Fórmula de Disponibilidade**.

$$A = \frac{MTBF}{MTBF + MTTR} \times 100 \qquad (4.1)$$

Estabelece a relação entre esta métrica e a disponibilidade do serviço: mantendo constante o MTBF, qualquer redução do MTTR aumenta a disponibilidade global. A quantificação desse ganho na infraestrutura da ARTE, I.P., exigiria a determinação do MTBF em produção, o que se situa fora do âmbito deste trabalho.

Esta solução assenta no paradigma de Infraestrutura como Código (IaC). Na **Fase 1**, a gestão das firewalls é feita por scripts Python sobre SSH, enquanto a consulta à SSoT recorre a pedidos HTTP à REST API do NetBox. O acesso às FortiGate é instanciado através da classe *ConnectHandler* da biblioteca Netmiko, com tratamento estruturado das exceções *NetmikoTimeoutException* e *NetmikoAuthenticationException* [6].

Esta abordagem permitiu limpar as associações de segurança do túnel IPsec através da instrução *diagnose vpn ike gateway clear HUB-TO-SPOKE*, executada em modo operacional da CLI, e reconfigurar parâmetros em modo de configuração. A separação entre os dois modos garantiu que a remediação não afetasse a configuração adjacente.

Na **Fase 2**, o licenciamento comercial cedido pela ARTE, I.P. permitiu substituir este mecanismo de contingência pela REST API do FortiOS. As correções passaram a ser aplicadas por chamadas PUT diretamente à árvore de configuração (CMDB) do equipamento [39], autenticadas por token com restrição de *trusted hosts*, o que eliminou as credenciais em texto claro usadas na fase anterior. A deteção deixou de depender de consulta periódica: o PRTG passou a notificar o orquestrador por webhook, suprimindo a janela de até 10 segundos do modelo anterior. A auditoria de conformidade alargou-se às regras de filtragem — uma classe de avaria que a **Fase 1** não detetava e que se revelou a mais frequente no conjunto de ensaios.

A **Tabela 8** organiza o paralelo operacional e as evoluções técnicas registadas entre o modelo do **Projeto I** e a presente implementação autorregenerativa.

Tabela 8 - Comparação das características técnicas e operacionais entre o Projeto I (diagnóstico passivo) e o Projeto II (automação ativa em ciclo fechado).

Métrica e Característica Técnica	Diagnóstico Passivo (Projeto I)	Automação Ativa em Ciclo Fechado (Projeto II)
Paradigma Operacional	Reativo e consultivo: deteta, classifica e notifica	Autónomo (Active Self-Healing proativo)
Ação sobre o equipamento	Nenhuma — apenas leitura (GET)	Escrita via PUT à CMDB (Fase 2) e CLI por Netmiko/SSH (Fase 1)
Tempo de deteção (MTTD)	Inferior a 10 s , tempo bastante aceitável	Equivalente, com notificação por *webhook* na Fase 2
Tempo de resolução (MTTR)	Inalterado: 15–30 min (casos simples) a 60 min (desvios de configuração)	4 a 7 s nos ciclos documentados nos *syslogs*
Causas-raiz isoladas	4: mpls_down, *Proxy-ID* vazio, *gateway* remoto inválido, tráfego a zero	5 na Fase 2, acrescentando policy_disabled
Modelo de deteção	Polling cíclico à API do PRTG	Polling na Fase 1; notificação por webhook na Fase 2
Autenticação nos equipamentos	Credenciais em texto claro nos scripts	Token de API com restrição de trusted hosts (Fase 2)

4.2 Análise crítica e limitações do ambiente simulado

A validação experimental de arquiteturas Autónomas em ambientes controlados por hipervisores, como o VMware Workstation Pro, constitui um passo obrigatório para mitigar riscos de engenharia, mas apresenta discrepâncias estruturais que importa acautelar face a uma rede de produção real de escala nacional. No laboratório de testes, a emulação das firewalls FortiGate e do plano de transporte

assenta na partilha de recursos de processamento central (CPU) do servidor anfitrião. Numa infraestrutura física corporativa, os equipamentos da Fortinet utilizam processadores proprietários de aplicação específica (ASICs), como as unidades de processamento de rede (NP) e de conteúdo (CP), que efetuam o descarregamento (offloading) do tráfego de criptografia IPsec diretamente em hardware. Esta divergência arquitetural significa que os contadores de desempenho e volumetria capturados via REST API no laboratório refletem o stress de CPU do hipervisor e não a eficiência real dos *buffers* de memória dos circuitos integrados que operam na Sede da ARTE, I.P.

Outra limitação crítica do ambiente simulado reside no determinismo absoluto e na ausência de ruído na camada de transporte (underlay). No ambiente virtual, a queda de um circuito MPLS ou um fator dos quais pode tornar uma SPOKE inacessível são emulados de forma binária através do desligamento instantâneo de uma interface lógica, provocando uma transição imediata de estado. No mundo real das telecomunicações que interligam as Lojas do Cidadão, os links de operadoras externas pouco sofrem apenas falhas totais e limpas. A infraestrutura enfrenta degradações parciais crónicas (brownouts), caracterizadas por variações dinâmicas de latência, perda intermitente de pacotes e rotação de rotas assimétricas. Estas perturbações flutuantes podem induzir o orquestrador n8n a falsos diagnósticos ou a loops infinitos de reconfiguração, exigindo algoritmos de mitigação de flapping substancialmente mais complexos do que os temporizadores fixos validados em ambiente laboratorial.

A par das limitações do ambiente, a própria árvore de diagnóstico apresenta limites de discriminação. O rótulo *power_or_hardware_failure* nomeia a causa mais provável, não uma causa apurada pelo sistema: o teste ICMP estabelece apenas que o equipamento está inacessível com o transporte confirmado operacional. Distinguir entre uma falha de alimentação, uma avaria de hardware e uma rotura de circuito ainda não refletida na monitorização exigiria telemetria externa à rede, nomeadamente a integração com os sistemas de alimentação ininterrupta das instalações. O mesmo se aplica ao estado *zero_traffic* da **Fase 1**, que agrega por construção todas as falhas do plano de dados não atribuíveis a desvio de configuração.

Por fim, o fator escala introduz desafios de concorrência e condições de corrida (race conditions) que a simulação de um único par *Hub-and-Spoke* é incapaz de replicar. O protótipo laboratorial provou a eficiência da automação em ciclo fechado para resolver os incidentes sequenciais acumulados no Metabase. Contudo, na topologia da ARTE, I.P., uma instabilidade generalizada num operador de telecomunicações pode provocar a queda simultânea de dezenas de túneis VPN IPsec. Perante este cenário de tempestade de alertas, as requisições HTTPS síncronas direcionadas ao n8n e à base de dados MongoDB sofreriam estrangulamento por contenção de recursos. O esgotamento das pools de conexões e a latência no processamento de *payloads* JSON aninhados poderiam arrastar o tempo de resposta para além dos limites estipulados nos SLAs

governamentais, expondo a necessidade futura de introduzir clusters distribuídos de alta disponibilidade para suportar a operação real.

4.3 Trabalho futuro

O encerramento do **Projeto II** abre caminho para um roteiro de evolução tecnológica recomendado para a equipa de comunicações residente na ARTE, I.P. Este planeamento incremental procura elevar o nível de maturidade de Engenharia de Fiabilidade de Redes (SRE) da instituição e acelerar a transição definitiva rumo ao novo paradigma **NetOps 2.0**, estruturando-se em quatro eixos fundamentais de desenvolvimento técnico:

Eixo 1: Consolidação de Políticas Declarativas Estritas e Validação Avançada na CMDB

Uma vez que a fase final do projeto introduziu as operações de escrita na CMDB via API REST nativa sob licenciamento empresarial, o primeiro vetor de evolução futura foca-se no robustecimento da segurança transacional dessas chamadas HTTPS. A arquitetura programática futura deve assegurar que as credenciais e os acessos de escrita com permissão de super-administrador passem a ser geridos com base em tokens dinâmicos de curta duração incluídos no cabeçalho HTTP de cada pedido, adotando o formato *Authorization: Bearer <API-TOKEN>.*

Em cenários que exijam autenticação baseada em sessão, o sistema tratará dinamicamente a validação de tokens de segurança *CSRF* através do cabeçalho *X-CSRFTOKEN* e da gestão dos respetivos cookies da sessão de autenticação, protegendo a API contra tentativas de falsificação de pedidos entre sites. Estas chamadas HTTP serão validadas e integradas de forma rígida contra o inventário de rede NetBox, que atua como Fonte Única de Verdade (SSoT) da instituição, expandindo o pilar de IPAM para validar a consistência de sub-redes e blocos CIDR antes de submeter as alterações, eliminando desvios configuracionais na produção.

Eixo 2: Integração Nativa com Plataformas de Helpdesk e Gestão de Serviços de TI (ITSM)

O segundo vetor estratégico propõe a junção do orquestrador n8n com as APIs de plataformas corporativas de gestão de serviços de TI (ITSM), como o Jira Service Management, ServiceNow ou Zammad. O objetivo principal reside na formalização e no controlo rigoroso de auditoria de cada evento de autocura desencadeado pelo ecossistema, garantindo que o registo das alterações na rede física permaneça transparente para as equipas humanas de suporte e monitorização.

Para mitigar o risco de abertura duplicada de incidentes em cenários onde múltiplas falhas de ligação geram alarmes em cascata, o plano futuro assenta na geração e correspondência de IDs de correlação únicos (Correlation ID). Logo após a interceção do alarme de quebra de conectividade, o n8n efetua uma consulta prévia na API do sistema de ITSM em busca de incidentes abertos que partilhem o

mesmo valor no campo *correlation_id* atribuído àquela filial específica. Se o incidente já se encontrar registado, o fluxo limita-se a adicionar notas operacionais ao ticket ativo; caso contrário, abre dinamicamente uma nova ficha de suporte técnico com os metadados contextuais em JSON colhidos do erro.

A integração incluirá regras para a supressão de loops infinitos de sincronização. Para evitar que uma atualização realizada pelo n8n despolete um evento de alteração no ITSM que reative o fluxo de automação, o sistema implementará validações de autoria no campo de histórico *sys_updated_by*, descartando eventos em que o autor coincida com a conta de serviço do próprio robô. A severidade e a prioridade do incidente serão calculadas dinamicamente com base nas tabelas de impacto e urgência de negócio associadas ao perfil da filial afetada. No final do processo, se o fluxo de autocura alcançar o sucesso operacional, o estado do ticket passa para fechado com o rótulo *auto_recovered*, anexando os metadados do MongoDB para fins de auditoria histórica.

Eixo 3: Expansão do Modelo Event-Driven para Telemetria Streaming de Alta Escala (gNMI)

Como terceiro vetor de melhoria contínua, recomenda-se evoluir o modelo de orquestração atual para uma arquitetura estritamente orientada a eventos (*Event-Driven*) dos equipamentos. Num primeiro momento, esta transição envolverá a implementação de *Automation Stitches* no FortiOS, transferindo o evento despoletador de monitorização (atualmente dependente do PRTG) para a própria firewall, garantindo a sinalização instantânea de incidentes lógicos locais. A longo prazo, e considerando uma infraestrutura de produção nacional com centenas de filiais, o roteiro tecnológico deverá culminar numa arquitetura de *streaming* de telemetria centralizada de alta escala baseada em gRPC (gRPC Network Management Interface - gNMI), lado a lado a um barramento de mensagens (como Apache Kafka ou RabbitMQ).

No plano prático futuro, os *stitches* a configurar no FortiOS associarão os eventos despoletadores (*triggers*) focados no registo de logs internos a ações de alerta imediato. Os engenheiros de rede mapearão os *triggers* de eventos específicos do FortiOS, tais como a perda de conectividade com o gateway de monitorização detetada pelo *link monitor* (logid 22922) ou a perda abrupta de sessões com vizinhos BGP (logid 20300), estabelecendo as diretivas técnicas necessárias no equipamento para despachar o webhook HTTPS com o payload JSON do log diretamente ao *listener* do n8n, conforme exemplificado na **Figura 57**.

```
config system automation-trigger
    edit "VPN_Down_Trigger"
        set event-type event-log
        set logid 22922 20300
    next
end
config system automation-action
    edit "n8n_Webhook_Action"
        set action-type webhook
        set protocol https
        set uri "https://n8n.arte.ip/v1/webhook/fortigate"
        set method post
        set headers "Content-Type:application/json"
    next
end
config system automation-stitch
    edit "Closed_Loop_Stitch"
        set trigger "VPN_Down_Trigger"
        set action "n8n_Webhook_Action"
    next
end
```

Figura 57 - Exemplo de configuração de um Automation Stitch na CLI do FortiOS para a deteção de eventos e envio nativo de webhooks para o orquestrador.

No exato milissegundo em que o hardware da firewall deteta o colapso do canal físico ou lógico, o *webhook* é disparado por canais *out-of-band* diretos ao orquestrador. O processo inicializa a triagem instantaneamente, reduzindo o tempo de interrupção a um valor impercetível para os utilizadores finais da ARTE, I.P.

Eixo 4: Implementação de um Ecossistema de ChatOps Bidirecional e Interativo

O quarto eixo de evolução retrata a transição das notificações de alerta unidirecionais atuais para um ecossistema bidirecional de *ChatOps*, integrado nativamente no Microsoft Teams — a plataforma de comunicação unificada padrão da instituição. Esta abordagem transforma o canal de conversação das equipas de engenharia de redes e SRE num terminal interativo de monitorização e intervenção em tempo real.

A infraestrutura lógica será gerida através de um sistema multiagente implementado no n8n, onde *workflows* independentes atuam como microsserviços especializados: um Agente Analisador de Incidentes para interpretar a severidade do alarme, um Agente de Inteligência de Logs para ler e filtrar características de depuração de mensagens de sistema e um Agente de Resolução focado nas tomadas de decisão curativas. Sempre que uma falha de grande impacto for sinalizada, o modelo publicará a informação no Microsoft Teams recorrendo a cartões dinâmicos (*Adaptive Cards*) [43].

Estes cartões apresentarão os detalhes analíticos da quebra de conectividade e disponibilizarão botões interativos diretamente na interface gráfica de chat. O *workflow* de execução do n8n tirará partido de nós de pausa e espera (*Wait*), suspendendo temporariamente a execução de remediações de alta criticidade enquanto aguarda pela receção de um sinal de retorno (*callback*) autenticado. Ao clicar no botão "Autorizar Remediação" no Microsoft Teams, o operador humano

valida o token de autorização, instruindo o n8n a desbloquear o nó de espera e a executar o script corretivo.

Desta forma, as equipas de comunicações poderão interagir em ambiente de chat com o *bot* para injetar comandos específicos de diagnóstico em tempo real (como a monitorização ao vivo de sessões IKE ou a verificação de tabelas de encaminhamento estático), centralizando as operações num único local de colaboração e acelerando a resposta operacional perante incidentes de grande impacto [43], [44].

Referências

[1] I. editor in Cheif, «Journal UGC-CARE IJCRT(ISSN: 2320-2882) | UGC Approved Journal | UGC Journal | UGC CARE Journal | UGC-CARE list, New UGC-CARE Reference List, UGC CARE Journals, ugc care list of journal, ugc care list, UGC Approved list, list of ugc approved journalInternational Peer Reviewed Journal and Refereed Journal, Follow ugc approved journal, UGC CARE, UGC CARE list, UGC CARE list of Journal, UGCCARE, care journal list, UGC-CARE list, New UGC-CARE Reference List, New ugc care journal list, Research Journal, Research Journal Publication, Research Paper, Low cost research journal, Free of cost paper publication in Research Journal, High impact factor journal, Journal, Research paper journal, UGC CARE journal, UGC CARE Journals, ugc care list of journal, ugc approved list, ugc approved list of journal, ugc approved journal, UGC CARE Journal, ugc approved list of journal, ugc care journal, UGC CARE list, UGC-CARE, care journal, UGC-CARE list, Journal publication, ISSN approved, Research journal, research paper, research paper publication, research journal publication, high impact factor, free publication, index journal, publish paper, publish Research paper, low cost publication, ugc approved journal, UGC CARE, ugc approved list of journal, ugc care journal, UGC CARE list, UGCCARE, care journal, UGC-CARE list, New UGC-CARE Reference List, UGC CARE Journals, ugc care list of journal, ugc care list 2020, ugc care approved journal, ugc care list 2020, new ugc approved journal in 2020, ugc care list 2021, ugc approved journal in 2021, Scopus, web of Science.», *IJCRT - International Journal of Creative Research Thoughts (IJCRT)*, Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.ijcrt.org/

[2] V. R. M. Lima, A. Callado, M. Bonfim, e E. Gonçalves, «Comparative Performance Analysis of IaC Tools for Deploying Containerized SDN Topologies in Cloud Environments», em *Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC)*, SBC, mai. 2026, pp. 463–476. doi: 10.5753/sbrc.2026.19975.

[3] M. Björklund, «The YANG 1.1 Data Modeling Language», Internet Engineering Task Force, Request for Comments RFC 7950, ago. 2016. doi: 10.17487/RFC7950.

[4] T. Graf, A. Elhassany, A. H. Feng, B. Claise, e P. Lucente, «YANG Message Keys for Message Broker Integration», Internet Engineering Task Force, Internet Draft draft-ietf-nmop-yang-message-broker-message-key-01, fev. 2026. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://datatracker.ietf.org/doc/draft-ietf-nmop-yang-message-broker-message-key

[5] R. Enns, M. Björklund, A. Bierman, e J. Schönwälder, «Network Configuration Protocol (NETCONF)», Internet Engineering Task Force, Request for Comments RFC 6241, jun. 2011. doi: 10.17487/RFC6241.

[6] A. Bierman, M. Björklund, e K. Watsen, «RESTCONF Protocol», Internet Engineering Task Force, Request for Comments RFC 8040, jan. 2017. doi: 10.17487/RFC8040.

[7] «RESTCONF API | Guides and References | Cisco Crosswork NSO Documentation». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://nso-docs.cisco.com/guides/development/core-concepts/northbound-apis/restconf-api

[8] «MongoDB Monitoring Tool | SolarWinds». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.solarwinds.com/solarwinds-observability/integrations/mongodb-monitoring
[9] bobranks, *b0br4nks/silent-security-drift*. (15 de janeiro de 2026). TeX. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://github.com/b0br4nks/silent-security-drift
[10] School of Electrical Engineering and Informatics, Bandung Institute of Technology, E. Mulyana, G. Fakih, e School of Electrical Engineering and Informatics, Bandung Institute of Technology, «Network Automation with a Single Source of Truth in a Heterogeneous Environment», *ijeei*, vol. 14, n.º 1, pp. 92–100, mar. 2022, doi: 10.15676/ijeei.2022.14.1.6.
[11] «Network engineers take on NetDevOps roles to advance stalled automation efforts», Network World. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.networkworld.com/article/4123790/network-engineers-take-on-netdevops-roles-to-advance-stalled-automation-efforts.html
[12] B. Lee, «How I Built a Self-Healing Home Lab That Fixes Itself», Virtualization Howto. Acedido: 13 de junho de 2026. [Em linha]. Disponível em: https://www.virtualizationhowto.com/2025/10/how-i-built-a-self-healing-home-lab-that-fixes-itself/
[13] M. H. Behringer *et al.*, «Autonomic Networking: Definitions and Design Goals», Internet Engineering Task Force, Request for Comments RFC 7575, jun. 2015. doi: 10.17487/RFC7575.
[14] «Technical Tip: How to flush a VPN tunnel | Community». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://community.fortinet.com/fortigate-3/technical-tip-how-to-flush-a-vpn-tunnel-98315
[15] «The Three Levels of Self-Healing Networks: From Auto-Detection to True Autonomy - NetBrain». Acedido: 13 de junho de 2026. [Em linha]. Disponível em: https://www.netbrain.com/blog/self-healing-networks/
[16] «Ciclo PDCA», *Wikipédia, a enciclopédia livre*. 13 de fevereiro de 2026. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://pt.wikipedia.org/w/index.php?title=Ciclo_PDCA&oldid=71725366
[17] H. Shefler, «Netops 2.0: The Next Generation of Network Operations», NetOp AI. Acedido: 30 de julho de 2026. [Em linha]. Disponível em: https://netop.ai/blog/netops-2-0-the-next-generation-of-network-operations/
[18] «(PDF) Intelligent Automation Framework for Multi-Vendor Network Infrastructure Using AI and Open-Source Tools», *ResearchGate*, jan. 2026, doi: 10.46431/mejast.2025.8410.
[19] «Network Management Operations (nmop)». Acedido: 15 de julho de 2026. [Em linha]. Disponível em: https://datatracker.ietf.org/wg/nmop/about/
[20] «IT Automation Solutions | Streamline DevOps & CI/CD». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://intelligentvisibility.com/automation
[21] «Why Network Automation is Critical in IT». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://blog.cloudmylab.com/network-automation-critical-it
[22] «netmiko.fortinet.fortinet_ssh API documentation». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://ktbyers.github.io/netmiko/docs/netmiko/fortinet/fortinet_ssh.html
[23] «FortiOS REST API | SecGW for Mobile Networks Deployment». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://docs.fortinet.com/document/fortigate/7.2.0/secgw-for-mobile-networks-

deployment/238243/docs.fortinet.com/document/fortigate/7.2.0/secgw-for-mobile-networks-deployment/238243/fortios-rest-api
[24] «Webhook node documentation | n8n Docs». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/
[25] «Permanent trial mode for FortiGate-VM | Administration Guide». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://docs.fortinet.com/document/fortigate/8.0.0/administration-guide/441460/docs.fortinet.com/document/fortigate/8.0.0/administration-guide/441460/permanent-trial-mode-for-fortigate-vm
[26] Sergio, «Getting Started n8n with DockerCompose», Medium. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://medium.com/@learning.by.playing.2023/getting-started-n8n-with-dockercompose-eb602aaed5c0
[27] «Burned by PEP 668 on Ubuntu 24.04 — How I Fixed the Monthly Automation Script», DEV Community. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://dev.to/linou518/burned-by-pep-668-on-ubuntu-2404-how-i-fixed-the-monthly-automation-script-59eg
[28] «PEP 668 : r/wsl2». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.reddit.com/r/wsl2/comments/1hro9r6/pep_668/
[29] «cannot install on Ubuntu 24..04 · Issue #1533 · nvbn/thefuck», GitHub. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://github.com/nvbn/thefuck/issues/1533
[30] «Python package installation in Ubuntu 24.04 AEE | DNAnexus Documentation». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://documentation.dnanexus.com/developer/apps/dependency-management/python-package-installation-in-ubuntu-24-04-aee
[31] «Using Netmiko with Fortigate : r/networking». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.reddit.com/r/networking/comments/tqk0vb/using_netmiko_with_fortigate/
[32] «ignio AIOps Platform for Autonomous IT Operations - Digitate». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://digitate.com/products/ignio-aiops/
[33] «n8n Learning Journey #6: Webhook Trigger - The Real-Time Responder That Powers Instant Event Automation : r/n8n». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.reddit.com/r/n8n/comments/1n93m7z/n8n_learning_journey_6_webhook_trigger_the/
[34] S. M. Expert, «How to Execute FortiGate REST API Requests? | Our Blog». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.insoftconsultancy.com/blog/how-to-execute-fortigate-rest-api-requests/
[35] «Running a python script in n8n - Built with n8n - n8n Community». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://community.n8n.io/t/running-a-python-script-in-n8n/89289?tl=zh_TW
[36] «Python package installation in Ubuntu 24.04 AEE | DNAnexus Documentation». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://documentation.dnanexus.com/developer/apps/dependency-management/python-package-installation-in-ubuntu-24-04-aee

[37] secritservice, «IPSEC VPN tunnel stays up when your parent WAN interface goes down», r/fortinet. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.reddit.com/r/fortinet/comments/1ivr91y/ipsec_vpn_tunnel_stays_up_when_your_parent_wan/
[38] «MongoDB node documentation | n8n Docs». Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.mongodb/
[39] L. Kaklauskas e R. Pilypas, «SOLUTION OF REMOTE MANAGEMENT FOR COMPUTER NETWORK INFRASTRUCTURE USING INTEGRATED NETBOX», *PROFESSIONAL STUDIES: THEORY AND PRACTICE*, vol. 29, pp. 4–12, mar. 2025, doi: 10.56131/pstp.2025.29.1.342.
[40] «IPsec VPN Troubleshooting in Fortigate firewall», Techclick. Acedido: 11 de junho de 2026. [Em linha]. Disponível em: https://www.techclick.in/ipsec-vpn-troubleshooting-in-fortigate-firewall-1
[41] J. Rice-Jones, «I built a self-healing home lab with n8n», XDA. Acedido: 13 de junho de 2026. [Em linha]. Disponível em: https://www.xda-developers.com/build-self-healing-home-lab-with-n8n/
[42] «Understand Close Loop Automation in Cloud Based Software Defined Network - Cisco». Acedido: 13 de junho de 2026. [Em linha]. Disponível em: https://www.cisco.com/c/en/us/support/docs/cloud-systems-management/crosswork-network-controller/221798-understand-close-loop-automation-in-clou.html
[43] «I built a fully automated AI DevOps system using n8n — and turned it into a multi-agent incident response platform - Built with n8n», n8n Community. Acedido: 13 de junho de 2026. [Em linha]. Disponível em: https://community.n8n.io/t/i-built-a-fully-automated-ai-devops-system-using-n8n-and-turned-it-into-a-multi-agent-incident-response-platform/287644
[44] Hemanthrajg, «Smart Incident Automation with ServiceNow + n8n», Medium. Acedido: 13 de junho de 2026. [Em linha]. Disponível em: https://medium.com/@hemanthrajg/smart-incident-automation-with-servicenow-n8n-87da68eb524d